



قسم هندسة البرمجيات ونظم المعلومات الذكية

VitaMate تطبيق جوال ذكي ومتكامل لتتبع الصحة ونمط الحياة

مقدم استكمالاً لمتطلبات الحصول على درجة الإجازة في هندسة البرمجيات ونظم المعلومات الذكية

إعداد:

سلام محمد العياش

آمنة أيمن زيتون

إشراف:

د. كادان الجمعة

م. رغد الحسني

العام الدراسي 2025-2026

دمشق - سوريا



Faculty of Artificial Intelligence Engineering
Department of Software Engineering and Intelligent Information Systems

VitaMate: An Intelligent Mobile Application for Integrated Health and Lifestyle Tracking

Senior Project 2 Report

Submitted in partial fulfillment of the requirements for the Bachelor's Degree in Software Engineering and Intelligent Information Systems

Prepared by

Salam Mohammed Al-Ayash

Amenah Ayman Zaitoun

Supervised by

Dr. Kadan Jumaa

Eng. Raghad Al-Hossny

Academic Year 2025–2026

Damascus, Syria

Dedication

To our families, whose unwavering support, patience, and encouragement accompanied us throughout our academic journey.

To our instructors and everyone who contributed to our learning and growth.

This work is dedicated with sincere gratitude and appreciation.

Acknowledgements

We would like to express our sincere gratitude and appreciation to our supervisors, Dr. Kadan Jumaa and Eng. Raghad Al-Hossny, for their valuable guidance, continuous support, constructive feedback, and academic supervision throughout the development of the VitaMate project.

Their expertise and recommendations played an essential role in directing the project, addressing its academic and technical challenges, and improving the quality of its analysis, design, implementation, testing, and documentation.

We also extend our sincere appreciation to the Faculty of Artificial Intelligence Engineering and the Department of Software Engineering and Intelligent Information Systems at the Syrian Private University for providing the academic knowledge, technical foundation, and learning environment that supported us throughout our years of study.

We would also like to thank all instructors and academic staff members who contributed to our learning and helped us develop the knowledge and skills required to complete this project.

Finally, we express our deepest gratitude to our families for their patience, encouragement, understanding, and continuous support throughout our academic journey and during the completion of this project.

Contents

Dedication	3
Acknowledgements	4
List of Abbreviations and Scientific Term.....	13
الملخص	14
Abstract	15
1.1 Introduction	17
1.2. Project Overview and Background	17
1.3. Problem Statement and Its Practical and Academic Importance	17
1.4 Project Objectives	18
1.4.1 General Objective	18
The general objective of VitaMate is to design, implement, and evaluate an intelligent mobile application that integrates multiple health and lifestyle-tracking services within one coordinated platform. The system supports daily self-monitoring through tracking, reminders, progress indicators, reports, and selected intelligent services.	18
1.4.2 Specific Objectives	18
1.5 Project Scope, Constraints, and Assumptions.....	19
1.5.1 In-Scope	19
1.5.2 Out-of-Scope.....	19
1.5.3 Project Constraints	19
1.5.4 Project Assumptions	20
1.6 Project Contribution.....	20
1.7 Research and Development Methodology	20
1.8 Report Organization.....	21
1.9 Key Definitions and Concepts	21
1.10 Chapter Summary	22
Chapter 2: Development Methodology and Project Management	23
2.2 Completed Baseline from Senior Project 1	24
2.2.1 Requirements Engineering and Analysis	24
2.2.2 Project Planning	24
2.2.3 Preliminary System Design.....	25
2.3 Senior Project 2 Development Scope	26

2.4 Selected Development Methodology	27
2.4.1 Agile Scrum	27
2.4.2 Rationale for Selecting Agile Scrum	28
2.4.3 Adaptation to the Project Team	28
2.4.4 Definition of Done	29
2.4.5 Sprint Review and Retrospective	30
2.5 Project Lifecycle	30
2.6 Project Schedule and Gantt Chart	31
2.8 Team Roles and Responsibilities	33
2.9 Quality Management	33
2.10 Project Management and Collaboration Tools	34
2.12 Software Configuration Management	36
2.13 Chapter Summary	36
Chapter 3	37
Theoretical Framework and Related Work	37
3.1 Introduction	38
3.2 Digital Health and Mobile Health	38
3.3 Integrated Lifestyle Self-Monitoring	39
3.4 Behavior-Change Support	39
3.5 Personalized and Constraints-Aware Coordination	40
3.6 AI-Assisted Food Analysis	41
3.7 Data Quality, Privacy, and Health Safety	41
3.8 Review of Related Studies	42
3.9 Comparative Analysis of Existing Applications	44
3.10 Identified Research and System Gap	45
3.11 Current Technology Trends	46
3.11.1 Personalized Digital Interventions	46
3.11.2 Connected Multi-Domain Health Data	46
3.11.3 AI-Assisted User Interaction	46
3.11.4 User-in-the-Loop Validation	47
3.11.5 Engagement-Oriented Design	47
3.11.6 Privacy and Responsible Health Communication	47

3.12 Chapter Summary	47
4.1 Introduction.....	49
4.2 Feasibility Study	49
4.2.1 Technical Feasibility	49
4.2.2 Economic Feasibility	50
4.2.3 Operational Feasibility	51
4.3 Requirements Elicitation and Sources	51
4.4 Stakeholders and Needs Analysis	53
4.5 Functional Requirements	54
4.5.1 Functional Requirements Classification	55
4.6 Non-Functional Requirements	60
4.7 Requirements Prioritization Using MoSCoW	63
4.8 UML Requirements Models	64
4.8.1 Main Use Cases.....	64
4.8.2 Overall Use Case Diagram.....	65
4.8.3 Use Case Specifications	67
4.8.4 Activity Diagrams	73
4.8.5 Sequence Diagrams.....	78
4.9 Agile Requirements Models	81
4.9.1 User Stories.....	81
4.9.2 Story Mapping	82
4.9.3 Product Backlog.....	83
4.9.4 Sprint Backlog	84
4.10 Requirements Analysis and Documentation	85
4.10.1 Requirements Traceability Matrix	85
4.10.2 Initial Test Cases.....	90
Chapter 5: Architectural and Detailed Design	92
Chapter Introduction	93
5.1 High-Level System Design	93
5.2 Architectural Pattern and Rationale	94
5.3 Database Design.....	95
5.3.1 Entity–Relationship Design	95

5.4 External Interface and API Design	99
5.4.1 API Architecture and Communication Flow:	100
5.4.2 General API Design Conventions	100
5.4.3 Main Backend API Groups	101
5.4.4 AI Food-Analysis Interface.....	102
5.4.5 Request Validation and Error Handling.....	103
5.4.6 OpenAPI and API Documentation.....	104
5.5 Security, Authentication, and Authorization Design	104
5.5.1 Authentication Design	104
5.5.2 Authorization and User Roles.....	105
5.5.3 Data Protection and Secure Communication	105
5.5.4 OWASP-Related Protection Considerations.....	105
5.6 User Interface and User Experience Design	106
5.7 Detailed Module Design	106
5.8 Intelligent Service Design.....	108
5.8.1 Input and Validation Design	108
5.8.2 Provisional Analysis and User Review.....	108
5.8.3 Nutrition Finalization Design	109
5.9 Chapter Summary	109
Chapter 6: Development Environment and Software Implementation.....	110
6.1 Senior Project 1 Implementation Baseline.....	111
6.2 Development Tools and Libraries.....	111
6.3 Development Environment and Project Structure	112
6.3.1 Project Organization	112
6.4 Programming Languages and Frameworks	113
6.5 Database Server and Runtime Environment.....	114
6.6 Dependencies and Configuration Files	114
6.7 Source Control and Repository Management	115
6.8 Continuous Integration and Continuous Delivery	116
6.9 Test Environment.....	116
6.10 Software Implementation.....	117
6.10.1 Project and Folder Structure	117

6.10.2 Backend and Business-Logic Implementation.....	119
6.10.3 Frontend and User Interface Implementation	120
6.10.4 API Implementation.....	129
6.10.6 AI Food-Analysis Implementation	132
6.10.7 Senior Project 2 Integration and Refinement.....	134
6.11 Implementation Challenges and Solutions.....	135
6.12 Chapter Summary	136
Chapter 7: Testing and Evaluation.....	137
7.1 Test Plan.....	138
7.2 Test Scope.....	138
7.3 Test Strategy and Types.....	139
7.4 Test Environment.....	140
7.5 Test Cases and Execution Results.....	141
7.6 Backend Testing.....	141
7.7 Frontend Testing	142
7.8 Integration and End-to-End Testing.....	143
7.9 Performance and Load Testing	143
7.10 Security and Privacy Testing	144
7.11 Usability and User Evaluation	145
7.12 Quality Metrics	145
7.13 Acceptance Criteria and Remaining Validation Items.....	146
Additional Validation Items.....	146
7.14 Chapter Summary	147
Chapter 8: Results and Analysis	148
8.1 Final System Results.....	149
8.2 Comparative Analysis with Existing Systems	150
8.3 Achievement of Project Objectives	151
8.4 Key Performance Indicators (KPIs).....	152
8.5 AI Food-Analysis Results	152
8.6 Strengths and Weaknesses	153
8.7 Limitations of the Final Version	154
8.8 Chapter Summary	155

Conclusion	156
References	157

List of table

Table 1 :List of Abbreviations and Scientific Terms	13
Table 2 :Completed Outcomes of Senior Project 1	26
Table 3 :Adapted Scrum Activities	29
Table 4 :Definition Done of	29
Table 5 :VitaMate Development Lifecycle	30
Table 6 :Team Roles and Responsibilities	32
Table 7 :Senior Project 2 Risk Register	33
Table 8 :Main Database Domains and Entities	98
Table 9 General API Design Conventions	101
Table 10: Main VitaMate API Groups	102
Table 11:Principal API Response Categories	104
Table 12:Main VitaMate Modules and Responsibilities	107
Table 13:AI Food-Analysis Design Responsibilities	109
Table 14:Main Development Tools and Libraries	112
Table 15 :Development and Runtime Environments	112
Table 16 :Main Languages and Frameworks	114
Table 17 :Database and Runtime Configuration	114
Table 18:Main Dependency and Configuration Files	115
Table 19:Source-Control Configuration	116
Table 20:Continuous-Integration Jobs	116
Table 21:Test Environments and Tools	117
Table 22 :Simplified VitaMate Project Structure	118
Table 23 :Main Backend Modules and Responsibilities	119
Table 24 :Main Frontend Modules and Interfaces	121
Table 25 :Representative AI-Meal Analysis Endpoint	131
Table 26 :Notification-Flow Responsibilities	132
Table 27 :Main AI Pipeline Components	133
Table 28 :Main Senior Project 2 Additions and Refinements	135

Table 29 : Main Implementation Challenges and Applied Solutions	135
Table 30 :Test Plan Summary	138
Table 31 :Included and Excluded Test Scope	139
Table 32 :Executed Test Types	140
Table 33 :Test Environment.....	141
Table 34 :Representative Test Cases and Results	141
Table 35 : Backend Test Summary	142
Table 36 :Frontend Test Summary.....	143
Table 37: Integration Test Status	143
Table 38 :Verified Performance Results	144
Table 39 :Security and Privacy Test Summary.....	144
Table 40 :Verified Quality Metrics	145
Table 41 :Acceptance Status	146
Table 42 :Final Implemented System Capabilities	149
Table 43 :Comparative Analysis of VitaMate and Existing Applications.....	151
Table 44 :Achievement of Project Objectives	152
Table 45 :summarizes the verified final performance and testing indicators of VitaMate.....	152
Table 46 :Final AI Food-Analysis Results.....	153
Table 47 :Main Strengths and Weaknesses.....	154
Table 48 :Final-Version Limitations.....	155

List of figures

Figure 1 :Gantt Chart for Senior Project 2	31
Figure 2:Summary of Related Studies	43
Figure 3: Comparison of Existing Health Applications.....	45
Figure 4: Technical Feasibility Factors.....	50
Figure 5: Requirements Elicitation Methods and Sources.....	53
Figure 6: Requirements Sources and Refinement Activities	54
Figure 7: Functional Requirements of the VitaMate System.....	60
Figure 8: Non-Functional Requirements of the VitaMate System	63
Figure 9 : MoSCoW Prioritization of VitaMate Requirements	64
Figure 10:Main Use Cases of the VitaMate System.....	65

Figure 11: High-Level Use Case Diagram.....	66
Figure 12: Use Case Specification for UC-01 — Manage Account and Health Profile.....	68
Figure 13: Use Case Specification for UC-02 — Manage Daily Health Tracking	69
Figure 14: Use Case Specification for UC-03 — Manage Nutrition and AI-Assisted Meals	70
Figure 15: Use Case Specification for UC-04 — Manage Medications and Chronic Conditions	70
Figure 16: Use Case Specification for UC-05 — Manage Goals and Unhealthy Habits	71
Figure 17: Use Case Specification for UC-07 — View Reports and Supportive Guidance.....	73
Figure 18: Activity Diagram for AI-Assisted Meal Analysis.....	75
Figure 19: Activity Diagram for Unhealthy-Habit Cessation.....	76
Figure 20: Activity Diagram for Health-Tracker Recording and Motivation-State Update.....	77
Figure 21: Sequence Diagram for AI-Assisted Meal Analysis.....	78
Figure 22 :Sequence Diagram for Health-Tracker and Motivation Update.....	79
Figure 23 :Sequence Diagram for Unhealthy-Habit Cessation and Motivation Update.....	80
Figure 24: Representative User Stories.....	82
Figure 25: VitaMate User Story Map	83
Figure 26: VitaMate Product Backlog	84
Figure 27: VitaMate Sprint Backlog.....	85
Figure 28: Functional Requirements Traceability Matrix	88
Figure 29: Non-Functional Requirements Traceability Matrix	89
Figure 30: Initial Test Cases	91
Figure 31 1presents the final high-level architecture of VitaMate	94
Figure 32: VitaMate Login Interface	122
Figure 33 :VitaMate Home and Daily Health Overview	123
Figure 34: Hydration Tracking Interface	124
Figure 35 :Nutrition Tracking Interface.....	125
Figure 36 :Activity and Steps Interface	126
Figure 37: Unhealthy-Habit Management Interface	127
Figure 38 :Medication Management Interface.....	128
Figure 39 :Profile and Self-Management Interface.....	129
Figure 40 :Notification Planning and Delivery Flow.....	132
Figure 41 :AI Food-Analysis Flow	133

List of Abbreviations and Scientific Term

Table 1 :List of Abbreviations and Scientific Terms

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
DRF	Django REST Framework
ERD	Entity–Relationship Diagram
JWT	JSON Web Token
ML	Machine Learning
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

المخلص

يشهد مجال الصحة الرقمية توسعًا متزايدًا في استخدام تطبيقات الهاتف لدعم المتابعة اليومية وتحسين الوعي بالعادات الصحية. ومع ذلك، تركز كثير من التطبيقات الحالية على جانب واحد مثل التغذية أو النشاط أو النوم، مما يؤدي إلى تشتت البيانات وضعف الربط بين السلوكيات الصحية المختلفة. يهدف مشروع **VitaMate** إلى تصميم وتنفيذ وتقييم تطبيق ذكي ومتكامل لمتابعة نمط الحياة والصحة الشخصية ضمن منصة موحدة تجمع التغذية، وشرب الماء، والنشاط البدني، والنوم، والأدوية، والعادات غير الصحية، والأهداف، والحالات المزمنة، والتنبيهات، والتقارير، وتحليل صور الطعام بمساعدة الذكاء الاصطناعي.

اعتمد تطوير المشروع على منهجية هندسة برمجيات تطبيقية باستخدام **Agile Scrum**، وشمل تحليل المتطلبات، ونمذجة النظام، وتصميم المعمارية وقاعدة البيانات، وتنفيذ تطبيق الهاتف باستخدام **Flutter**، وتطوير الخدمات الخلفية باستخدام **Django REST Framework**، وربط الوحدات عبر واجهات **REST** وقاعدة بيانات مركزية. كما استُخدمت طبقة قيود مركزية لتنسيق الأهداف والقيود الصحية وسلوك المتنبعات، مع الحفاظ على مراجعة المستخدم للنتائج الذكية غير المؤكدة.

أظهرت النتائج اكتمال الوظائف الأساسية وتكامل الواجهة الأمامية والخدمات الخلفية بصورة مستقرة. اجتاز النظام الخلفي 240 اختبارًا آليًا، كما اجتازت اختبارات **Flutter** بالكامل بنسبة تغطية سطرية بلغت 53.26%. وأنجز خط تحليل الطعام جميع صور مجموعة التقييم البالغ عددها 35 صورة دون فشل تشغيلي، بمتوسط زمن تحليل قدره 6.81 ثانية، ومتوسط تقاطع على اتحاد للتجزئة بلغ 90.74%، ونسبة نجاح في ربط النتائج النهائية بلغت 91.43%.

خلص المشروع إلى أن **VitaMate** يقدم نموذجًا عمليًا ومتكاملًا لدعم المراقبة الذاتية وتنظيم السلوكيات الصحية اليومية. ومع ذلك، ما تزال بعض مخرجات الذكاء الاصطناعي بحاجة إلى تأكيد المستخدم ومزيد من التحقق. ولا يُعد النظام أداة للتشخيص الطبي أو بديلًا عن المختصين الصحيين.

الكلمات المفتاحية: الصحة الرقمية، تطبيقات الصحة المتنقلة، تتبع نمط الحياة، طبقة القيود، تحليل صور الطعام، الذكاء الاصطناعي، أجايل سكرم.

Abstract

Digital health has increasingly adopted mobile applications to support daily self-monitoring and improve awareness of health-related behaviors. However, many existing applications focus on a single area, such as nutrition, activity, or sleep, which results in fragmented data and limited coordination between related lifestyle factors. VitaMate aims to design, implement, and evaluate an intelligent mobile application that integrates nutrition, hydration, physical activity, sleep, medications, unhealthy habits, personal goals, chronic-condition support, notifications, reports, and AI-assisted food-image analysis within one unified platform.

The project was developed using an applied software engineering methodology based on Agile Scrum. The work included requirements analysis, system modeling, architectural and database design, mobile implementation using Flutter, backend development using Django REST Framework, and module integration through REST APIs and a centralized database. A centralized Constraints Layer was also used to coordinate health goals, restrictions, and tracker behavior while preserving user review of uncertain intelligent outputs.

The final results demonstrated successful implementation and stable integration of the main system functions. The backend passed 240 automated tests, while all Flutter tests passed with 53.26% line coverage. The AI food-analysis pipeline completed all 35 images in the evaluation set without runtime failure, achieved an average analysis time of 6.81 seconds, a mean segmentation IoU of 90.74%, and a finalize-mapping success rate of 91.43%.

The project concludes that VitaMate provides a practical and integrated approach to lifestyle self-monitoring and daily health management. However, some AI-generated results still require user confirmation and further validation. VitaMate is intended as a lifestyle-support tool and not as a medical diagnostic system or a replacement for healthcare professionals.

Keywords: Digital Health, Mobile Health Application, Lifestyle Tracking, Constraints Layer, Food-Image Analysis, Artificial Intelligence, Agile Scrum.

Chapter 1: Introduction

1.1 Introduction

Digital technologies have transformed personal health and lifestyle tracking. However, many mobile health applications focus on isolated areas such as nutrition, activity, hydration, or sleep, which limits the user's ability to view related health behaviors within one system.

VitaMate addresses this limitation through an intelligent mobile application that integrates nutrition, hydration, activity, sleep, medications, habits, goals, chronic-condition support, reminders, reports, and AI-assisted food analysis.

The project was developed across two academic stages. Senior Project 1 established the main requirements, design, frontend, backend, and prototype foundations, while Senior Project 2 focused on integration, refinement, testing, and final evaluation.

This chapter presents the project background, problem, objectives, scope, constraints, contribution, methodology, and report structure.

1.2. Project Overview and Background

Digital health technologies support personal health awareness by enabling users to record daily data, review progress, and receive reminders. However, lifestyle factors such as nutrition, activity, sleep, medications, and chronic conditions are interconnected, while many existing applications manage them separately.

VitaMate addresses this fragmentation through a unified mobile application that combines health profiles, nutrition, hydration, activity, sleep, medications, habits, goals, dashboards, and reports. The system also coordinates user data, health goals, chronic conditions, and recorded behaviors to provide more consistent reminders, restrictions, and supportive guidance.

VitaMate is intended for self-monitoring and lifestyle support only. It does not provide medical diagnosis or replace healthcare professionals.

Senior Project 1 established the Flutter frontend, Django REST backend, modular health services, and initial prototype. Senior Project 2 focused on completing integration, refining intelligent services and interfaces, expanding testing, and evaluating the final system.

1.3. Problem Statement and Its Practical and Academic Importance

Health and lifestyle factors such as nutrition, hydration, activity, sleep, medications, habits, and chronic conditions are closely related. However, many existing applications manage these areas separately, resulting in fragmented information, increased user effort, and inconsistent follow-up.

The main problem addressed by VitaMate is the lack of a unified system that coordinates multiple health-tracking domains and reflects their relationships. This limitation becomes more significant when chronic conditions, medication schedules, personal goals, and daily behaviors influence one another.

VitaMate therefore combines tracking, reminders, goals, progress indicators, reports, and motivational feedback within one coordinated platform.

1. Practical Significance

VitaMate reduces the need for multiple separate applications by enabling users to organize daily health data, review progress, manage reminders and medications, and identify areas requiring attention within one system.

2. Academic and Technical Significance

The project applies software engineering principles through requirements analysis, Agile Scrum, UML modeling, architectural and database design, frontend and backend implementation, traceability, and structured testing.

Its main technical contribution is the use of a centralized Constraints Layer that coordinates user goals, chronic-condition restrictions, and tracker behavior across the system.

1.4 Project Objectives

1.4.1 General Objective

The general objective of VitaMate is to design, implement, and evaluate an intelligent mobile application that integrates multiple health and lifestyle-tracking services within one coordinated platform. The system supports daily self-monitoring through tracking, reminders, progress indicators, reports, and selected intelligent services.

1.4.2 Specific Objectives

The specific objectives of the project are to:

1. Develop secure user-account and health-profile management for personalized tracking.
2. Integrate nutrition, hydration, activity, sleep, medications, habits, and personal goals within one application.
3. Enable meal recording and monitoring of calories, macronutrients, and selected micronutrients.
4. Support water and beverage tracking according to daily hydration targets.
5. Monitor workouts, steps, active time, and related activity indicators.
6. Enable sleep recording and progress monitoring toward personal sleep goals.
7. Support medication schedules, reminders, adherence records, and medication history.
8. Consider selected chronic conditions and health restrictions when processing tracker data, goals, alerts, and supportive guidance.
9. Coordinate health modules through a centralized Constraints Layer to reduce inconsistent system behavior.
10. Provide AI-assisted food-image analysis and nutrition support with user review and confirmation.

11. Generate dashboards, notifications, motivational feedback, and periodic reports from recorded data.
12. Evaluate the system through documented functional, integration, performance, usability, and quality-related testing.

These objectives are evaluated through implemented functions, requirements traceability, test results, integration verification, performance measurements, usability evaluation, and intelligent-service analysis within the approved project scope.

1.5 Project Scope, Constraints, and Assumptions

1.5.1 In-Scope

VitaMate is a mobile health and lifestyle-tracking application supported by a Flutter frontend, Django REST backend, and centralized database.

Its scope includes authentication, health profiles, nutrition, hydration, activity, steps, sleep, medications, unhealthy habits, personal goals, notifications, dashboards, and periodic reports.

The system also supports selected chronic-condition restrictions, centralized health constraints, AI-assisted food analysis, and personalized nutrition support through REST-based client–server communication.

The project scope covers requirements analysis, system and database design, implementation, integration, testing, evaluation, documentation, and academic demonstration.

1.5.2 Out-of-Scope

VitaMate does not provide medical diagnosis, prescribe treatment, replace healthcare professionals, modify medications, or function as a hospital or electronic medical-record system.

Full integration with all wearable devices, medical devices, and external health platforms is outside the current scope.

AI-generated food and nutrition results are supportive estimates and must be reviewed by the user before being added to health records.

1.5.3 Project Constraints

The project is subject to the following constraints:

1. **Time:** Development, integration, testing, and documentation must be completed within the graduation-project schedule.
2. **Team size:** The system is developed by a two-member team, limiting parallel work.
3. **Technical dependencies:** The application depends on stable backend APIs, database services, libraries, and device permissions.
4. **AI and data:** Food-analysis quality depends on dataset coverage, image quality, food variety, segmentation confidence, and estimation methods.

5. **Health safety:** Health-related rules require cautious wording and reliable references.
6. **Security and privacy:** Personal and health data must be protected through authentication, access control, and appropriate data-protection practices.

These constraints influenced the project architecture, implementation priorities, testing scope, and final delivery.

1.5.4 Project Assumptions

The project was planned according to the following assumptions:

1. Both team members remain available during development and testing.
2. Required tools, libraries, APIs, databases, and testing environments remain available.
3. Frontend and backend development can be coordinated through defined API contracts.
4. Academic and technical feedback is provided during project reviews.
5. Target devices support the required permissions and services, including notifications and step tracking.

1.6 Project Contribution

VitaMate's main contribution is the integration of nutrition, hydration, activity, sleep, medications, habits, goals, chronic-condition support, notifications, and reports within one coordinated platform.

A central technical contribution is the Constraints Layer, which connects user profiles, health restrictions, chronic conditions, goals, and tracker behavior to reduce duplicated or inconsistent rules.

The project also integrates review-assisted AI food analysis and personalized nutrition support while preserving user control over uncertain results.

From a software engineering perspective, VitaMate demonstrates the design, implementation, integration, and testing of a modular multi-service mobile health system. Its contribution is architectural and integrative rather than diagnostic.

1.7 Research and Development Methodology

VitaMate was developed using an applied software engineering methodology combining descriptive, analytical, engineering, and evaluative approaches.

The descriptive and analytical approach was used to review digital-health concepts, related studies, existing applications, and system gaps. The engineering approach covered requirements analysis, UML modeling, architecture, database design, frontend and backend development, API integration, and module implementation.

Agile Scrum was used to organize development into iterative sprints supporting continuous integration, testing, and refinement. The evaluative approach included requirements traceability,

functional and integration testing, performance measurements, usability evaluation, and intelligent-service analysis.

Senior Project 1 established the project foundations, while Senior Project 2 focused on integration, refinement, testing, and final evaluation.

1.8 Report Organization

This report contains eight chapters followed by the conclusion, references, and appendices.

Chapter 1 introduces the project background, problem, objectives, scope, contribution, and methodology. Chapter 2 presents the development methodology and project management. Chapter 3 reviews the theoretical framework, related studies, existing applications, and research gap. Chapter 4 defines the requirements, stakeholders, UML models, and traceability.

Chapter 5 presents the system architecture and detailed design. Chapter 6 describes the development environment and implementation. Chapter 7 reports the testing and evaluation results. Chapter 8 analyzes the results, objective achievement, performance indicators, strengths, and limitations.

The conclusion summarizes the main outcomes, challenges, lessons learned, and future work.

1.9 Key Definitions and Concepts

Digital Health: The use of digital technologies to support health awareness, monitoring, and management.

Lifestyle Tracking: Recording and reviewing behaviors such as nutrition, hydration, activity, sleep, medications, and habits.

Integrated Health Tracking: Coordinating multiple health and lifestyle domains through shared data and connected services.

Health Profile: Personal and health-related information used to configure system targets, restrictions, and behavior.

Constraints Layer: Centralized logic that coordinates health restrictions, goals, and tracker behavior across VitaMate.

AI-Assisted Food Analysis: Using computer-vision methods to estimate food information from meal images for user review.

Personalized Supportive Guidance: Non-diagnostic feedback based on user data, goals, behaviors, and health restrictions.

Chronic-Condition-Aware Support: Considering selected chronic conditions when configuring trackers, restrictions, alerts, and guidance.

1.10 Chapter Summary

This chapter introduced VitaMate as an intelligent and integrated mobile application for health and lifestyle tracking. It explained the fragmentation problem addressed by the project, defined its objectives and boundaries, and presented its practical and technical significance. The chapter also identified the main project contribution, summarized the research and development methodology, clarified the essential concepts, and outlined the structure of the report. The following chapter presents the development methodology and project-management practices adopted throughout the VitaMate project.

Chapter 2: Development Methodology and Project Management

2.1 Introduction

VitaMate is a multi-module software project that combines a mobile application, backend services, a centralized database, health-tracking functions, notifications, and intelligent services. The interdependence between these components requires structured planning, incremental implementation, continuous integration, and systematic quality control.

The project was developed across two academic stages. Senior Project 1 established the main analytical, planning, design, and technical foundations. Senior Project 2 builds on this baseline to complete the remaining implementation, integrate the system modules, refine the user experience, expand testing, and prepare the final system for evaluation and delivery.

To ensure a clear distinction between the two stages, this chapter first summarizes only the work that was completed within the defined scope of Senior Project 1. It then presents the development methodology, schedule, team organization, risk-management approach, quality plan, monitoring mechanisms, and software configuration-management practices adopted for Senior Project 2.

2.2 Completed Baseline from Senior Project 1

Senior Project 1 focused on transforming the initial VitaMate concept into a structured software-engineering foundation. The completed work covered requirements engineering, project planning, preliminary system design, implementation foundations, an independent AI prototype, and prototype-level quality assurance.

Features that remained partially implemented, required further integration, or needed additional validation are not classified as completed Senior Project 1 outcomes. They are addressed as part of the Senior Project 2 scope.

2.2.1 Requirements Engineering and Analysis

The requirements-analysis stage was completed during Senior Project 1. It included defining the project problem, objectives, scope, stakeholders, user classes, system functions, functional requirements, non-functional requirements, constraints, assumptions, and principal use cases.

The analysis also produced initial test cases and a Requirements Traceability Matrix linking selected requirements with their corresponding design, implementation, and verification elements. These artifacts established a controlled baseline for subsequent development.

2.2.2 Project Planning

The initial project-management package was completed through the preparation of the Project Charter, Statement of Work, Product Backlog, sprint-based plan, Gantt chart, team responsibilities, assumptions, and risk register.

These documents defined the project direction, expected outputs, task dependencies, work distribution, and initial delivery schedule. They also provided an organizational basis for tracking scope changes and transferring unfinished work to Senior Project 2.

2.2.3 Preliminary System Design

Senior Project 1 completed the preliminary design package required to guide implementation. This package included the high-level client–server architecture, database design, system-module structure, selected UML diagrams, API planning, frontend organization, and initial user-interface designs.

The design adopted a layered structure separating the presentation, business-logic, data-access, and storage responsibilities. This separation provided a maintainable foundation for extending the application during the second stage.

2.2.4 Backend Foundation

A structured Django REST backend foundation was implemented during Senior Project 1. It provided the main project organization, database-connected services, API communication structure, authentication support, and reusable backend components required by the health-tracking modules.

This outcome is classified as a completed **technical foundation**, rather than a claim that every backend service or cross-module workflow had reached its final form.

2.2.5 Frontend Foundation

A modular Flutter application foundation was also implemented. It included the principal application structure, routing, authentication flow, API-client organization, reusable interface components, and feature-based folders for the major VitaMate domains.

As with the backend, this outcome represents a completed application foundation. Individual screens or workflows that still required redesign, correction, or complete backend integration were transferred to Senior Project 2 and are not counted as fully completed Senior Project 1 features.

2.2.6 Standalone AI Food-Analysis Prototype

Senior Project 1 produced a standalone experimental prototype for food-image analysis. The prototype included stages for food-region segmentation, mask processing, depth estimation, visual recognition, label mapping, estimated weight calculation, experimental nutrition lookup, and structured API output.

The prototype itself was completed and evaluated as an independent technical artifact. However, its integration with the Flutter application, Django backend, nutrition tracker, and final user workflow was outside the completed Senior Project 1 baseline and was transferred to Senior Project 2.

2.2.7 Prototype-Level Testing and Quality Assurance

Testing and quality-assurance evidence was prepared for the Senior Project 1 prototype. The work included backend tests, Flutter analysis and tests, selected integration flows, performance measurements, pre-commit checks, security-oriented secret scanning, and a GitHub Actions CI pipeline.

These activities verified the technical foundation and selected prototype flows. They did not constitute final system acceptance, complete usability evaluation, comprehensive security validation, or production-deployment approval.

Area	Completed Outcome
Requirements Engineering	Problem, objectives, scope, stakeholders, functional and non-functional requirements, use cases, initial test cases, and RTM
Project Planning	Project Charter, SOW, Product Backlog, sprint plan, Gantt chart, responsibilities, assumptions, and risk register
Preliminary Design	High-level architecture, database design, module structure, UML models, API planning, and initial UI/UX designs
Backend Foundation	Structured Django REST project, database connectivity, authentication support, and reusable API components
Frontend Foundation	Modular Flutter structure, routing, authentication flow, API-client organization, and reusable UI components
AI Prototype	Independent experimental food-image-analysis pipeline with structured local API output
Quality-Assurance Foundation	Backend and Flutter tests, selected integration tests, performance evidence, static checks, secret scanning, and CI support

Table 2 :Completed Outcomes of Senior Project 1

The completed Senior Project 1 work therefore provided a verified analytical and technical baseline. It did not represent a final deployed application, and it did not imply that all proposed modules had been fully integrated or validated.

2.3 Senior Project 2 Development Scope

Senior Project 2 begins from the completed baseline described in Section 2.2. Its purpose is not to repeat the requirements and preliminary design activities of the first stage, but to transform the existing prototype foundation into a stable, integrated, and evaluated final system.

The second stage focuses on the following development areas:

1. Completing the integration between the Flutter frontend, Django REST backend, database, and shared health services.
2. Refining and unifying the user interfaces according to a consistent visual identity.
3. Improving user-profile management and selected chronic-condition workflows.
4. Correcting and stabilizing nutrition, hydration, activity, steps, sleep, medication, habit, goal, dashboard, and report workflows.
5. Integrating and coordinating the notification system with the relevant trackers and schedules.
6. Refining the Constraints Layer to coordinate health profiles, chronic conditions, goals, health restrictions, and tracker behavior.
7. Integrating the AI food-analysis service with the nutrition workflow while enabling user review and correction.
8. Improving motivational feedback, points, progress indicators, and health-summary calculations.
9. Expanding functional, integration, end-to-end, performance, security, real-device, and user-oriented testing.
10. Completing documentation, code refinement, deployment preparation, and final academic delivery.

These areas represent the development scope of Senior Project 2 and must not be interpreted as completed results until their implementation and testing evidence is presented in Chapters 6, 7, and 8.

2.4 Selected Development Methodology

2.4.1 Agile Scrum

VitaMate follows Agile Scrum as its primary development methodology. Scrum organizes development into short iterations called sprints. Each sprint contains a defined objective, selected backlog items, implementation activities, integration work, testing, and review.

The Product Backlog acts as the central repository for requirements, defects, improvements, integration tasks, technical work, and documentation activities. Items are selected for each sprint according to their priority, dependencies, technical risk, and importance to the final system.

The recurring sprint workflow consists of:

1. Reviewing the current requirements and system status.

2. Selecting and prioritizing backlog items.
3. Refining the design and API contracts when required.
4. Implementing frontend, backend, database, or intelligent-service changes.
5. Integrating the affected components.
6. Executing the related tests.
7. Correcting identified defects.
8. Reviewing the sprint results and updating the backlog.
- 9.

2.4.2 Rationale for Selecting Agile Scrum

Agile Scrum was selected because VitaMate contains several interconnected modules whose requirements and dependencies may become clearer during implementation and integration. A strictly sequential methodology would make late changes more difficult and could postpone critical integration problems until the end of the project.

Scrum supports VitaMate by:

- Dividing the system into manageable development increments.
- Allowing high-risk dependencies to be identified early.
- Supporting continuous integration and testing.
- Enabling requirements and priorities to be refined.
- Providing regular opportunities for supervisor feedback.
- Allowing defects to be returned to the backlog and scheduled systematically.
- Supporting parallel work between frontend, backend, AI, testing, and documentation activities.

2.4.3 Adaptation to the Project Team

Scrum was adapted to the needs of a two-member graduation-project team. The team did not assign separate full-time Scrum roles. Instead, backlog management, sprint planning, development, testing, review, and documentation responsibilities were shared according to the technical focus of each task.

Sprint activities were kept lightweight to avoid unnecessary administrative overhead. The emphasis was placed on clear sprint goals, small tasks, defined acceptance conditions, frequent integration, and documented outputs.

Scrum Activity	Application in VitaMate
Product Backlog	Stores requirements, defects, improvements, integration tasks, tests,

	and documentation work
Sprint Planning	Defines the sprint goal, selected tasks, dependencies, and expected outputs
Sprint Execution	Includes design refinement, implementation, integration, testing, and defect correction
Sprint Review	Evaluates completed work and identifies remaining issues
Sprint Retrospective	Reviews the team workflow and identifies process improvements
Backlog Refinement	Updates priorities and divide large items into manageable tasks

Table 3 :Adapted Scrum Activities

2.4.4 Definition of Done

A backlog item was considered complete only when its required functionality had been implemented, integrated, reviewed, and verified. Completion also required appropriate input validation, error handling, data consistency, interface conformity, testing, and documentation updates. An isolated interface screen or backend endpoint was not considered complete when the related end-to-end workflow remained unfinished.

Criterion	Completion Requirement
Implementation	The required frontend and backend functionality is implemented.
Validation	Required inputs and invalid-data cases are handled correctly.
Integration	The feature communicates correctly with related modules and services.
Data handling	Records are stored, retrieved, updated, and deleted according to the requirement.
User control	Sensitive or AI-generated changes require appropriate user confirmation.
Error handling	Expected failures and edge cases produce clear system responses.
Testing	Relevant unit, API, integration, or interface tests are completed.
Security	Authentication, authorization, ownership, and permissions are verified.
User interface	The feature follows the application's visual and usability conventions.
Documentation	Technical changes and relevant report sections are updated.
Review	The completed item is reviewed and accepted by the project team.

Table 4 :Definition Done of

2.4.5 Sprint Review and Retrospective

At the end of each sprint, the completed work was reviewed against the sprint objective and the acceptance conditions of the selected backlog items. Unfinished tasks, identified defects, and newly discovered dependencies were returned to the Product Backlog and reprioritized for later work.

Sprint retrospectives were used to improve the development process. The principal improvements included dividing complex integrations into smaller tasks, testing frontend and backend changes together, validating cross-module effects earlier, and reserving dedicated time for stabilization and documentation.

2.5 Project Lifecycle

The VitaMate development lifecycle combines software-engineering activities with iterative Scrum delivery. The main lifecycle stages are shown in Table 2.3.

Stage	Main Activities	Main Output
Analysis	Problem definition, requirements elicitation, stakeholder analysis, and scope definition	Approved requirements baseline
Planning	Backlog organization, task prioritization, schedule preparation, and risk identification	Project and sprint plans
Design	Architecture, database, UML, API, and UI/UX design	Design package
Implementation	Frontend, backend, database, and AI-service development	Implemented components
Integration	API connection, shared-data coordination, and cross-module workflows	Integrated system increments
Testing	Functional, integration, performance, security, usability, and device testing	Verified test evidence
Evaluation	Requirements verification, objective assessment, and KPI analysis	Final evaluation results
Delivery	Documentation, code refinement, demonstration, and submission	Final project package

Table 5 :VitaMate Development Lifecycle

The stages are not executed only once. Design, implementation, integration, and testing are repeated across the sprints whenever a module or shared service is added or modified.

2.6 Project Schedule and Gantt Chart

The Gantt chart presents the Senior Project 2 schedule, including the planned sprints, principal development activities, integration tasks, testing activities, documentation work, academic

milestones, and final delivery date. It also shows the chronological sequence, duration, responsibility, and dependencies of the principal project tasks.

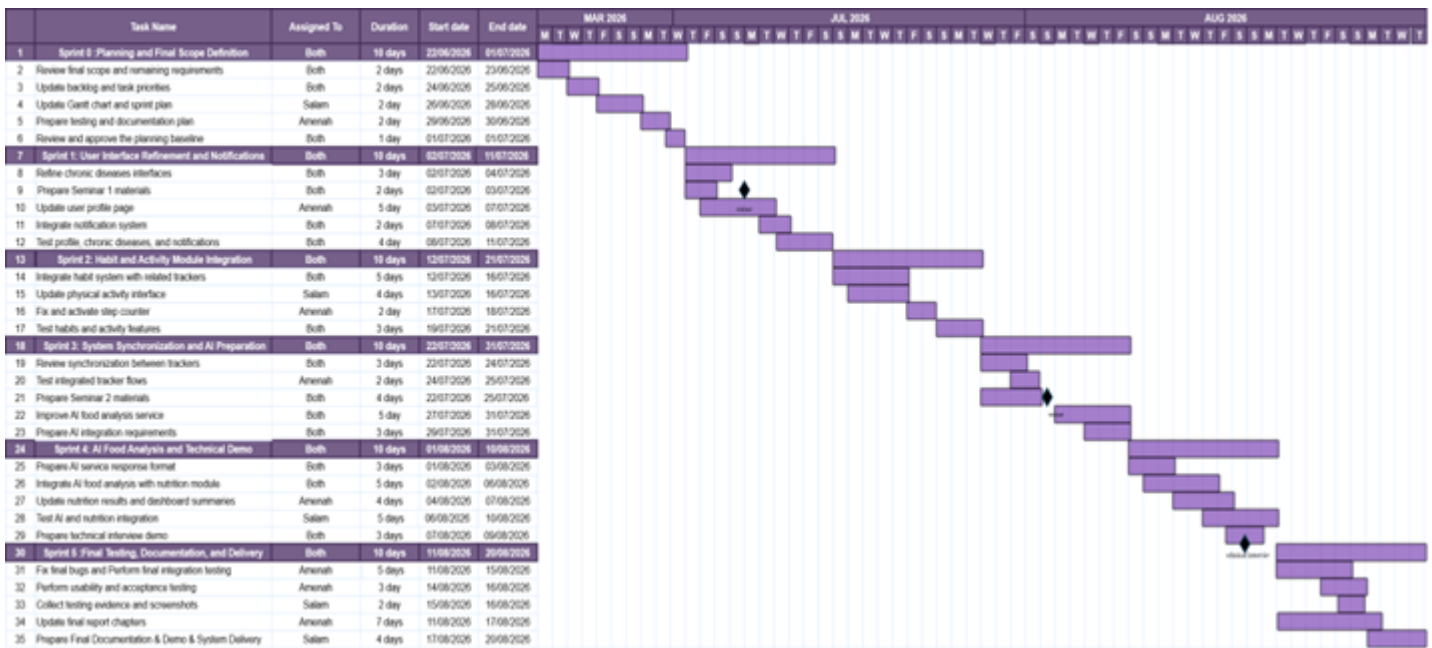


Figure 1 :Gantt Chart for Senior Project 2

The schedule begins with scope refinement and planning, followed by interface and module improvements, tracker integration, AI food-analysis preparation and integration, system stabilization, testing, documentation, and final delivery. Activities that do not depend directly on one another may proceed in parallel, while integration and testing activities follow the completion of their related implementation tasks.

2.7 Risk Management

Risk management is treated as a continuous activity integrated with sprint planning, backlog refinement, testing, and project reviews. Risks are evaluated according to their probability and potential impact, and each major risk is associated with a preventive or corrective response.

Risk	Probability	Impact	Treatment Plan
Schedule pressure before final delivery	High	High	Prioritize essential requirements, reserve the final sprint for testing and documentation, and defer non-essential polish
Limited two-member team capacity	High	High	Divide work by technical responsibility, limit parallel high-risk tasks, and review workload during each sprint
Cross-module integration defects	High	High	Define API contracts, integrate incrementally, and execute regression tests after shared-service changes
AI-service accuracy or integration limitations	High	High	Preserve user confirmation, display uncertain results cautiously, support manual correction, and prevent automatic saving of unreliable outputs
Inconsistent health rules across modules	Medium	High	Centralize shared rules through the Constraints Layer and test conflicting conditions and goals
Health-safety or unsupported recommendation risk	Medium	High	Use authoritative references, avoid diagnostic claims, and require cautious user-facing wording
Notification or device-permission failures	Medium	Medium	Test on supported Android devices, verify permissions and scheduling behavior, and provide clear fallback handling
Data privacy or security defects	Medium	High	Apply secure authentication, protect secrets, restrict access, and perform security-oriented verification
Late defects in reports or dashboard aggregation	Medium	High	Test aggregated endpoints early and verify calculations against source records
Documentation not matching implementation	Medium	Medium	Update documentation during each sprint and perform a final traceability review

Table 6 :Team Roles and Responsibilities

This distribution does not prevent responsibilities from overlapping. Tasks are assigned according to dependencies, workload, and the expertise required during each sprint.

2.8 Team Roles and Responsibilities

The project team consists of two students supported by two academic supervisors. Responsibilities were distributed according to the technical needs of each sprint, while integration, testing, review, and documentation remained shared responsibilities.

Member	Role	Main Responsibilities
Dr. Kadan Jumaa	Academic Supervisor	Academic guidance, project-direction review, quality follow-up, and evaluation of major deliverables
Eng. Raghad Al-Hossny	Technical Supervisor	Technical and organizational guidance, progress review, and feedback on development stages
Amenah Ayman Zaitoun	Software Engineer	Requirements analysis, documentation, system design, technical coordination, and contribution to frontend and backend development
Salam Mohammed Al-Ayash	Software Engineer	Frontend and backend development, database integration, implementation of system functions, testing, and technical refinement
Both Students	Shared Project Team	Sprint planning, integration, code review, testing, documentation, issue resolution, and presentation preparation

Table 7 :Senior Project 2 Risk Register

2.9 Quality Management

Quality management aims to ensure that VitaMate satisfies its requirements, produces consistent output, protects user data, and remains maintainable. Quality is controlled throughout development rather than being evaluated only after implementation.

The principal quality practices include:

1. Defining acceptance conditions for important backlog items.
2. Reviewing frontend, backend, database, and API changes before integration.

3. Running static analysis and automated tests.
4. Testing affected integration workflows after shared changes.
5. Checking database migrations and API-response consistency.
6. Recording defects and returning unresolved items to the backlog.
7. Maintaining requirements traceability.
8. Reviewing user-facing health messages for clarity and safety.
9. Updating technical documentation when implementation changes.
10. Preventing credentials and sensitive tokens from being committed to the repository.

Quality Mechanism	Purpose
Acceptance Criteria	Defines the conditions required before a task is considered complete
Code Review	Detects logic, structure, consistency, and maintainability issues
Static Analysis	Identify code-quality problems before execution
Automated Testing	Verifies repeatable backend and frontend behavior
Integration Testing	Confirms communication between system components
Regression Testing	Detects defects introduced by later changes
Security Checks	Detect secrets, insecure configuration, and access-control issues
Traceability Review	Confirms that requirements are linked to implementation and tests
Documentation Review	Ensure that the report reflects the actual system

Table 2.7. Quality-Control Mechanisms

pre-commit checks, Django testing, Flutter analysis, Flutter tests, and selected Flutter integration tests. The guide also explicitly requires code reviews, automated testing, acceptance criteria, and static-analysis mechanisms within project quality management.

2.10 Project Management and Collaboration Tools

Project management relied on lightweight tools suitable for the size of the team. The emphasis was placed on clear planning records and verifiable technical outputs rather than complex administrative processes.

Tool or Artifact	Use
Product Backlog	Organizing requirements, defects, improvements, and technical tasks
Sprint Plan	Defining the objective and workload of each sprint
Gantt Chart	Monitoring the project timeline and milestones
Risk Register	Recording and reviewing major project risks
Requirements Traceability Matrix	Linking requirements with design, implementation, and testing
Git Repository	Version control and source-code collaboration
GitHub Actions	Automated analysis, testing, and security-oriented checks during Senior Project 1
Pre-commit Checks	Detecting selected quality issues before commits
Gitleaks	Detecting exposed secrets and sensitive tokens
Test Reports	Recording functional, integration, and performance evidence

Table 2.8. Project Management and Collaboration Tools

2.11 Monitoring and Continuous Evaluation

Project progress is monitored through recurring Scrum activities and technical evidence. Sprint Planning defines the intended work, while Sprint Reviews compare planned outputs with completed results. Unfinished tasks, defects, or newly identified dependencies are returned to the Product Backlog.

Continuous monitoring includes:

- Reviewing progress against the sprint goal.
- Monitoring blocked tasks and technical dependencies.
- Reviewing changes with the supervisors.

- Evaluating test and integration results.
- Updating the risk register when necessary.
- Comparing the actual timeline with the Gantt chart.
- Reviewing unresolved defects before starting a new sprint.
- Verifying that documentation remains aligned with implementation.

A task is considered complete only when its required implementation, integration, verification, and documentation conditions have been satisfied. Completing an isolated screen or backend endpoint is not sufficient when the related end-to-end workflow remains incomplete.

2.12 Software Configuration Management

Git is used to preserve source-code history, coordinate changes, and support rollback when defects occur. During Senior Project 1, repository and CI evidence were maintained through GitHub and GitHub Actions.

The configuration-management process includes:

1. Maintaining a stable primary branch.
2. Developing isolated changes in separate task- or feature-oriented branches.
3. Reviewing and testing changes before merging.
4. Using clear commit messages linked to the implemented task or correction.
5. Avoiding the storage of credentials and sensitive configuration in the repository.
6. Keeping database migrations, dependency files, and environment documentation synchronized with the code.
7. Recording stable versions associated with academic demonstrations and final delivery.

2.13 Chapter Summary

This chapter distinguished the completed Senior Project 1 baseline from the approved development scope of Senior Project 2. The first project stage established the requirements, planning, preliminary design, frontend and backend foundations, standalone AI food-analysis prototype, and prototype-level quality-assurance evidence. Functions that remained incomplete, non-integrated, or insufficiently validated were transferred to Senior Project 2 rather than being presented as completed outcomes.

The chapter also presented the Agile Scrum methodology adopted for VitaMate, its adaptation to the two-member development team, the project lifecycle, sprint schedule, risk-management approach, team responsibilities, quality practices, project-management tools, monitoring mechanisms, and software configuration-management process. Together, these elements provide the organizational and technical-management framework required to complete, integrate, test, and evaluate the final system.

Chapter 3

Theoretical Framework and Related Work

3.1 Introduction

This chapter presents the theoretical concepts and previous research relevant to the VitaMate project. It discusses digital and mobile health, integrated lifestyle self-monitoring, behavior-change support, personalized health guidance, constraints-aware coordination, and AI-assisted food analysis.

The chapter also reviews recent studies related to mobile health applications, digital self-monitoring, personalized interventions, chronic-condition support, and food-image analysis. Existing health and lifestyle applications are then compared to identify their main strengths and limitations and to clarify the gap addressed by VitaMate.

3.2 Digital Health and Mobile Health

Digital health refers to the use of digital technologies, software systems, connected devices, health data, and intelligent techniques to support health-related needs. The World Health Organization describes digital health as a broad field that includes mobile health applications, health-information systems, artificial intelligence, connected devices, telemedicine, and data-driven health services [1].

Mobile health, commonly referred to as mHealth, applies mobile technologies to health monitoring, information delivery, communication, and self-management. Mobile applications provide a practical environment for recording daily health information, displaying progress, sending reminders, and supporting interaction with personal health data.

Mobile devices are particularly suitable for lifestyle monitoring because they are frequently available to users and can support repeated daily interactions. However, the practical value of an MHealth application depends on the quality of its data, usability, personalization, privacy controls, and the clarity of its health-related outputs.

VitaMate belongs primarily to the mHealth and personal health-management categories. It uses a mobile interface supported by backend services, a centralized database, and selected intelligent functions to help users monitor several health and lifestyle dimensions.

The World Health Organization's updated classification recognizes digital applications and services as tools that can address individual needs and support broader health goals, while emphasizing the importance of clearly identifying the problem, user need, and digital capability addressed by each intervention.

3.3 Integrated Lifestyle Self-Monitoring

Self-monitoring is the process through which individuals record and review their behaviors, conditions, or progress over time. In lifestyle applications, this may include recording meals, water consumption, physical activity, sleep, medications, symptoms, habits, and personal goals.

Digital self-monitoring can make health information easier to record, organize, and review than paper-based tracking. It may also provide immediate summaries and visual feedback. A systematic review and meta-analysis by Berry et al. found that digital self-monitoring of diet and physical activity supported improvements in weight-related outcomes, physical activity, and calorie intake among the studied population. The review also found that tailored interventions were more effective than non-tailored approaches [2].

Despite these benefits, tracking multiple lifestyle dimensions through separate applications can create fragmented records. Users may receive isolated results without understanding the relationships between their behaviors. For example, sleep patterns may affect energy and activity, while nutrition and hydration may influence progress toward personal health goals.

Integrated self-monitoring aims to address this problem by organizing several related domains within one system. A shared environment can reduce repeated profile entry, provide a broader view of progress, and enable system components to use common user information.

VitaMate applies this concept by integrating several trackers and shared services within one platform. The purpose is not only to store different records, but also to coordinate the way these records contribute to reminders, progress summaries, constraints, and supportive guidance.

3.4 Behavior-Change Support

Recording data alone may not be sufficient to support consistent lifestyle improvement. Digital health applications frequently use behavior-change techniques to help users establish goals, observe progress, remember planned actions, and maintain engagement.

Common techniques include:

1. Self-monitoring of behavior and outcomes.
2. Goal setting.

3. Progress feedback.
4. Reminders and prompts.
5. Motivational messages.
6. Rewards and achievement indicators.
7. Personalization according to recorded behavior.

A systematic review by Aguiar et al. examined behavior-change techniques used in mHealth applications to improve adherence to self-reporting and self-management. The review found that effective applications often combined several techniques, although the results depended on how the techniques were implemented rather than merely on their presence [3].

This finding is relevant to VitaMate because the application combines tracking with goals, notifications, motivational feedback, progress indicators, and periodic reports. These elements should operate as a coordinated experience rather than as independent decorative features.

Behavior-change mechanisms must also avoid excessive notifications or misleading rewards. Reminders should be relevant to the user's schedule and current progress, while motivational indicators should reflect meaningful actions and verified records. Therefore, the effectiveness of the motivational system depends on correct calculations, understandable feedback, and appropriate timing.

3.5 Personalized and Constraints-Aware Coordination

Personalization refers to adapting system behavior according to the characteristics, goals, preferences, and recorded information of an individual user. In a lifestyle application, personalization may affect daily targets, reminders, progress indicators, recommendations, and the information emphasized on dashboards.

Basic personalization may rely on profile attributes such as age, sex, height, weight, activity level, and personal objectives. More advanced personalization may also consider chronic conditions, medications, health restrictions, recent tracking data, and the relationships between multiple goals.

In an integrated system, personalization creates a coordination challenge. A rule generated by one module may conflict with information stored in another module. For example, a general nutrition goal may become unsuitable when a related chronic-condition restriction is active. Similarly, a motivational recommendation should not encourage an action that conflicts with a user-specific limitation.

VitaMate addresses this challenge through a centralized Constraints Layer. The layer is intended to collect and coordinate applicable goals, chronic-condition-related restrictions, profile information, and tracker rules. This approach reduces the need to duplicate health-related logic across separate modules and provides a single point for resolving priorities and conflicts.

The Constraints Layer does not replace professional medical judgment. Its purpose is to support internal system consistency by controlling how verified rules and recorded user information influence tracker behavior and user-facing outputs.

3.6 AI-Assisted Food Analysis

Manual food logging may require users to search for food items, select portions, and enter nutritional values. This process can increase the time and effort required to maintain consistent nutrition records. Image-based food analysis attempts to reduce part of this burden by extracting food-related information from meal photographs.

A typical food-image-analysis pipeline may include:

1. Detecting or segmenting the food region.
2. Recognizing the food category.
3. Estimating the portion or weight.
4. Mapping the recognized item to nutritional information.
5. Presenting the result for user review.

Deep-learning methods, particularly convolutional neural networks and related computer-vision models, have been widely studied for food classification and nutrient-identification tasks. A systematic review by Kaur, Kumar, and Gupta examined research on deep neural networks for food-image classification and nutritional identification and identified food recognition, object detection, and nutrition estimation as central components of image-based dietary assessment [4].

Food-image analysis remains challenging because meals may contain multiple items, overlapping ingredients, sauces, mixed dishes, uncertain scale, poor lighting, and partially hidden food. Correctly identifying a food does not automatically guarantee an accurate portion or nutritional estimate.

For this reason, VitaMate treats AI-generated food outputs as assisted estimates. The user must be able to review the detected item and estimated amount before the result is included in the nutrition record. Low-confidence or unsupported cases should be presented cautiously rather than converted automatically into final nutritional values.

3.7 Data Quality, Privacy, and Health Safety

Health-related applications process information that may be personal, sensitive, or capable of influencing user decisions. Therefore, data quality, privacy, security, and communication safety are essential design considerations.

Data quality depends on the correctness, completeness, consistency, and timeliness of recorded information. User-entered values may contain mistakes, sensor data may be unavailable, and intelligent estimates may contain uncertainty. The system should therefore validate inputs, identify missing information, and distinguish confirmed values from estimated outputs.

Privacy and security require appropriate authentication, authorization, protected communication, controlled data access, and secure handling of credentials. Health information should be collected only when it contributes to an identified system function and should not be exposed unnecessarily.

Health safety also requires clear limitations. The application should not present supportive guidance as a medical diagnosis, prescribe treatment, or recommend independent medication changes. Outputs related to chronic conditions or uncertain AI results should use cautious wording and provide opportunities for user confirmation.

The World Health Organization emphasizes that digital-health systems require proper research and evaluation to demonstrate their benefits, avoid harm, and ensure accountable implementation. Digital innovation should complement appropriate health services rather than replace evidence-based professional care [5].

3.8 Review of Related Studies

Recent research has examined digital self-monitoring, personalization, medication adherence, user engagement, chronic-condition management, and image-based dietary assessment. Table 3.1 summarizes the studies most relevant to the design of VitaMate.

Reference	Study Focus	Main Findings	Relevance to VitaMate
Berry et al., 2021 [2]	Digital self-monitoring of diet and physical activity	Digital tracking supported improvements in weight-related outcomes, physical activity, and calorie intake. Tailored interventions generally achieved better results than non-tailored approaches.	Supports integrated tracking and personalized progress feedback.
Tong et al., 2021 [6]	Personalized mobile technologies for lifestyle behavior change	Personalized mobile interventions showed a moderate positive effect on lifestyle behaviors, although the reviewed	Supports adapting targets and feedback to individual user

		studies were heterogeneous and frequently small.	information.
Aguiar et al., 2022 [3]	Behavior-change techniques in mHealth applications	Self-monitoring, goal setting, feedback, prompts, and related techniques were commonly used to support user adherence. Effectiveness depended on implementation quality.	Supports goals, reminders, progress indicators, and motivational feedback.
Jakob et al., 2022 [7]	Factors affecting adherence to mHealth applications	Personalization, individualized reminders, stable and user-friendly design, and complementary support were repeatedly associated with improved adherence.	Supports interface simplicity, notification personalization, and technical stability.
Belete et al., 2023 [8]	Mobile reminders and medication adherence in type 2 diabetes	The meta-analysis found that mobile text-message reminders improved medication adherence compared with usual care, while long-term effectiveness remained insufficiently studied.	Supports medication scheduling and reminders while indicating the need for careful evaluation.
Arshed et al., 2023 [9]	mHealth interventions for cardiovascular medication adherence	Most reviewed studies reported positive effects, but several produced no significant improvement, indicating that reminders alone may not be consistently effective.	Supports combining reminders with adherence recording and broader user support.
Kaur et al., 2023 [4]	Deep-learning methods for food-image classification and nutrient identification	Food recognition, object detection, and nutrition estimation were identified as key components, with image quality and portion estimation remaining important challenges.	Supports the AI food-analysis pipeline and user confirmation of estimated results.
Milne-Ives et al., 2023 [10]	Behavior-change techniques and engagement with mHealth applications	Goal setting, self-monitoring, behavioral feedback, prompts, rewards, and social support showed the most evidence of potential association with engagement.	Supports the design of goals, feedback, reminders, and motivation features.

Figure 2: Summary of Related Studies

The reviewed studies indicate that mobile health applications can support self-monitoring, medication adherence, behavioral engagement, and lifestyle improvement. However, their effectiveness depends on personalization, usability, technical stability, appropriate reminders, and meaningful feedback rather than on the presence of tracking functions alone.

The studies also show that health applications require cautious evaluation. Positive findings are not always consistent across user groups or intervention types, and many studies remain limited by short durations, small samples, or differences in outcome measurement.

Therefore, VitaMate should be evaluated according to its actual implemented functions and should not claim clinical effectiveness without appropriate clinical evidence.

3.9 Comparative Analysis of Existing Applications

Several existing applications provide functions related to the VitaMate scope. MyFitnessPal focuses primarily on nutrition and fitness logging, Samsung Health provides a connected wellness view, Medisafe specializes in medication management, and Google Health provides activity, sleep, health-data tracking, and personalized coaching.

The comparison in Table 3.2 is based on the functions described by the official product sources. A limitation identified in the table represents a difference from the intended VitaMate scope rather than a general defect in the compared application.

Application	Primary Focus	Main Strengths	Limitations Relative to VitaMate
MyFitnessPal [11]	Nutrition and fitness tracking	Food diary, calorie and nutrient tracking, exercise logging, progress monitoring, meal scanning, and personalized nutrition support	Primarily centered on nutrition and fitness rather than medication management and coordinated chronic-condition constraints
Samsung Health [12]	General health and wellness monitoring	Connected monitoring of activity, sleep, nutrition, stress, mindfulness, and selected health indicators	Does not present a centralized constraints mechanism for coordinating chronic conditions, goals, medications, and tracker rules

Medisafe [13]	Medication management	Medication schedules, dose reminders, refill alerts, appointment support, interaction information, and adherence-related functions	Primarily medication-oriented and does not provide integrated nutrition, activity, sleep, hydration, and food-image analysis
Google Health [14]	Activity, sleep, fitness, and health-data monitoring	Activity tracking, sleep insights, health-data organization, fitness planning, and personalized coaching	Focuses mainly on fitness, sleep, and connected health data rather than integrated medication, nutrition-analysis, and health-constraint coordination

Figure 3: Comparison of Existing Health Applications

MyFitnessPal provides extensive nutrition and fitness functions, including diary-based tracking, nutrient monitoring, exercise records, progress review, and AI-supported nutrition functions. Samsung Health combines activity, sleep, nutrition, stress, and wellness indicators within a connected view. Medisafe provides dedicated medication reminders, refill management, appointments, and health measurements. Google Health emphasizes activity, sleep, health data, and personalized fitness coaching.

The comparison shows that existing applications provide strong functions within their principal domains. However, no single compared application, based on the reviewed official descriptions, combines the full intended VitaMate scope of lifestyle tracking, medication management, chronic-condition-aware constraints, motivational coordination, periodic reporting, and reviewable AI-assisted food analysis.

3.10 Identified Research and System Gap

The reviewed studies and applications reveal three main gaps.

First, much of the literature evaluates a specific health behavior, intervention, or condition, such as diet and activity self-monitoring, medication reminders, chronic-condition management, or engagement techniques. This provides valuable evidence for individual functions but offers limited guidance on coordinating several health domains within one application.

Second, existing applications commonly provide either broad wellness tracking or specialized management of a particular domain. Broad applications may integrate activity, sleep, and nutrition, while specialized applications may provide more detailed nutrition or medication functions. Coordination between chronic conditions, health restrictions, personal goals, tracker calculations, reminders, and motivational feedback remains limited in the compared systems.

Third, image-based food analysis can reduce manual logging effort, but food recognition and portion estimation remain uncertain. Applications that use such functions should distinguish estimated outputs from confirmed user records and allow users to review and correct results before they affect nutritional calculations.

VitaMate addresses these gaps through an integrated architecture that connects multiple lifestyle trackers with shared user information. Its proposed distinction lies in combining:

1. Multi-domain health and lifestyle tracking.
2. Medication and chronic-condition support.
3. A centralized Constraints Layer for coordinating goals and restrictions.
4. Notifications and motivational feedback connected to recorded behavior.
5. Periodic dashboards and reports.
6. AI-assisted food analysis with user review and confirmation.

The VitaMate contribution is therefore primarily integrative and architectural. The system does not claim to replace specialized medical platforms or introduce a clinically validated diagnostic method.

3.11 Current Technology Trends

Current digital-health development is moving toward greater personalization, connected data, intelligent assistance, and continuous user engagement. The principal trends relevant to VitaMate include the following:

3.11.1 Personalized Digital Interventions

Health applications increasingly adapt goals, feedback, reminders, and displayed information according to individual characteristics and recorded behavior. Research suggests that personalization may improve lifestyle outcomes and adherence, although its effectiveness depends on appropriate implementation and evaluation [6], [7].

3.11.2 Connected Multi-Domain Health Data

Modern health platforms are moving toward connected views that combine activity, sleep, nutrition, wellness indicators, and other user data. This trend supports a broader interpretation of user behavior but also creates a need for consistent calculations, shared definitions, and controlled data access [12], [14].

3.11.3 AI-Assisted User Interaction

Artificial intelligence is increasingly used to support food recognition, personalized coaching, pattern identification, and health-data interpretation. Such functions can reduce user effort but

must communicate uncertainty and preserve user control, especially when outputs may influence health-related decisions [4], [11], [14].

3.11.4 User-in-the-Loop Validation

User-in-the-loop design requires the user to review, confirm, or correct uncertain system outputs. This approach is particularly important for meal recognition, portion estimation, automatically generated recommendations, and imported health data.

3.11.5 Engagement-Oriented Design

Goal setting, self-monitoring, feedback, reminders, rewards, and social support remain central components of digital behavior-change interventions. Current research emphasizes that these mechanisms should be meaningful, personalized, and connected to verified user actions [3], [10].

3.11.6 Privacy and Responsible Health Communication

As applications collect more personal and health-related data, privacy, authorization, data minimization, secure storage, and responsible communication become increasingly important. Intelligent outputs should be presented as supportive information when they have not been clinically validated for diagnosis or treatment.

These trends influenced VitaMate design direction, particularly its emphasis on personalization, integration, controlled intelligent assistance, user confirmation, and consistent health-related communication.

3.12 Chapter Summary

This chapter presented the theoretical and research foundations of VitaMate. It discussed digital health, integrated self-monitoring, behavior-change support, personalization, constraints-aware coordination, AI-assisted food analysis, data quality, privacy, and health safety.

The review of related studies showed that mobile technologies can support lifestyle tracking, engagement, and medication adherence, but their outcomes depend on personalization, usability, stability, and appropriate intervention design. The comparison of existing applications demonstrated that current systems provide strong specialized or general wellness functions, while coordination across health trackers, medications, chronic-condition restrictions, intelligent services, and motivational mechanisms remains limited.

Based on these findings, VitaMate is positioned as an integrated lifestyle-support platform whose contribution lies in coordinating several health-related services through shared user data, centralized constraints, and reviewable intelligent assistance.

Chapter 4

Requirements Analysis and System Modeling

4.1 Introduction

This chapter defines the final requirements baseline of the VitaMate system and presents the models used to describe its expected behavior. It evaluates the technical, economic, and operational feasibility of the project, identifies the sources from which the requirements were derived, and analyzes the needs of the verified stakeholders.

The chapter then presents the functional and non-functional requirements, their priorities, and the principal interactions between the user and the system. UML models, Agile requirements artifacts, initial test cases, and the Requirements Traceability Matrix are used to connect the requirements with the subsequent design, implementation, and testing activities.

Only requirements included in the approved final project scope are treated as system commitments. Features that remain outside the current release are identified through the MoSCoW prioritization rather than being presented as completed functions.

4.2 Feasibility Study

The feasibility study evaluates whether VitaMate can be developed and operated using the available technologies, resources, skills, and project schedule. The assessment covers technical, economic, and operational feasibility.

4.2.1 Technical Feasibility

VitaMate is technically feasible because its principal components can be developed using established mobile, backend, database, and artificial-intelligence technologies. The system uses a Flutter mobile frontend, Django REST backend services, a structured data layer, and REST APIs for communication between the client and server.

The Senior Project 1 baseline demonstrated that the frontend and backend foundations could be developed separately and connected through defined API endpoints. It also provided reusable foundations for authentication, user profiles, tracking modules, health-related services, notifications, gamification, and intelligent-service integration.

The required development skills are available within the project team, including mobile development, backend development, database integration, API design, testing, and basic machine-learning service development. The modular structure also allows individual services to be implemented and tested incrementally.

The AI food-analysis service introduces additional technical challenges related to image quality, segmentation confidence, food recognition, portion estimation, and service integration. Its feasibility therefore depends on maintaining a reviewable workflow in which users can confirm or correct uncertain results before they are saved.

The principal technical dependencies are summarized as follows:

Factor	Available Support	Feasibility Consideration
Mobile Development	Flutter-based application foundation	Supports modular mobile-interface development
Backend Services	Django and Django REST Framework	Supports APIs, authentication, business logic, and data processing
Data Storage	Structured relational data layer	Supports connected health and lifestyle records
Authentication	Token-based authentication foundation	Supports protected client–server communication
Device Services	Notifications and step-related device capabilities	Depend on permissions and supported Android services
AI Food Analysis	Separate FastAPI-based prototype	Requires final integration, validation, and user confirmation
Development Skills	Mobile, backend, database, testing, and AI experience	Adequate for the planned academic scope
Testing Environment	Local development, VM, device, and automated-test support	Supports iterative verification and integration testing

Figure 4: Technical Feasibility Factors

Overall, the project is technically feasible within its defined academic scope. Advanced clinical decision-making, unrestricted wearable-device integration, and fully automatic nutrition estimation remain outside the confirmed technical commitment.

4.2.2 Economic Feasibility

The development of VitaMate relies primarily on open-source frameworks, development tools, and existing team hardware. Flutter, Django, Django REST Framework, Python-based AI libraries, Git, and the database technologies used in the project do not require direct commercial licensing for the academic development stage.

The main project cost is therefore represented by development time, testing effort, computing resources, internet access, and the devices required for implementation and validation. No verified commercial revenue or financial return is claimed because VitaMate is currently an academic software project rather than a deployed commercial service.

Future public deployment may introduce additional costs related to server hosting, database storage, domain registration, backup services, monitoring, AI computing resources, and application-store publishing. These costs must be calculated according to the final deployment scale and are not included as unsupported numerical estimates in this report.

The project is economically feasible for academic implementation because it can be developed and demonstrated without purchasing specialized enterprise software or dedicated medical hardware.

4.2.3 Operational Feasibility

VitaMate is operationally feasible as a mobile self-monitoring application because its main functions follow familiar user interactions, including form entry, progress cards, reminders, daily records, summaries, and reports. The system is primarily designed for one external user class: the regular user.

No separate human administrator is required within the current mobile workflow. Authentication, data processing, reminder compilation, constraint evaluation, progress calculations, and report generation are handled through internal system services.

The application is intended to reduce the operational burden of using several separate health applications. However, daily tracking may still require repeated user input. VitaMate addresses this limitation through concise data-entry flows, reusable food and activity records, favorites, reminders, automatic calculations where supported, and AI-assisted meal analysis with manual correction.

Operational risks include notification fatigue, incomplete user records, inaccurate manual input, unavailable device permissions, and misunderstanding of supportive health outputs. These risks are addressed through configurable reminders, input validation, editable records, clear progress indicators, user confirmation, and explicit limitations regarding medical diagnosis and treatment.

The system is therefore operationally feasible as a lifestyle-tracking and self-management tool, provided that its outputs remain understandable, editable, and clearly distinguished from professional medical advice.

4.3 Requirements Elicitation and Sources

The VitaMate requirements were developed and refined iteratively throughout the project. The initial requirements baseline was established during Senior Project 1 and was updated during Senior Project 2 according to implementation findings, supervisor feedback, technical dependencies, and changes in the approved project scope.

The requirements were derived from documented and verifiable sources rather than from user surveys or interviews. These sources included the Senior Project 1 requirements documents, scientific literature, analysis of related applications, supervisor feedback, internal team analysis, Product Backlog refinement, and issues identified during prototype implementation and testing.

The documented requirement sources are summarized in Table 4.2.

Source	Application in VitaMate	Main Requirements Derived
Senior Project 1	Review of the previous SRS,	Core system functions,

Documentation	scope, use cases, constraints, risks, and traceability records	stakeholders, quality requirements, and initial system boundaries
Scientific Literature	Review of digital health, self-monitoring, behavior change, medication adherence, and food-image analysis studies	Personalization, reminders, usability, privacy, health safety, and AI-related requirements
Related Application Analysis	Comparison with existing health, nutrition, fitness, and medication applications	Integrated tracking, user control, dashboards, reminders, and identified system gaps
Supervisor Feedback	Review of scope, diagrams, technical decisions, and academic deliverables	Scope refinement, safety requirements, integration priorities, and documentation improvements
Team Analysis	Analysis of user workflows, module relationships, business rules, and technical constraints	Detailed functional flows, shared services, validation rules, and cross-module requirements
Product Backlog Refinement	Review and prioritization of tasks during sprint planning	Required features, defect corrections, integration tasks, and deferred functions
Prototype Implementation and Testing	Analysis of defects, incomplete flows, data inconsistencies, and usability issues	Updated integration, validation, reliability, performance, and error-handling requirements
Method or Source	Application in VitaMate	Main Requirements Derived
Senior Project 1 SRS and Project Documents	Review of the previous requirements, use cases, scope, constraints, risks, and traceability records	Core functional areas and initial quality requirements
Literature and Reference-System Analysis	Review of digital-health studies and existing health applications	Tracking, personalization, reminders, privacy, usability, and AI-related requirements
Supervisor Feedback	Review of project scope, diagrams, implementation direction, and academic deliverables	Scope refinement, safety considerations, integration priorities, and documentation requirements
Team Analysis and Workshops	Internal analysis of user workflows, module dependencies, technical risks, and API	Detailed workflows, shared-service requirements, constraints, and integration needs

	behavior	
Product Backlog Refinement	Revision and prioritization of requirements during sprint planning	Must-have functions, corrections, deferred features, and acceptance conditions
Prototype Implementation and Testing	Observation of defects, missing workflows, data inconsistencies, and usability issues	Updated integration, validation, performance, and reliability requirements

Figure 5: Requirements Elicitation Methods and Sources

Requirements were accepted into the final baseline when they were relevant to the project problem, technically feasible within the available resources, and traceable to a verified stakeholder need or technical dependency.

Health-related requirements were limited to tracking, organization, reminders, constraints, and supportive guidance. Diagnostic, therapeutic, or medication-changing functions were excluded from the project scope unless supported and reviewed through an appropriate professional process.

4.4 Stakeholders and Needs Analysis

VitaMate does not have an external commercial client or a separate human system administrator within its current scope. The principal stakeholders are the end user, academic supervisors, the development team, and future maintainers of the system.

Stakeholder	Role and Interest	Main Needs and Expectations	Influence
Regular User	Primary external actor who uses the mobile application	Secure account access, simple tracking, understandable summaries, relevant reminders, editable data, consistent guidance, and privacy protection	High

Academic Supervisors	Review the project's academic and technical quality	Correct scope, evidence-based decisions, clear documentation, reliable implementation, measurable testing, and compliance with the graduation-report guide	High
Development Team	Designs, implements, integrates, tests, and documents the system	Clear requirements, stable API contracts, modular architecture, manageable backlog, testable components, and controlled versions	High
Future Developers and Maintainers	May extend or maintain the system after the current project	Readable code, modular services, documented APIs, database integrity, traceable requirements, and reproducible environments	Medium
University Evaluation Committee	Evaluates the final academic and technical outcome	Clear contribution, complete evidence, working demonstration, justified decisions, and consistent results	Medium

Figure 6: Requirements Sources and Refinement Activities

A requirement was included in the final baseline when it was relevant to the project objectives, technically feasible within the available resources, and supported by a documented stakeholder need or system dependency.

Health-related requirements were limited to self-monitoring, data organization, reminders, constraints, progress feedback, and supportive guidance. Diagnostic functions, treatment prescriptions, and independent medication changes were excluded from the system scope.

4.5 Functional Requirements

The functional requirements define the services and behaviors that the final VitaMate system is expected to provide. They cover the complete approved project scope, regardless of whether a function was initially developed during Senior Project 1 or completed, integrated, or refined during Senior Project 2.

Each requirement is assigned a unique identifier to support its later connection with use cases, architectural components, implementation modules, initial test cases, and the Requirements

Traceability Matrix. The requirement descriptions define expected system behavior and do not indicate implementation status.

4.5.1 Functional Requirements Classification

ID	Functional Requirement
FR-01	The system shall allow a new user to create an account using the required personal and authentication information.
FR-02	The system shall allow a registered user to sign in and sign out securely.
FR-03	The system shall maintain an authenticated session and restrict protected functions to authenticated users.
FR-04	The system shall allow the user to view and update personal information and the profile image.
FR-05	The system shall allow the user to enter and update health data, lifestyle information, preferences, and personal objectives.
FR-06	The system shall allow the user to delete the account and its associated personal records through a confirmation process.
FR-07	The system shall display a daily home dashboard summarizing the enabled health and lifestyle modules.
FR-08	The system shall provide direct access from the home dashboard to the main tracking and management functions.
FR-09	The system shall calculate applicable daily targets using the available profile and health information.
FR-10	The system shall recalculate applicable targets when relevant profile or health information changes.
FR-11	The system shall calculate and display a health score using defined health and tracker indicators.
FR-12	The system shall calculate the health score independently from motivational points and rewards.
FR-13	The system shall apply personal goals, health indicators, chronic-condition restrictions, and tracker rules through the Constraints Layer and update affected summaries when related records change.
FR-14	The system shall organize physical-activity functions into Workouts, Steps, and Active Time sections.
FR-15	The system shall request the required permission and obtain step data from the supported device pedometer service.
FR-16	The system shall display daily steps and progress toward the defined step goal.

FR-17	The system shall estimate and display distance, active time, and calories burned from steps.
FR-18	The system shall allow the user to create and update daily or weekly physical-activity goals.
FR-19	The system shall allow the user to browse suggested activities, favorite activities, and activity history.
FR-20	The system shall allow the user to add or remove favorite activities and start a session directly from a favorite activity.
FR-21	The system shall allow the user to configure a live activity session by selecting the activity type, planned duration, and intensity.
FR-22	The system shall allow the user to start, pause, resume, and stop a live activity session.
FR-23	The system shall display elapsed time, planned duration, estimated calories, and steps when applicable during a live activity session.
FR-24	The system shall support keeping the mobile screen active during a live activity session.
FR-25	The system shall allow the user to record a previously completed activity using its type, date, time, duration, and intensity.
FR-26	The system shall allow the user to view, update, or delete recorded activities and recalculate the affected activity summaries.
FR-27	The system shall estimate daily calorie and macronutrient targets using the available profile information and selected nutrition objective.
FR-28	The system shall display consumed and remaining calories and the daily progress of protein, carbohydrates, and fat.
FR-29	The system shall provide a complete nutrition breakdown containing the available macronutrient and micronutrient values.
FR-30	The system shall allow the user to record a meal manually by selecting its category, food items, and quantities.
FR-31	The system shall allow the user to search the available food database.
FR-32	The system shall provide access to recently used and favorite food items for faster meal entry.
FR-33	The system shall display meal details, including food items, quantities, calories, and available nutrient values.
FR-34	The system shall allow the user to update or delete a recorded meal.
FR-35	The system shall organize recorded meals by date and meal category and allow the user to review previous days.

FR-36	The system shall calculate daily nutrition totals from all confirmed meals and relevant beverages.
FR-37	The system shall apply applicable dietary constraints to recorded meals and nutrient totals.
FR-38	The system shall display a warning when a food or nutrient value conflicts with an applicable health restriction.
FR-39	The system shall allow the user to capture a meal image using the device camera or upload a supported image from the device.
FR-40	The system shall validate the submitted meal image before sending it to the AI food-analysis service.
FR-41	The AI food-analysis service shall identify supported food items in the submitted image.
FR-42	The AI food-analysis service shall estimate the quantity or weight of detected food items when sufficient visual information is available.
FR-43	The system shall map detected food items to the available nutrition database.
FR-44	The system shall calculate and display the estimated nutritional values of the analyzed meal.
FR-45	The system shall display relevant confidence and limitation information with the AI-generated result.
FR-46	The system shall allow the user to modify, confirm, or reject the AI-generated result before it is saved as a meal.
FR-47	The system shall calculate an initial daily hydration target using available user-profile information.
FR-48	The system shall adjust the hydration target according to applicable activity information and health constraints.
FR-49	The system shall allow the user to record consumed water.
FR-50	The system shall allow the user to record supported beverages and their consumed quantities.
FR-51	The system shall classify supported beverages and calculate their available calories, nutrients, caffeine, and hydration-related values.
FR-52	The system shall allow the user to update or delete water and beverage records.
FR-53	The system shall display consumed water, remaining amount, daily target progress, and hydration-related feedback.
FR-54	The system shall allow the user to enable and configure hydration reminders.
FR-55	The system shall include relevant beverage records in nutrition totals, reports, and applicable constraint calculations.

FR-56	The system shall allow the user to add, update, and delete sleep records.
FR-57	The system shall calculate sleep duration from the recorded sleep and wake times.
FR-58	The system shall allow the user to define a sleep-duration goal and preferred sleep and wake schedule.
FR-59	The system shall allow the user to configure sleep and wake reminders.
FR-60	The system shall display daily and weekly sleep history, goal progress, and available sleep indicators.
FR-61	The system shall allow the user to add, update, and remove a medication record.
FR-62	The system shall allow a medication to be associated with a registered chronic condition when applicable.
FR-63	The system shall allow the user to define medication dose times, frequency, duration, and applicable instructions.
FR-64	The system shall allow the user to enter refill information and configure refill reminders.
FR-65	The system shall generate reminders for scheduled medication doses.
FR-66	The system shall display medication doses scheduled for the current day.
FR-67	The system shall allow a scheduled dose to be marked as taken, skipped, or missed.
FR-68	The system shall maintain medication-dose history and calculate medication-adherence indicators.
FR-69	The system shall not recommend changing a medication dose or treatment plan.
FR-70	The system shall allow the user to register, update, and remove a supported chronic condition.
FR-71	The system shall allow the user to enter and update the required condition-related information.
FR-72	The system shall allow the user to record supported health readings associated with a registered condition.
FR-73	The system shall display the history and available trends of condition-related readings.
FR-74	The system shall generate applicable nutrition, hydration, activity, medication, and lifestyle constraints from registered condition information.
FR-75	The system shall display a warning when recorded data or behavior conflicts with an applicable condition-related constraint.
FR-76	The system shall adjust relevant goals or tracker targets when an approved condition rule requires an adjustment.

FR-77	The system shall display a condition summary containing its readings, medications, restrictions, and progress indicators.
FR-78	The system shall display the trackers and medications associated with a registered chronic condition.
FR-79	The system shall allow the user to create a measurable personal health or lifestyle goal.
FR-80	The system shall allow the user to define the goal category, target value, start date, duration, and completion criteria.
FR-81	The system shall allow the user to configure reminders related to a personal goal.
FR-82	The system shall update goal progress using linked tracker data or confirmed manual progress.
FR-83	The system shall allow the user to update, complete, archive, or delete a personal goal while preserving relevant completed-goal history.
FR-84	The system shall allow the user to select a supported unhealthy behavior and record its baseline frequency, timing, and current level.
FR-85	The system shall allow the user to select gradual reduction or complete quitting as the behavior-change objective.
FR-86	The system shall create a structured behavior-change plan with measurable stages and a defined duration.
FR-87	The system shall allow the user to record daily adherence to the behavior-change plan.
FR-88	The system shall allow the user to record a relapse without deleting previously achieved progress.
FR-89	The system shall display habit progress, adherence history, applicable streaks, and relapse information.
FR-90	The system shall provide configurable habit reminders and supportive motivational feedback.
FR-91	The system shall connect a habit plan with an affected tracker when the behavior influences another supported health domain.
FR-92	The system shall allow the user to update, pause, resume, complete, or cancel a habit plan while preserving its relevant history.
FR-93	The system shall provide centralized settings for notifications and reminders.
FR-94	The system shall allow the user to enable or disable notifications for each supported module.
FR-95	The system shall allow the user to configure notification times, frequencies, schedules, and quiet hours.

FR-96	The system shall permit approved critical health-related alerts to bypass quiet hours.
FR-97	The system shall detect unavailable notification permission and guide the user to the appropriate device settings.
FR-98	The system shall schedule supported notifications locally and synchronize or restore them after relevant application, device, or preference changes.
FR-99	The system shall award motivational points for defined user actions while preventing duplicate rewards for the same confirmed action.
FR-100	The system shall maintain point-transaction history and display applicable streaks, daily completion indicators, achievements, and motivational-feed events.
FR-101	The system shall generate daily, weekly, and periodic summaries from verified health and lifestyle records.
FR-102	

Figure 7: Functional Requirements of the VitaMate System

4.6 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational conditions that the VitaMate system must satisfy. Unlike functional requirements, which describe the services provided by the system, non-functional requirements specify the expected levels of performance, security, privacy, usability, reliability, maintainability, compatibility, data integrity, and health-related safety.

Each requirement is expressed using a measurable acceptance criterion to support later verification and evaluation.

ID	Category	Non-Functional Requirement	Measurable Acceptance Criterion
NFR-01	Performance	The system shall provide responsive access to the core non-AI functions under normal operating conditions.	At least 95% of the tested core API requests shall complete within 2 seconds in the defined test environment.
NFR-02	Performance	The mobile interface shall remain responsive while retrieving or processing system data.	Loading operations shall display a progress indicator, and the interface shall not become unresponsive during the defined acceptance tests.
NFR-03	Security	The system shall restrict	All tested requests without

		protected functions and records to authenticated users.	valid authentication shall be rejected with an appropriate response.
NFR-04	Security	User passwords shall be protected using a secure password-hashing mechanism.	No password shall be stored or transmitted as plain text during the defined security inspection.
NFR-05	Communication Security	Communication with externally accessible backend services shall use a protected connection.	Client-server communication shall use HTTPS in the deployed environment.
NFR-06	Privacy	A user shall access only the personal and health-related records associated with that account.	All attempted cross-user data-access cases in the authorization test set shall be rejected.
NFR-07	Usability	The system shall provide clear and consistent navigation for non-technical users.	Each defined core task shall be reachable within no more than five main user actions after entering its related module.
NFR-08	Usability	The system shall present understandable validation, warning, and error messages.	Each tested invalid or failed operation shall display a message that identifies the problem and the available corrective action.
NFR-09	Reliability	Successfully saved user records shall remain available after restarting the application or restoring the connection.	All committed records in the persistence test set shall remain available after application restart and reconnection.
NFR-10	Reliability	Failed or repeated requests shall not create duplicate or partially stored records.	Each retry scenario shall result in no more than one valid confirmed record and no incomplete record.
NFR-11	Data Integrity	Dashboard values, tracker totals, and reports shall remain consistent with their source records.	All calculated values in the regression test dataset shall match the verified underlying records after creation, update, and deletion operations.

NFR-12	Maintainability	The system shall maintain clear separation between frontend features, backend services, data access, and intelligent services.	The project structure shall preserve modular boundaries, and static analysis shall report no unresolved critical errors before final delivery.
NFR-13	Compatibility	The mobile application shall operate correctly in the defined Android testing environment.	The main acceptance workflows shall pass on the selected Android emulator and target physical device.
NFR-14	Stability	The application shall remain stable during the execution of the main user workflows.	No unexpected application termination shall occur while completing the defined end-to-end acceptance scenario.
NFR-15	Notification Reliability	Scheduled reminders shall remain consistent with the user's saved preferences.	The notification test set shall confirm correct scheduling, cancellation, and rescheduling after relevant preference or application changes.
NFR-16	Health Safety	The system shall present health-related outputs as supportive information rather than medical diagnosis or treatment.	All relevant interfaces shall include the approved safety wording and shall not display diagnostic or medication-changing instructions.
NFR-17	AI Transparency	AI-generated food-analysis results shall communicate uncertainty and remain subject to user control.	The user shall be able to review, edit, reject, or confirm every AI-generated meal result before it changes a confirmed nutrition record.
NFR-18	Recoverability	The system shall handle service or connection failures without losing previously confirmed user data.	Previously committed records shall remain unchanged after every simulated API or connection failure in the recovery test set.

Figure 8: Non-Functional Requirements of the VitaMate System

The non-functional requirements form the quality baseline against which the implemented system will be tested in Chapter 7. Numerical results and evidence will be reported only after the corresponding performance, security, usability, compatibility, and reliability tests have been completed.

4.7 Requirements Prioritization Using MoSCoW

The MoSCoW method was used to prioritize the functional requirements according to their importance to the final VitaMate release. The classification supports scope control and ensures that essential functions are completed and validated before optional enhancements.

The prioritization does not represent implementation status. A requirement may have been initiated during Senior Project 1 and completed or refined during Senior Project 2 while retaining the same priority.

Priority	Requirement IDs or Scope	Rationale
Must	FR-01–FR-18, FR-21–FR-23, FR-25–FR-31, FR-33–FR-98, FR-101–FR-104, and FR-107–FR-108	These requirements provide the essential account, profile, dashboard, tracking, AI food-analysis, hydration, sleep, medication, chronic-condition, goal, habit, notification, reporting, data-safety, and user-control functions required for the final system.
Should	FR-19, FR-20, FR-24, FR-32, and FR-99	These requirements improve usability and engagement through activity suggestions, favorites, quick session access, screen-control support, faster food entry, and motivational points. They are important but do not prevent the core trackers from operating.
Could	FR-100, FR-105, and FR-106	These requirements provide additional motivational-feed and intelligent-recommendation functions. They add value when sufficient time and validated data are available but must not delay core integration, stability, or testing.
Won't in the Current Release	Medical diagnosis, treatment prescription, medication-dose modification, physician or hospital dashboards, unrestricted wearable integration, fully automatic saving of unconfirmed AI results, and clinical decision support	These functions are outside the approved academic scope or require additional clinical validation, infrastructure, specialists, or development resources.

Figure 9 : MoSCoW Prioritization of VitaMate Requirements

Must requirements constitute the minimum acceptable functional release and receive the highest implementation and testing priority. Should requirements are included when the essential system remains stable and complete. Could requirements are considered after the Must and Should requirements have been verified.

Won't requirements define explicit project boundaries rather than permanently rejected features. They may be considered in future versions after further technical, medical, security, and usability evaluation.

4.8 UML Requirements Models

Unified Modeling Language (UML) models are used to represent the main interactions between the user and the VitaMate system in a structured and understandable form. They complement the textual requirements by illustrating the system boundaries, user goals, activity flows, decisions, and interactions among the participating system components.

The UML requirements modeling of VitaMate includes an overall use case diagram, detailed specifications for the main use cases, activity diagrams for selected important processes, and sequence diagrams for the main use cases. These models are based on the final approved system scope and therefore include functions developed during Senior Project 1 as well as functions integrated, extended, or refined during Senior Project 2.

The models focus on the behavior expected from the final system rather than the implementation status of individual functions. Detailed architectural components, database entities, application interfaces, and implementation structures are presented separately in the system design and implementation chapters.

4.8.1 Main Use Cases

The main use cases represent the principal goals and services that a user can access through the VitaMate application. Since the system includes several interconnected health and lifestyle modules, related functions were grouped into seven high-level use cases. This grouping provides complete coverage of the approved system scope while maintaining a clear and readable requirements model.

Account creation, user authentication, profile maintenance, and health-profile configuration are grouped within UC-01 because they collectively provide the access and personalization foundation required by the remaining system functions. Other detailed operations, including AI-result validation, health-constraint processing, notification delivery, progress recalculation, and data synchronization, are described later in the detailed use case specifications and workflow diagrams rather than being represented as independent high-level use cases.

Use Case ID	Use Case Name	Main Scope
-------------	---------------	------------

UC-01	Manage Account and Health Profile	Creating and managing the user account, authenticating the user, updating personal and health information, configuring preferences, and maintaining the health profile used for personalization.
UC-02	Manage Daily Health Tracking	Recording and reviewing physical activity, steps, active time, hydration, beverages, and sleep data; managing related targets and reminders; and monitoring daily progress.
UC-03	Manage Nutrition and AI-Assisted Meals	Recording meals manually, searching food items, reviewing nutrient intake, analyzing meal images, reviewing AI-generated results, and updating nutrition totals after user confirmation.
UC-04	Manage Medications and Chronic Conditions	Managing medications, dose schedules, adherence records, chronic conditions, health readings, restrictions, and condition-related warnings.
UC-05	Manage Goals and Unhealthy Habits	Creating personal goals, tracking progress, establishing behavior-change plans, recording adherence and relapse, and connecting goals or habit plans with relevant health trackers.
UC-06	Manage Notifications and Motivational Support	Configuring notification preferences and schedules, receiving reminders, reviewing motivational points and streaks, and interacting with motivational feedback and progress events.
UC-07	View Reports and Supportive Guidance	Viewing the home dashboard, health score, daily and periodic reports, progress trends, coordinated health summaries, and constraints-aware supportive recommendations.

Figure 10: Main Use Cases of the VitaMate System

The seven main use cases provide high-level coverage of the user-facing scope of VitaMate. They represent the major functional areas of the application without displaying detailed internal processing or technical components. The internal behavior of each use case is documented through the following use case specifications, activity diagrams, and sequence diagrams.

4.8.2 Overall Use Case Diagram

The overall use case diagram provides a high-level representation of the main interactions between the user and the VitaMate system. The **User** is the primary actor and interacts directly with the seven main use cases identified in Table 4.7.

The diagram focuses on the major goals that the user can accomplish through the application. Detailed internal functions, technical components, system services, and optional workflow relationships are intentionally omitted to preserve clarity and readability. These details are

documented separately in the use case specifications, activity diagrams, sequence diagrams, and system design chapter.

The system boundary labeled **VitaMate System** separates the services provided by the application from the external user. Each use case inside the system boundary represents a major functional area rather than an individual screen, button, or implementation component.

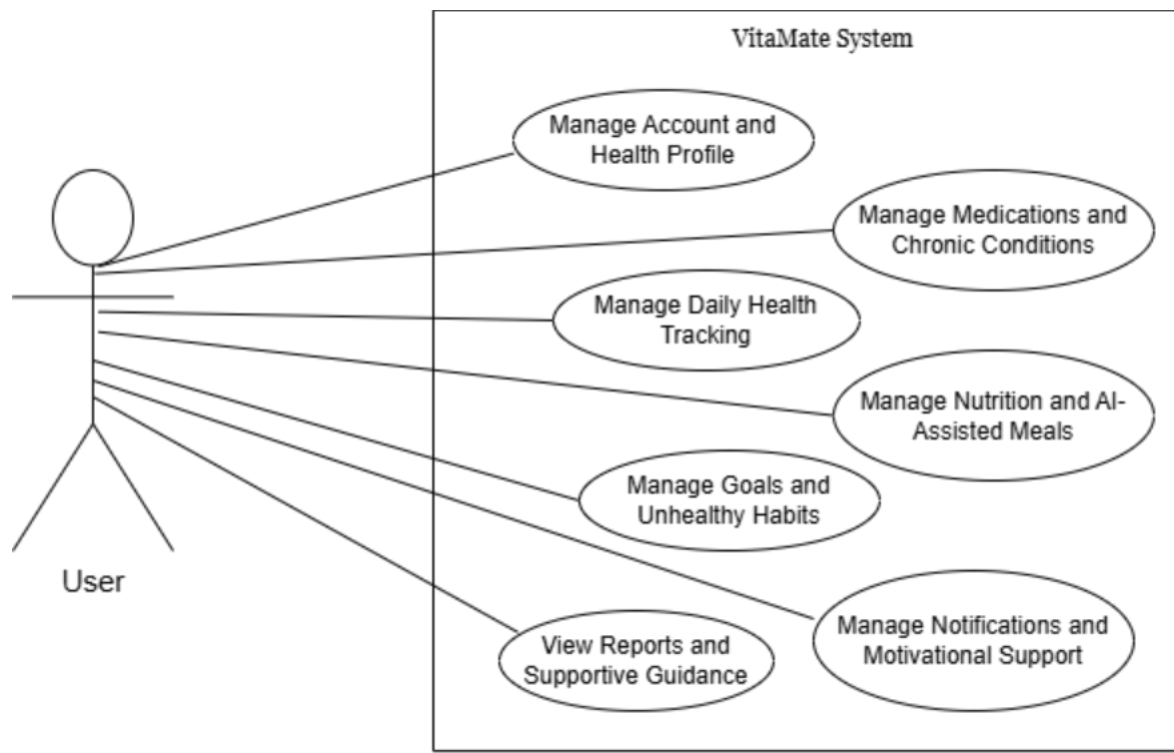


Figure 11: High-Level Use Case Diagram

Figure 4.1 summarizes the principal functional areas available to the user through VitaMate. These areas include account and health-profile management, daily health tracking, nutrition and AI-assisted meal management, medication and chronic-condition support, goals and unhealthy-habit management, notifications and motivational support, and integrated progress review and supportive guidance.

The diagram also reflects the integrated nature of VitaMate by grouping related functions under unified user goals. More detailed scenarios, alternative flows, validation conditions, and system responses are presented in the following use case specifications.

4.8.3 Use Case Specifications

The following use case specifications provide a structured description of the seven main interactions between the user and the VitaMate system. Each specification defines the purpose of the use case, participating actors, triggering event, required conditions, inputs, outputs, main workflow, and relevant alternative scenarios.

The specifications expand the high-level model presented in Figure 4.1 without introducing implementation-level details. Internal components, APIs, database operations, and service interactions are presented later through the sequence diagrams and the system design chapter.

4.8.3.1 UC-01 — *Manage Account and Health Profile*

This use case describes how the user creates and accesses an account and maintains the personal and health-related information required by the VitaMate personalization and tracking functions.

Field	Description
Use Case ID	UC-01
Use Case Name	Manage Account and Health Profile
Primary Actor	User
Secondary Actors	None
Description	Enables the user to create and access an account and manage personal, health, lifestyle, and preference information.
Trigger	The user selects registration, sign-in, profile viewing, profile editing, or account deletion.
Preconditions	The system is available. An existing account is required for sign-in, and authentication is required for profile management or account deletion.
Postconditions	The account or session is created, or the requested profile changes are stored. Relevant personalized targets are recalculated when necessary.
Inputs	Authentication credentials, personal and health information, lifestyle preferences, objectives, and optional profile image.
Outputs	Authentication result, updated profile, validation or confirmation message, and recalculated targets when applicable.
Main Flow	1. The user opens the account or profile function.2. The user selects the required operation.3. The user enters or updates the required data.4. The system validates the data.5. The system completes the operation and stores the approved changes.6. The system displays the updated result or destination

	screen.
Alternative Flow	A1: Invalid or incomplete data is rejected with a corrective message. A2: Duplicate registration information is rejected. A3: Invalid credentials prevent sign-in and allow retry. A4: An expired session requires re-authentication. A5: Account deletion requires explicit user confirmation.
Frequency	High
Priority	High — Must

Figure 12: Use Case Specification for UC-01 — Manage Account and Health Profile

UC-01 provides the account and personalization foundation required by the remaining VitaMate functions.

4.8.3.2 UC-02 — Manage Daily Health Tracking

Field	Description
Use Case ID	UC-02
Use Case Name	Manage Daily Health Tracking
Primary Actor	User
Secondary Actors	Device activity and notification services
Description	Enables the user to track physical activity, steps, active time, hydration, beverages, and sleep within the related daily trackers.
Trigger	The user opens a tracker, starts an activity session, records daily data, or responds to a reminder.
Preconditions	The user is authenticated. Required device permissions must be granted for automatic step tracking.
Postconditions	Valid records are stored, and the related totals, goals, history, and progress indicators are updated.
Inputs	Step data, activity details, water or beverage quantity, sleep and wake times, targets, and reminder settings.
Outputs	Updated tracker records, daily progress, history, calculated indicators, and confirmation or validation messages.
Main Flow	1. The user selects the required tracker.2. The system displays current progress and available actions.3. The user records data, starts an activity session, or allows supported device data to be retrieved.4. The system validates and stores the data.5. The system recalculates the affected totals and progress.6. The updated results are displayed.
Alternative Flow	A1: If permission is denied, the system displays guidance and disables automatic step retrieval. A2: Invalid data is rejected with a corrective message. A3: Updated or deleted records cause the related

	summaries to be recalculated. A4: A device or connection failure displays an error and allows retry without duplicating data.
Frequency	High
Priority	High — Must

Figure 13: Use Case Specification for UC-02 — Manage Daily Health Tracking

UC-02 combines related activity, hydration, beverage, and sleep tracking operations within one high-level daily health use case.

4.8.3.3 UC-03 — Manage Nutrition and AI-Assisted Meals

Field	Description
Use Case ID	UC-03
Use Case Name	Manage Nutrition and AI-Assisted Meals
Primary Actor	User
Secondary Actors	None
Description	Enables the user to record and manage meals manually or through AI-assisted food-image analysis and review daily nutrient intake.
Trigger	The user opens the nutrition module, adds a meal, or submits a meal image.
Preconditions	The user is authenticated, and the nutrition module is available. A valid image is required for AI analysis.
Postconditions	The confirmed meal is stored, and the related calorie, nutrient, and daily-progress values are updated.
Inputs	Meal category, food items, quantities, meal image, and user corrections or confirmation.
Outputs	Meal details, estimated nutrients, AI confidence or limitation messages, dietary warnings, and updated nutrition totals.
Main Flow	1. The user opens the nutrition module.2. The user selects manual entry or AI-assisted analysis.3. The system receives and validates the entered data or image.4. The system calculates or estimates the meal nutrients.5. The user reviews and confirms the result.6. The system stores the meal and updates the daily totals.
Alternative Flow	A1: Invalid meal data or image is rejected with guidance. A2: Low-confidence AI results are displayed with limitation messages. A3: The user modifies or rejects the AI result before saving. A4: A service failure displays an error and allows retry without saving an incomplete meal.
Frequency	High

Priority	High — Must
-----------------	-------------

Figure 14: Use Case Specification for UC-03 — Manage Nutrition and AI-Assisted Meals

UC-03 ensures that AI-generated nutrition data remains subject to user review and confirmation before affecting the user's records.

4.8.3.4 UC-04 — Manage Medications and Chronic Conditions

Field	Description
Use Case ID	UC-04
Use Case Name	Manage Medications and Chronic Conditions
Primary Actor	User
Secondary Actors	None
Description	Enables the user to manage medications, dose schedules, chronic conditions, health readings, adherence, and related restrictions.
Trigger	The user opens the medication or chronic-condition module or responds to a scheduled dose reminder.
Preconditions	The user is authenticated.
Postconditions	Valid records are stored, and the related schedules, adherence indicators, constraints, and summaries are updated.
Inputs	Medication details, dose schedules, refill information, dose status, condition information, and health readings.
Outputs	Medication schedule, dose reminders, adherence history, condition summary, applicable restrictions, and warning messages.
Main Flow	1. The user opens the required module.2. The user adds or updates medication, condition, reading, or dose status.3. The system validates and stores the information.4. The system updates reminders, adherence indicators, and applicable constraints.5. The system displays the updated records and summaries.
Alternative Flow	A1: Invalid or incomplete data is rejected with guidance. A2: The user marks a dose as taken, skipped, or missed. A3: A medication may be linked to a registered condition. A4: Conflicting data generates a warning without changing the prescribed treatment. A5: A failed operation allows retry without creating duplicate records.
Frequency	High
Priority	High — Must

Figure 15: Use Case Specification for UC-04 — Manage Medications and Chronic Conditions

UC-04 supports medication organization and condition-aware tracking without recommending medication-dose or treatment changes.

4.8.3.5 UC-05 — Manage Goals and Unhealthy Habits

Field	Description
Use Case ID	UC-05
Use Case Name	Manage Goals and Unhealthy Habits
Primary Actor	User
Secondary Actors	None
Description	Enables the user to create health goals, manage unhealthy-habit reduction or quitting plans, and monitor progress.
Trigger	The user creates or updates a goal, habit plan, or progress record.
Preconditions	The user is authenticated.
Postconditions	The goal or habit record is stored, and related progress, streaks, trackers, and motivational indicators are updated.
Inputs	Goal or habit type, target, duration, frequency, plan details, progress, adherence, and relapse records.
Outputs	Current status, progress percentage, streaks, motivational feedback, and validation messages.
Main Flow	1. The user opens the goals or habits module.2. The user creates or selects a goal or habit plan.3. The user enters the required details.4. The system validates and stores the data.5. The user records progress or adherence.6. The system updates and displays the results.
Alternative Flow	A1: Invalid data is rejected with guidance. A2: The user edits, pauses, completes, or deletes a plan. A3: A relapse is recorded without deleting previous progress. A4: Conflicting targets generate a warning before confirmation.
Frequency	High
Priority	High — Must

Figure 16: Use Case Specification for UC-05 — Manage Goals and Unhealthy Habits

UC-05 supports gradual behavior change while preserving the user's progress history.

4.8.3.6 UC-06 — Manage Notifications and Motivational Support

Field	Description
Use Case ID	UC-06
Use Case Name	Manage Notifications and Motivational Support
Primary Actor	User
Secondary Actors	Device notification service
Description	Enables the user to manage notification preferences and receive reminders, points, streaks, and motivational feedback.
Trigger	The user updates notification settings, completes a tracked action, or a scheduled reminder becomes due.
Preconditions	The user is authenticated, and notification permission is granted for local delivery.
Postconditions	Preferences are stored, applicable reminders are scheduled, and motivational indicators are updated.
Inputs	Notification preferences, reminder times, quiet hours, tracker events, completed goals, and user actions.
Outputs	Reminders, alerts, motivational messages, points, streaks, achievements, and updated notification status.
Main Flow	1. The user opens notification or motivation settings.2. The user configures preferences and schedules.3. The system validates and stores the settings.4. The system schedules applicable notifications.5. Relevant user actions update points, streaks, and motivational feedback.6. The system displays the updated results.
Alternative Flow	A1: If notification permission is denied, the system displays guidance. A2: Quiet hours delay non-critical notifications. A3: Invalid schedules are rejected with a corrective message. A4: Duplicate events do not generate duplicate points or notifications.
Frequency	High
Priority	High — Must

Table 4.13. Use Case Specification for UC-06 — Manage Notifications and Motivational Support

UC-06 supports engagement while keeping notification timing and preferences under user control.

4.8.3.7 UC-07 — View Reports and Supportive Guidance

Field	Description
Use Case ID	UC-07
Use Case Name	View Reports and Supportive Guidance
Primary Actor	User
Secondary Actors	None
Description	Enables the user to review dashboards, health indicators, reports, trends, and personalized supportive guidance.
Trigger	The user opens the home dashboard, reports, history, or guidance section.
Preconditions	The user is authenticated. Relevant records must exist for detailed analysis.
Postconditions	The requested summaries and guidance are displayed without modifying user records automatically.
Inputs	Profile data, tracker records, goals, conditions, medications, constraints, and selected reporting period.
Outputs	Dashboard summaries, health score, progress charts, trends, warnings, and supportive recommendations.
Main Flow	1. The user selects a dashboard or report.2. The system retrieves the relevant records.3. The system calculates summaries and trends.4. Applicable health constraints are considered.5. The system displays reports and supportive guidance.
Alternative Flow	A1: Missing data produces a limited summary with guidance to complete tracking. A2: Conflicting or risky data produces a warning. A3: A loading failure displays an error and allows retry. A4: Recommendations requiring changes are applied only after user confirmation.
Frequency	High
Priority	High — Must

Figure 17: Use Case Specification for UC-07 — View Reports and Supportive Guidance

UC-07 provides an integrated view of the user's progress while keeping motivational points separate from the calculated health score and avoiding diagnostic or treatment decisions.

4.8.4 Activity Diagrams

Activity diagrams are used to describe the workflow, decisions, and alternative paths of selected VitaMate processes. The diagrams focus on four important workflows: AI-assisted meal

logging, live activity tracking, medication-dose logging, and constraints-aware health coordination.

4.8.4.1 AI-Assisted Meal Analysis

This activity diagram shows the AI-assisted meal-analysis workflow from image submission to nutritional-value calculation. The system segments the food regions, identifies dish and ingredient candidates, estimates portion weights, and presents the combined result for user review and confirmation.

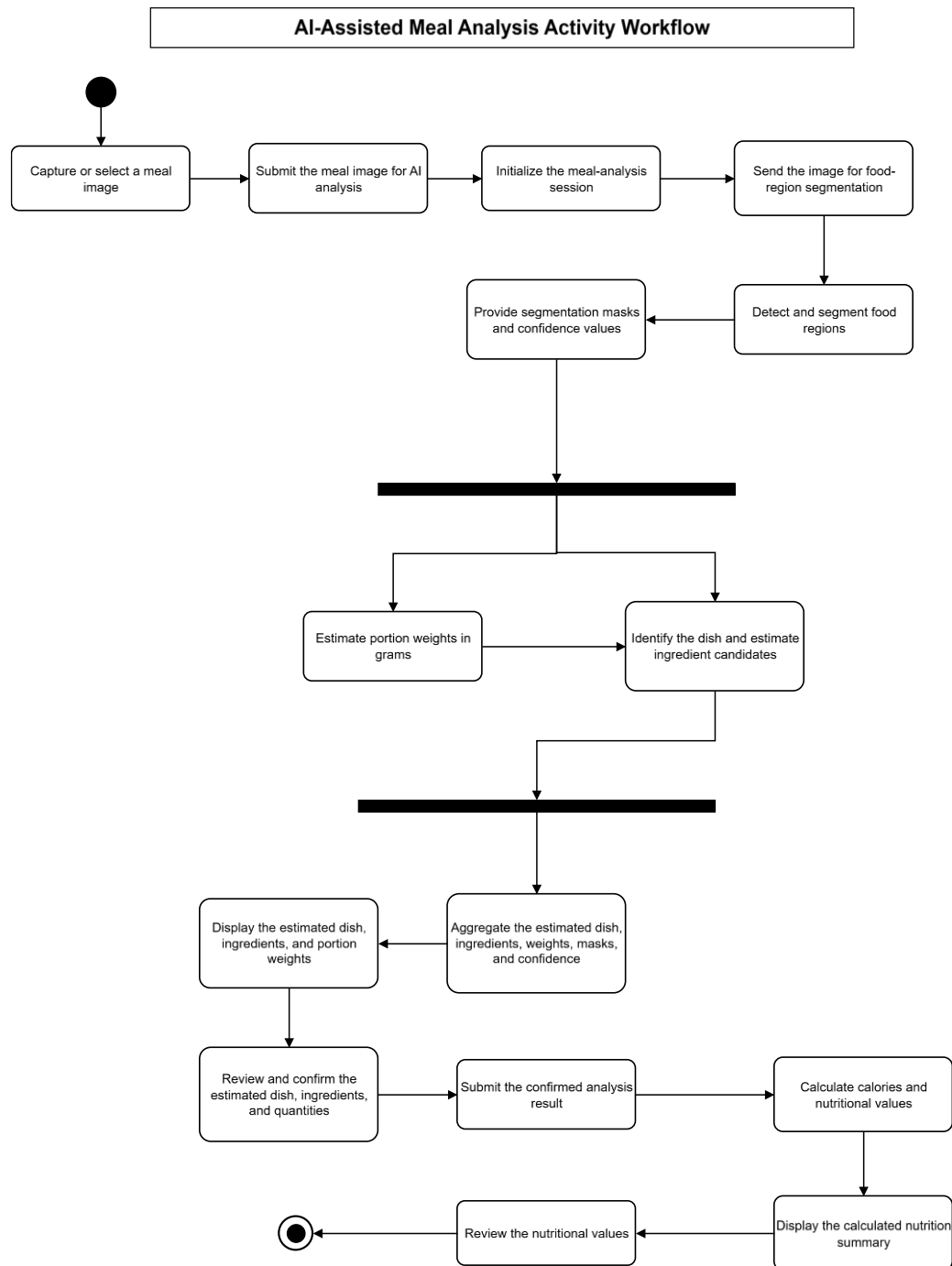


Figure 18: Activity Diagram for AI-Assisted Meal Analysis

Figure 18 demonstrates that calories and nutritional values are calculated only after the user reviews and confirms the estimated dish, ingredients, and quantities.

4.8.4.2 Unhealthy-Habit Cessation

This activity diagram shows how VitaMate creates or updates a habit-cessation plan and processes the user's daily status. The workflow distinguishes between a successful habit-free day and a relapse while preserving previous progress history.

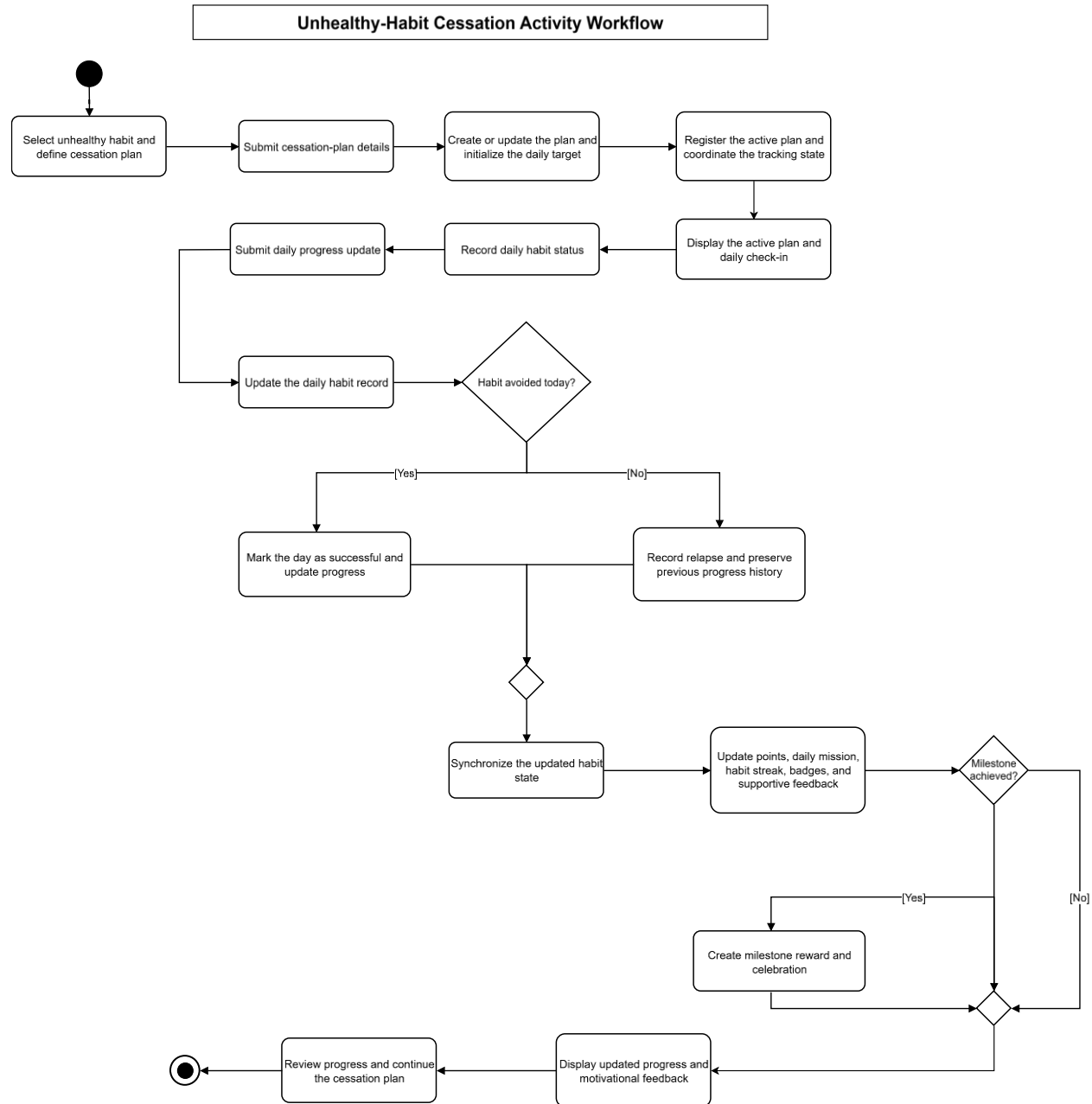


Figure 19: Activity Diagram for Unhealthy-Habit Cessation

Figure 19 demonstrates how each daily update affects the habit progress, motivational points, missions, streaks, badges, and milestone feedback.

4.8.4.3 Health-Tracker Recording and Motivation-State Update

This activity diagram shows how a health-related action is validated and processed through the selected tracker. Invalid data is returned for correction, while a successful update triggers the recalculation of tracker progress and the related motivational state.

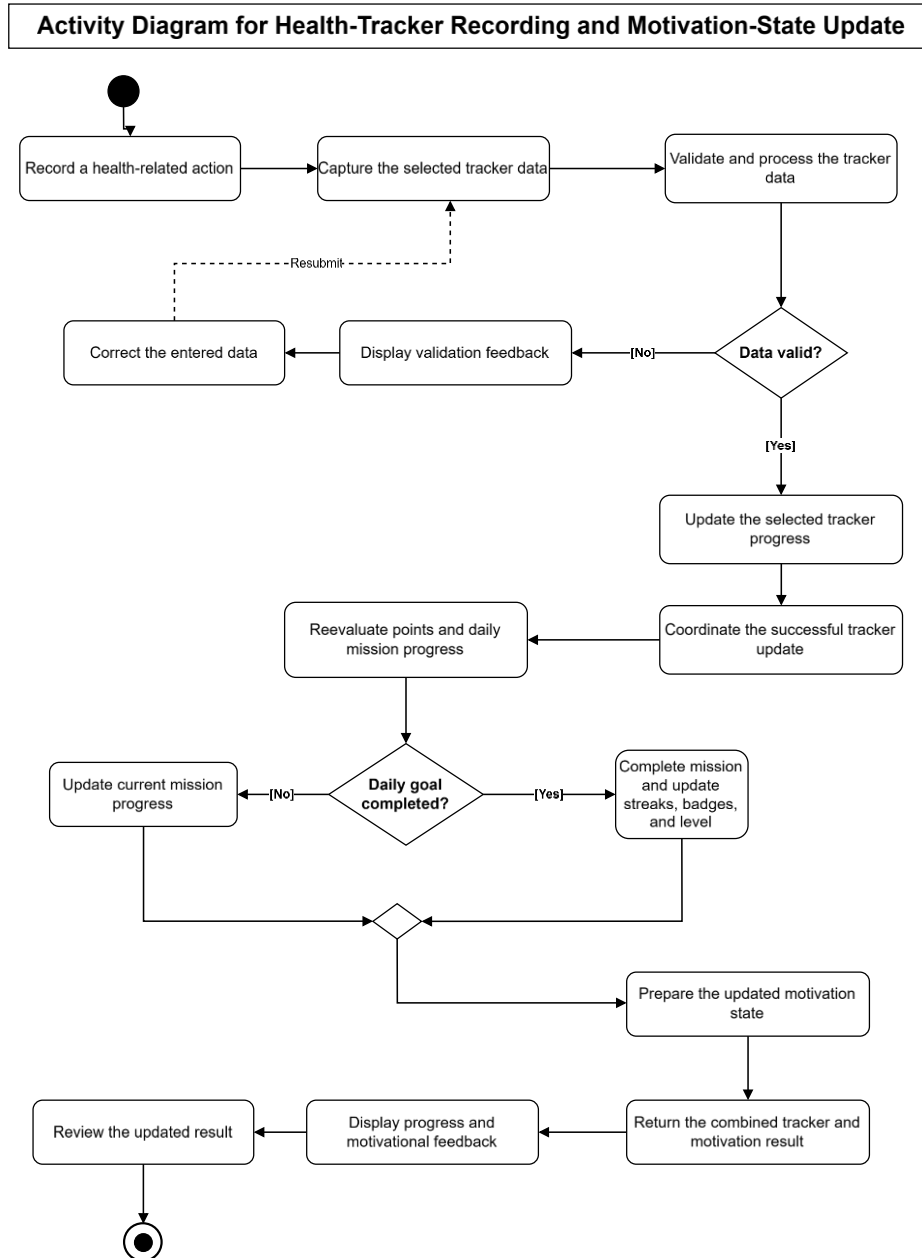


Figure 20: Activity Diagram for Health-Tracker Recording and Motivation-State Update

Figure 20 demonstrates how VitaMate updates mission progress, points, streaks, badges, and level status after a valid tracker record is successfully processed.

4.8.5 Sequence Diagrams

Sequence diagrams illustrate the ordered interactions between the user interface, backend services, supporting components, and data layer during important VitaMate workflows.

4.8.5.1 AI-Assisted Meal Analysis

This sequence diagram shows the AI-assisted meal-analysis workflow from image submission to confirmed nutrition calculation. The system analyzes the image, presents editable estimates, and calculates the final nutritional values only after user confirmation.

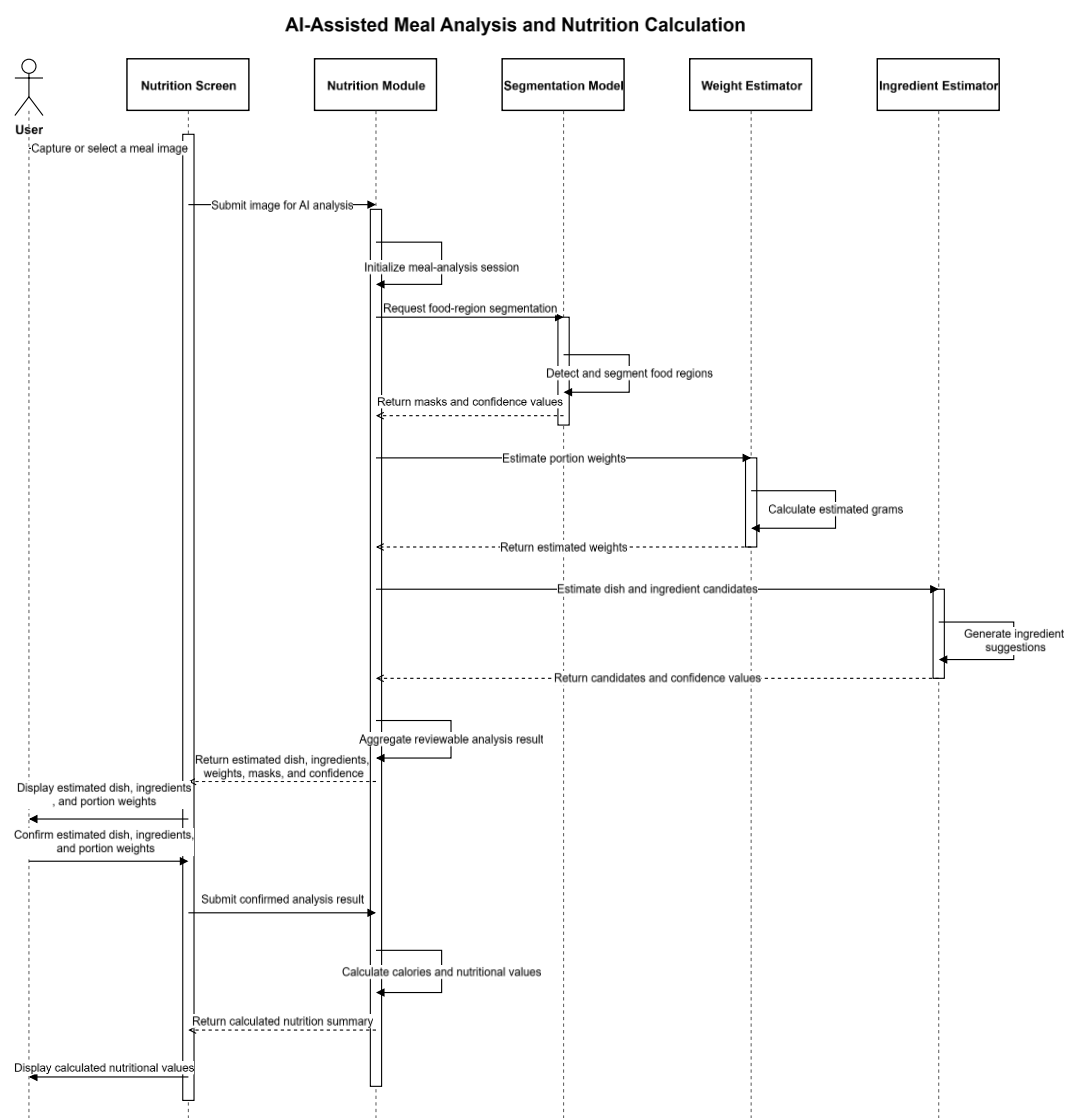


Figure 21: Sequence Diagram for AI-Assisted Meal Analysis

Figure shows the interaction between the user, VitaMate application, nutrition services, AI-analysis components, and data layer while preserving user control over uncertain results.

4.8.5.2 Health-Tracker and Motivation Update

This sequence diagram shows how a submitted health record is validated, stored, and reflected in the user's motivational progress. Motivation points, missions, streaks, and badges are updated only after successful tracker processing.

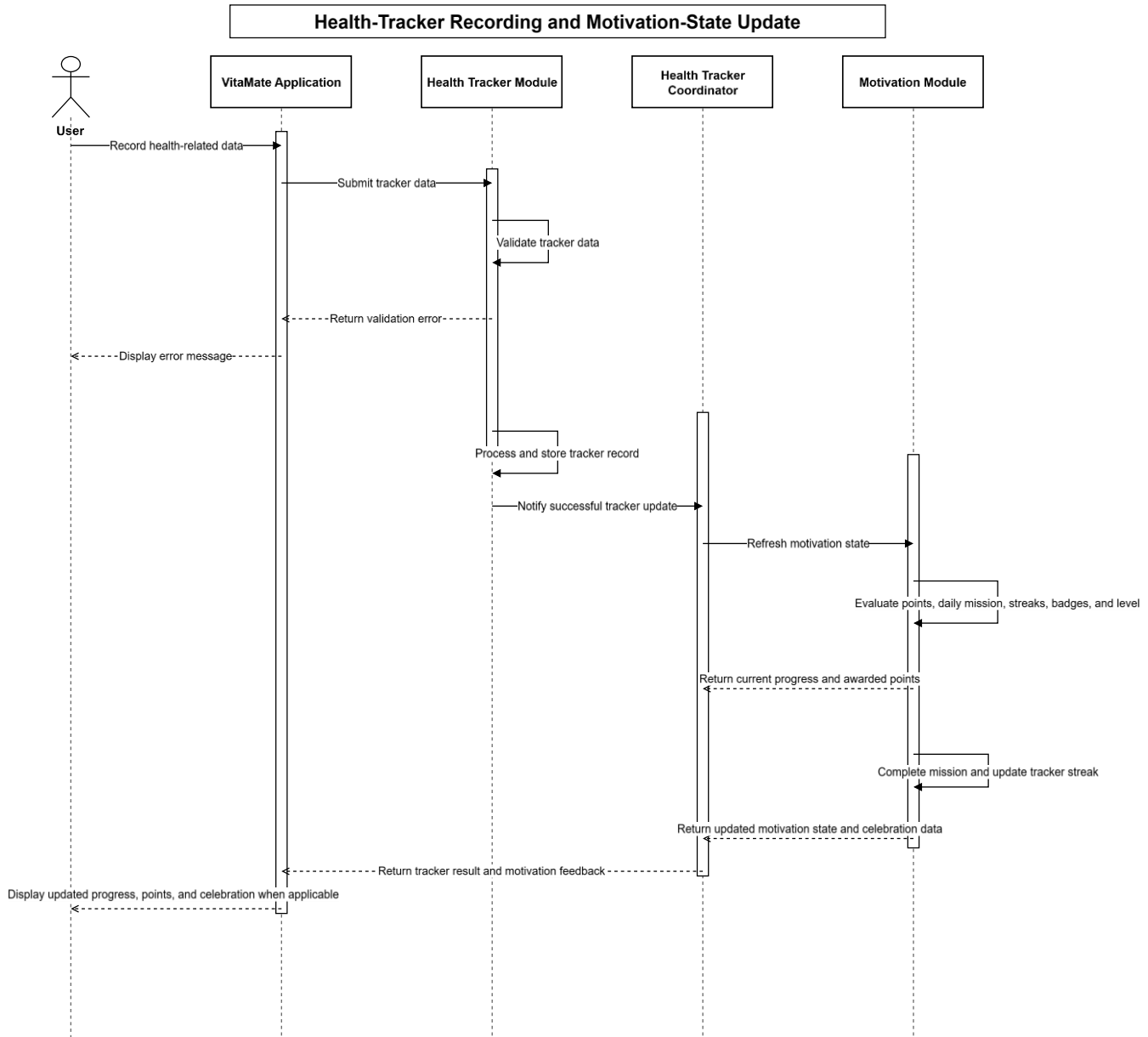


Figure 22 :Sequence Diagram for Health-Tracker and Motivation Update

Figure 22 shows the interaction between the user, VitaMate application, tracker services, motivation services, and data layer during record processing and feedback generation.

4.8.5.3 Unhealthy-Habit Cessation and Motivation Update

This sequence diagram shows how VitaMate creates or updates a habit-cessation plan and processes daily habit-free or relapse records. Each valid update modifies the adherence history and triggers the recalculation of missions, points, streaks, badges, and motivational feedback.

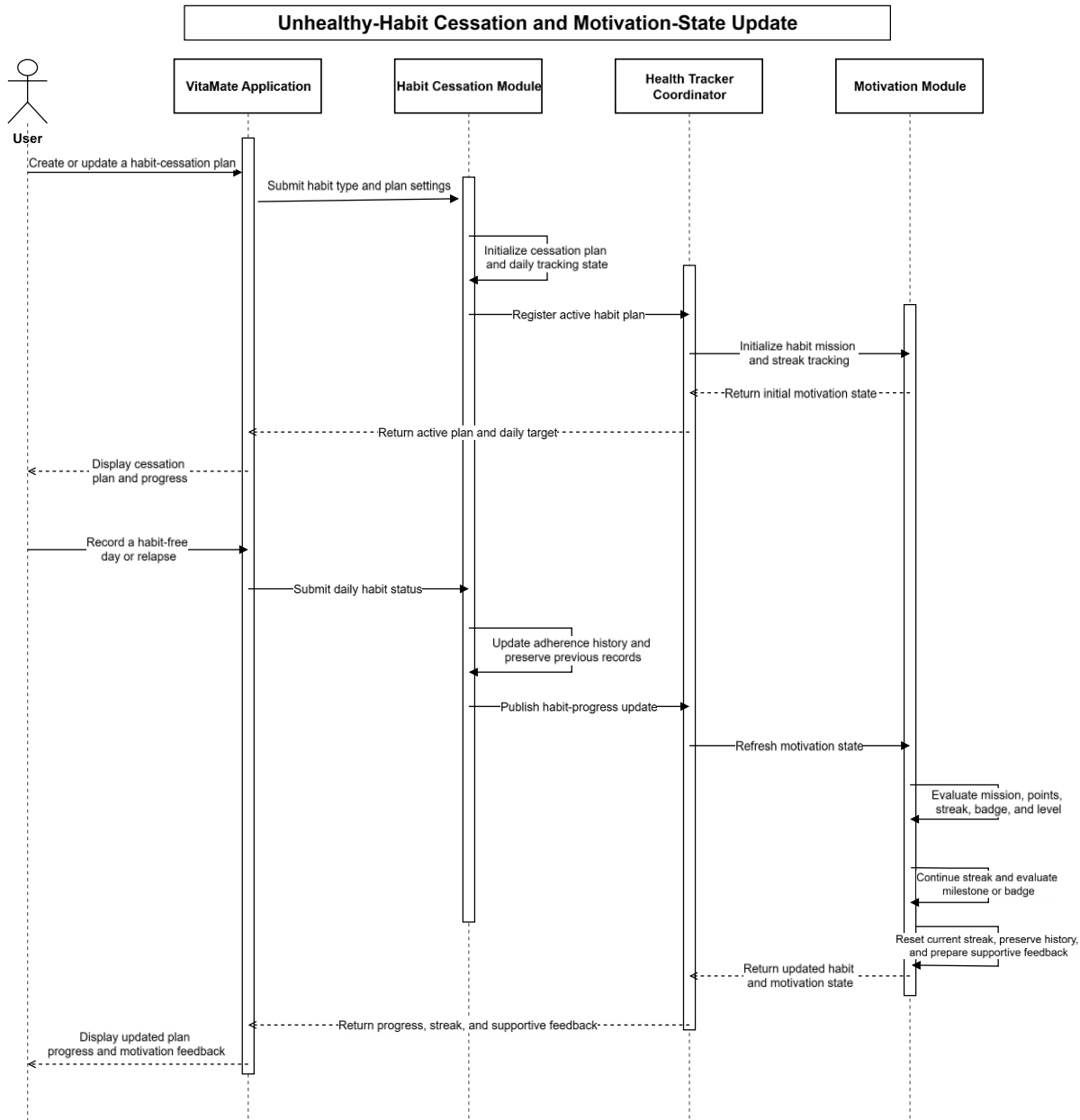


Figure 23 :Sequence Diagram for Unhealthy-Habit Cessation and Motivation Update

Figure 4.8 shows the interaction between the user, VitaMate application, habit-cessation module, health-tracker coordinator, and motivation module. Previous progress remains preserved, while the current streak and supportive feedback are updated according to the recorded status.

The sequence diagrams presented in this section clarify the interactions among the user interface, backend services, supporting system components, and data layer during the execution of VitaMate’s main use cases. They demonstrate how user requests are validated, processed, and stored, while highlighting cross-module coordination through the Constraints Layer. The diagrams also emphasize important system principles, including user confirmation of AI-generated results, preservation of historical progress, controlled notification delivery, and the provision of supportive guidance without medical diagnosis or automatic treatment modification.

4.9 Agile Requirements Models

VitaMate was developed using the Agile Scrum methodology. The project work was divided into short development iterations, with each sprint focusing on a defined set of functional, integration, testing, and refinement objectives. The Agile artifacts presented in this section document how requirements were organized, prioritized, implemented, and reviewed throughout the second development phase.

4.9.1 User Stories

User stories were used to express VitaMate requirements from the user’s perspective. Each story describes a user need and the expected value provided by the system.

ID	User Story
US-01	As a user, I want to manage my health profile so that VitaMate can personalize my targets and guidance.
US-02	As a user, I want to record daily health data so that I can monitor my progress over time.
US-03	As a user, I want to analyze a meal image so that I can record nutrition information more quickly.
US-04	As a user, I want to review and edit AI-generated meal results so that inaccurate information is not saved.
US-05	As a user, I want to manage medications and reminders so that I can follow my planned schedule.
US-06	As a user, I want to record chronic conditions and health readings so that relevant restrictions can be considered.
US-07	As a user, I want to set goals and track unhealthy habits so that I can monitor behavioral progress.
US-08	As a user, I want to control notification preferences and quiet hours so that reminders are delivered appropriately.
US-09	As a user, I want to view reports and trends so that I can understand my health-tracking progress.

US-10	As a user, I want coordinated warnings and supportive guidance so that conflicting health information is identified.
--------------	--

Figure 24: Representative User Stories

These user stories were further decomposed into implementation tasks and acceptance criteria during sprint planning.

4.9.2 Story Mapping

Story mapping was used to organize the representative user stories according to the principal activities performed by the VitaMate user. The backbone activities represent the main stages of the user journey, while the related stories identify the functions required to complete each activity.

The story map connects profile personalization, daily health tracking, AI-assisted meal recording, medication and chronic-condition management, notification control, progress review, and coordinated health guidance. This organization clarifies the relationships among user needs and maintains consistency between the User Stories, Product Backlog, system requirements, and implementation priorities.

Backbone Activity	User Tasks and Goals	Related User Stories	Related Product Backlog Items
Personalize the health experience	Create and maintain the health profile so that targets and guidance can reflect the user's information.	US-01	PB-01
Record daily health data	Record nutrition, hydration, activity, sleep, medication, goals, and habit-related data and follow daily progress.	US-02, US-07	PB-02, PB-07, PB-08
Analyze and confirm a meal	Submit a meal image, review the generated result, correct inaccurate information, and confirm or reject it before saving.	US-03, US-04	PB-03, PB-04
Manage medications and health conditions	Record medications, reminders, chronic conditions, and relevant health readings so that applicable restrictions can be considered.	US-05, US-06	PB-06
Control reminder	Configure notification preferences	US-08	PB-09

delivery	and quiet hours to receive reminders at appropriate times.		
Review health progress	View reports, dashboards, indicators, and trends to understand progress across the tracked health domains.	US-09	PB-11
Receive coordinated support	Receive coordinated warnings, motivational feedback, and supportive guidance while avoiding conflicting health information.	US-10	PB-05, PB-10

Figure 25: VitaMate User Story Map

The story map focuses on user-facing activities. Cross-cutting technical backlog items related to interface consistency, testing, runtime validation, security, edge cases, and documentation support all activities rather than belonging to one individual stage of

4.9.3 Product Backlog

The Product Backlog contains the principal functional and technical work items required to complete VitaMate. It was refined according to implementation progress, integration dependencies, identified defects, testing results, and feedback received during project reviews. Each item was assigned a priority and a status reflecting its verified state at the time of report preparation. An item was considered completed only when its required implementation, integration, validation, testing, and documentation conditions had been satisfied.

ID	Product Backlog Item	Priority	Status
PB-01	Complete account and health-profile management	High	In Progress
PB-02	Integrate nutrition, hydration, activity, sleep, and medication trackers	High	In Progress
PB-03	Integrate AI-assisted meal analysis with the nutrition workflow	High	In Progress
PB-04	Implement user review, editing, confirmation, and rejection of AI-generated meal results	High	In Progress
PB-05	Coordinate health modules through the Constraints Layer	High	In Progress
PB-06	Improve chronic-condition and	High	In Progress

	medication-management workflows		
PB-07	Correct step-counting and activity-session behavior	High	In Progress
PB-08	Merge habits and goals with related health trackers	Medium	In Progress
PB-09	Integrate notifications, quiet hours, and local reminder delivery	High	In Progress
PB-10	Refine motivational feedback, streaks, and points behavior	Medium	In Progress
PB-11	Stabilize reports, dashboards, and health indicators	High	In Progress
PB-12	Refine the Flutter user interface and improve usability consistency	Medium	In Progress
PB-13	Perform backend, frontend, integration, and end-to-end testing	High	In Progress
PB-14	Test the application on an Android device and Linux-based environment	High	Planned
PB-15	Resolve security, validation, permissions, and edge-case issues	High	In Progress
PB-16	Complete technical documentation and the final project report	High	In Progress

Figure 26: VitaMate Product Backlog

The Product Backlog reflects both user-facing requirements and technical work required to deliver an integrated and stable mobile health-tracking application. High-priority items were assigned to functions directly affecting system integration, data safety, AI-result confirmation, health constraints, and application reliability.

4.9.4 Sprint Backlog

The second development phase was organized into six sprint periods, beginning with Sprint 0 and ending with Sprint 5. Each sprint lasted approximately ten days and included planning, implementation, integration, testing, review, and refinement activities according to its defined objective.

Sprint	Main Objective	Selected Work Items
Sprint 0	Planning and scope refinement	Review the previous implementation, update the project scope, identify integration dependencies, refine the Product Backlog, and prepare the development

		environment.
Sprint 1	Core-module refinement	Refine profile management, improve chronic-condition interfaces, update user-data workflows, and begin notification integration.
Sprint 2	Tracker and behavior integration	Merge habits and goals with their related trackers, improve medication workflows, integrate notification preferences, and refine tracker interactions.
Sprint 3	AI and activity integration	Integrate AI-assisted meal analysis with the nutrition workflow, support user review and confirmation of generated results, and correct step-counting and activity-session behavior.
Sprint 4	Coordination and reporting	Stabilize reports and dashboards, improve Constraints Layer coordination, refine health indicators, and address cross-module inconsistencies.
Sprint 5	System validation and finalization	Conduct backend, frontend, integration, and end-to-end testing; validate the application on an Android device and Linux-based environment; address security, validation, permissions, and edge cases; clean the codebase; complete documentation; and prepare the final presentation and delivery.

Figure 27: VitaMate Sprint Backlog

The Sprint Backlog was refined when technical dependencies, integration issues, or newly identified defects affected the planned work. In particular, AI food analysis, notification scheduling, constraints coordination, tracker stabilization, and final system validation required coordination across multiple frontend and backend components.

4.10 Requirements Analysis and Documentation

This section documents the relationships among the analyzed requirements, system models, design and implementation components, and planned verification activities. Requirements traceability is used to ensure that each requirement is represented in the system design and associated with an identifiable validation or test activity.

4.10.1 Requirements Traceability Matrix

The Requirements Traceability Matrix links the VitaMate functional and non-functional requirements to their sources, related use cases, design and implementation components, and planned verification activities. It supports consistency among requirements analysis, system design, implementation, and testing and helps identify requirements that are missing, insufficiently represented, or not covered by a planned verification activity.

4.10.1.1 Functional Requirements Traceability

Requirement ID	Requirement Description	Source	Design	Test	Status
FR-01	The system shall allow the user to create an account and authenticate securely.	Senior Project 1 SRS and Project Documents	Authentication Module	TC-01	In Progress
FR-02	The system shall allow the user to view and update personal and health-profile information.	Senior Project 1 SRS and Project Documents	Profile Module	TC-02	In Progress
FR-03	The system shall allow the user to record hydration and beverage consumption.	Senior Project 1 SRS and Project Documents	Hydration Tracking Module	TC-03	In Progress
FR-04	The system shall allow the user to record and review sleep data.	Senior Project 1 SRS and Project Documents	Sleep Tracking Module	TC-03	In Progress
FR-05	The system shall record steps and activity-related information when device data is available.	Prototype Implementation and Testing	Activity Tracking Module and Device Activity Service	TC-03	In Progress
FR-06	The system shall allow the user to start, pause, resume, and complete a live activity session.	Product Backlog Refinement	Live Activity Session Module	TC-04	In Progress
FR-07	The system shall allow the user to add meals manually.	Senior Project 1 SRS and Project Documents	Nutrition Tracking Module	TC-05	In Progress
FR-08	The system shall analyze a submitted meal image and return food and nutrition estimates.	Literature and Reference-System Analysis	Meal Image Interface and AI Analysis Service	TC-06	In Progress
FR-09	The system shall allow the user to edit, confirm, or reject AI-generated meal results before saving.	Supervisor Feedback	Meal Review and Confirmation Module	TC-07	In Progress
FR-10	The system shall calculate and display daily nutrition totals.	Team Analysis and Workshops	Nutrition Calculation Module	TC-05	In Progress

FR-11	The system shall allow the user to manage medications and dose schedules.	Senior Project 1 SRS and Project Documents	Medication Management Module	TC-08	In Progress
FR-12	The system shall generate medication reminders according to saved schedules.	Product Backlog Refinement	Medication and Notification Modules	TC-08	In Progress
FR-13	The system shall allow the user to record dose status as taken or skipped.	Team Analysis and Workshops	Medication Adherence Module	TC-08	In Progress
FR-14	The system shall allow the user to record chronic conditions and related health readings.	Senior Project 1 SRS and Project Documents	Chronic Conditions Module	TC-09	In Progress
FR-15	The system shall consider medication and chronic-condition information when generating warnings or guidance.	Supervisor Feedback	Constraints Layer	TC-09	In Progress
FR-16	The system shall allow the user to create and manage health goals.	Senior Project 1 SRS and Project Documents	Goals Module	TC-10	In Progress
FR-17	The system shall allow the user to track unhealthy habits, progress, and relapse events.	Product Backlog Refinement	Habit Tracking Module	TC-10	In Progress
FR-18	The system shall maintain streaks and motivational progress without deleting previous records after relapse.	Prototype Implementation and Testing	Motivation and Habit-History Modules	TC-10	In Progress
FR-19	The system shall allow the user to configure notification preferences and quiet hours.	Product Backlog Refinement	Notification Settings Module	TC-11	In Progress
FR-20	The system shall deliver approved reminders through the device notification service.	Prototype Implementation and Testing	Notification Service and Local Scheduler	TC-11	In Progress
FR-21	The system shall provide motivational feedback based on recorded progress.	Literature and Reference-System Analysis	Motivation Module	TC-11	In Progress
FR-22	The system shall display health reports and progress summaries for selected periods.	Senior Project 1 SRS and Project	Reporting and Dashboard Module	TC-12	In Progress

		Documents			
FR-23	The system shall combine data from relevant trackers when generating coordinated summaries.	Team Analysis and Workshops	Reporting Module and Constraints Layer	TC-12	In Progress
FR-24	The system shall display supportive guidance and warnings without producing medical diagnoses or automatically modifying treatment.	Supervisor Feedback	Guidance Interface and Constraints Layer	TC-12	In Progress
FR-25	The system shall keep health indicators separate from motivational points.	Prototype Implementation and Testing	Health-Indicator and Motivation Modules	TC-12	In Progress

Figure 28: Functional Requirements Traceability Matrix

The matrix demonstrates that every functional requirement is connected to a documented source, at least one main use case, identifiable design and implementation components, and an initial test case. The execution results and final verification evidence will be documented in the testing and evaluation chapter.

4.10.1.2 Non-Functional Requirements Traceability

Requirement ID	Requirement Description	Source	Design	Test	Status
NFR-01	The system shall provide a clear and consistent mobile user interface.	Literature and Reference-System Analysis	Flutter User Interface	Usability Testing	In Progress
NFR-02	Common user actions shall provide clear success, loading, and error feedback.	Prototype Implementation and Testing	Flutter State and Feedback Components	Functional Testing	In Progress
NFR-03	The system shall validate submitted data before storing it.	Team Analysis and Workshops	Input Validation and Backend Serializers	Validation Testing	In Progress
NFR-04	Users shall access only their own private health records.	Team Analysis and Workshops	Authentication and Authorization Design	Security Testing	In Progress
NFR-05	General reference data shall be modifiable only by authorized internal	Team Analysis and Workshops	Backend Permission Design	Permission Testing	In Progress

	roles.				
NFR-06	Communication between the mobile application and backend shall use authenticated API requests.	Team Analysis and Workshops	Authenticated API Design	API Security Testing	In Progress
NFR-07	The system shall handle unavailable device permissions or services without application failure.	Prototype Implementation and Testing	Device-Service Error Handling	Device Testing	In Progress
NFR-08	AI-generated information shall be treated as an estimate and require user review before storage.	Supervisor Feedback	AI Review and Confirmation Design	TC-07	In Progress
NFR-09	The application shall maintain acceptable response times during ordinary user operations.	Prototype Implementation and Testing	Mobile Application, Backend API, and Database	Performance Testing	In Progress
NFR-10	The system shall support modular maintenance and future extension.	Team Analysis and Workshops	Modular Flutter, Backend, and AI-Service Design	Architecture Review	In Progress
NFR-11	Errors shall be logged and handled without exposing sensitive implementation details to users.	Prototype Implementation and Testing	Exception Handling and Application Error States	Error-Handling Testing	In Progress
NFR-12	Notification delivery shall respect user preferences and configured quiet hours.	Product Backlog Refinement	Notification Service and Local Scheduler	TC-11	In Progress
NFR-13	Stored health and progress data shall remain consistent across integrated modules.	Prototype Implementation and Testing	Database, Tracker Services, and Constraints Layer	Integration Testing	In Progress
NFR-14	The Android application shall operate in the selected real-device and Linux-based environment.	Supervisor Feedback	Android Build and Linux-Based Backend Environment	Environment Testing	Not Started

Figure 29: Non-Functional Requirements Traceability Matrix

4.10.1.3 Traceability Summary

The traceability matrices show that the main functional and non-functional requirements are represented within the use case model, implemented through defined application components, and associated with appropriate validation activities. This traceability supports systematic testing and reduces the risk of requirements being omitted during implementation or final verification.

4.10.2 Initial Test Cases

Initial test cases were defined to establish an early link between the functional requirements and the testing activities that will be performed during system verification. At this stage, the cases identify representative inputs and expected system behavior; their actual execution results will be documented later in the testing and evaluation chapter. This follows the report guide, which places initial test cases within requirements analysis while reserving detailed execution and results for the testing chapter.

Test Case ID	Related Requirement	Description	Inputs	Expected Output	Result
TC-01	FR-01	Authenticate a registered user using valid credentials	Valid email address and password	The user is authenticated and redirected to the main application interface	Planned
TC-02	FR-02	Update the user's health profile	Valid personal and health-profile information	The submitted data is validated, stored, and displayed in the updated profile	Planned
TC-03	FR-03, FR-04, FR-05	Record daily health-tracking data	Valid hydration, sleep, step, or activity data	The record is saved and the related daily progress is recalculated	Planned
TC-04	FR-06	Complete a live activity session	Selected activity, duration, and intensity	The session is tracked, completed, stored, and reflected in the activity summary	Planned
TC-05	FR-07, FR-10	Add a meal manually	Food items and valid quantities	Nutrient values are calculated, the meal is stored, and daily nutrition totals are updated	Planned
TC-06	FR-08	Analyze a valid meal image	Supported meal image	The system returns detected foods, estimated quantities, nutrient values, and confidence information	Planned
TC-07	FR-09	Review an AI-generated meal result	Generated meal-analysis result and user decision	The user can edit, confirm, or reject the result; no meal is stored before confirmation	Planned
TC-08	FR-11, FR-12, FR-13	Create a medication schedule and record a dose status	Valid medication details, schedule, and taken or	The schedule is stored, the reminder is prepared, and the confirmed dose status updates the adherence record	Planned

			skipped status		
TC-09	FR-14, FR-15	Record a chronic condition and apply relevant constraints	Valid condition information or related health reading	The data is stored and made available to the Constraints Layer for relevant warnings and guidance	Planned
TC-10	FR-16, FR-17, FR-18	Record progress or relapse for a goal or unhealthy habit	Valid goal, progress, completion, or relapse data	Progress is updated while previous history remains preserved	Planned
TC-11	FR-19, FR-20, FR-21	Apply notification preferences and quiet hours	Notification category settings and quiet-hour period	Allowed reminders are scheduled appropriately, while blocked notifications are deferred or suppressed	Planned
TC-12	FR-22, FR-23, FR-24, FR-25	Generate a coordinated health report	Selected reporting period and available tracker records	The system displays aggregated indicators, trends, warnings, and supportive guidance without diagnosis or automatic treatment modification	Planned

Figure 30: Initial Test Cases

4.11 Chapter Summary

This chapter presented the requirements analysis and system-modeling activities conducted for VitaMate. It examined the project's technical, economic, and operational feasibility, described the requirements-elicitation methods and sources, identified the principal stakeholders and their needs, and documented the functional and non-functional system requirements. The requirements were also prioritized using the MoSCoW technique to distinguish essential functions from lower-priority and deferred work.

The principal system workflows were modeled using use case, activity, and sequence diagrams. The Agile requirements were documented through User Stories, Story Mapping, the Product Backlog, and the Sprint Backlog. The Requirements Traceability Matrix connected each requirement to its source, related design component, planned test, and current status. Initial test cases were also defined to establish an early relationship between requirements analysis and system verification.

Together, these artifacts form the requirements baseline for the architectural and detailed design presented in Chapter Five. The execution of the planned tests, the obtained evidence, and the final evaluation results will be documented in Chapter Seven.

Chapter 5:

Architectural and

Detailed Design

Chapter Introduction

This chapter presents the architectural and detailed design of the VitaMate system. It describes the overall system structure, the adopted architectural pattern, database design, API interfaces, security controls, user-interface design, main system modules, and intelligent services. The design is based on the final implemented system and defines how its components interact to support integrated health and lifestyle tracking.

5.1 High-Level System Design

VitaMate is designed as a mobile client–server system consisting of a Flutter Android application, a Django REST backend, a PostgreSQL database, a notification subsystem, and a separate FastAPI runtime for AI-assisted meal analysis. The Flutter application provides the user interface and communicates with Django through authenticated REST API requests. Django manages authentication, business logic, health-tracking services, system coordination, and persistent data operations, while PostgreSQL serves as the principal database.

The notification subsystem follows a backend-planned and client-synchronized design. Django generates notification plans according to user preferences and health-related rules, while the Flutter client synchronizes and schedules them locally on the Android device. AI meal analysis is also accessed through Django rather than directly from Flutter. Django validates the uploaded image, forwards it to the separate FastAPI runtime, and stores the returned suggestions for user review before a final meal is created.

This design keeps the backend responsible for the main application workflows and data integrity, while the mobile client handles user interaction and local notification delivery. The AI runtime remains isolated as an internal service with its own processing environment.

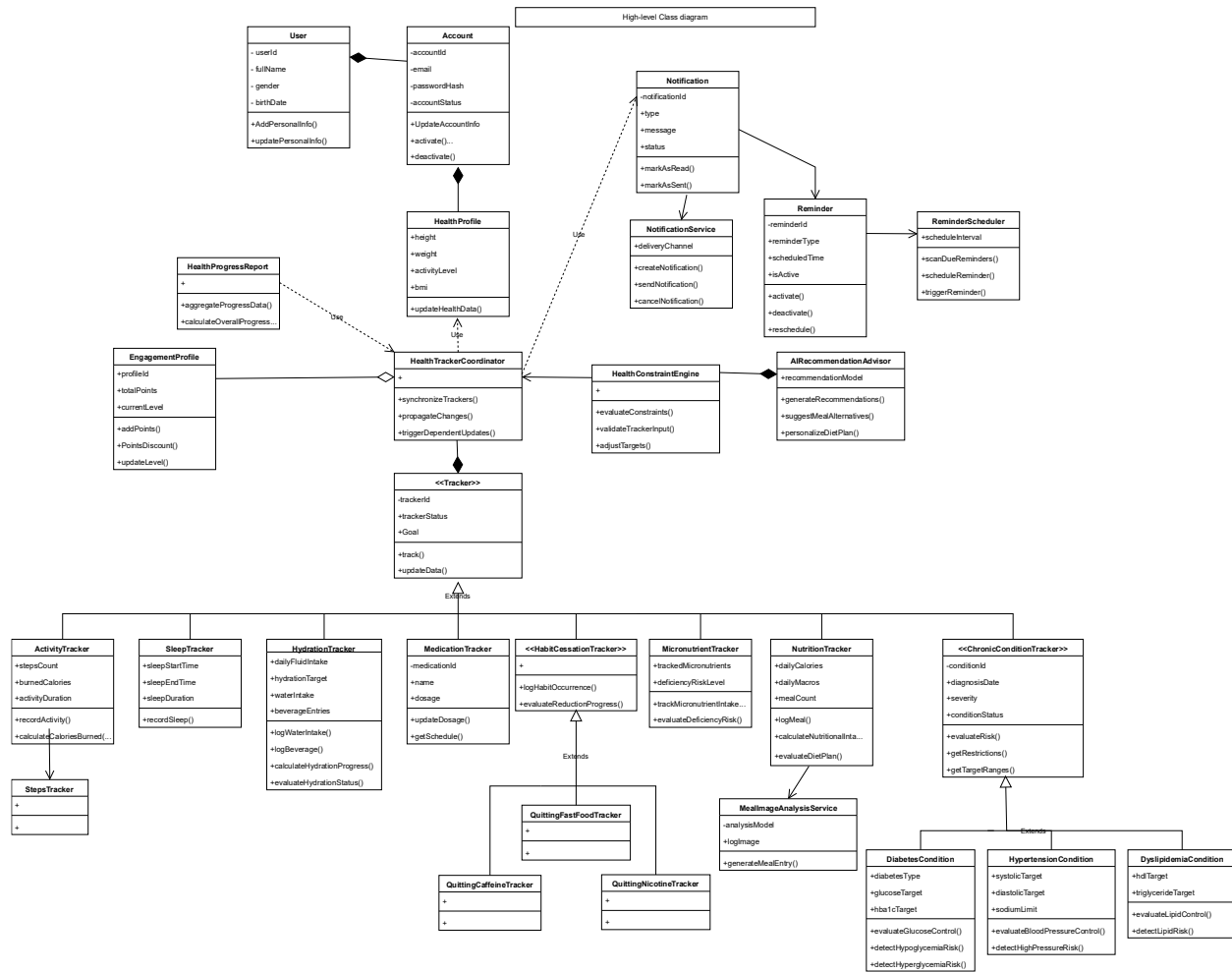


Figure 31 1presents the final high-level architecture of VitaMate

5.2 Architectural Pattern and Rationale

VitaMate follows a client–server architecture with clear separation between the Flutter mobile client and the Django backend. The backend is implemented as a feature-organized monolith composed of multiple Django applications. REST API views handle incoming requests, while the main business rules are implemented in domain services, orchestration services, and selected repository components. This organization keeps request handling separate from health-related processing and data-access responsibilities.

The Flutter application follows a feature-based structure and uses named routes, `ChangeNotifier` controllers, and local widget state. It is responsible for interface presentation, user input, and communication with the backend, while Django remains responsible for authorization, validation, business logic, and persistent data management.

The AI meal-analysis runtime is separated from Django as an internal FastAPI service. Django communicates with this runtime over HTTP using a service token, while Flutter does not access it directly. This separation isolates AI-specific dependencies and processing from the main backend while preserving Django as the central coordinator of user data and final meal processing.

This architectural pattern was selected to maintain clear responsibilities between the mobile interface, backend services, database operations, notification delivery, and AI inference, while keeping the main system manageable within a unified backend structure.

5.3 Database Design

VitaMate uses PostgreSQL as its principal backend database. SQLite is configured only as an optional local-development fallback and is not used as the production database. The database follows a relational design in which user accounts are linked to profile, health-tracking, nutrition, medication, chronic-care, notification, motivation, and AI-analysis records through primary and foreign-key relationships.

The database structure is organized by functional domain. Nutrition records include food items, nutrition facts, meals, and meal components, while activity, hydration, sleep, medications, chronic conditions, unhealthy habits, notifications, and motivation use dedicated models and historical logs. Business services validate and coordinate database operations so that final records remain consistent across the related modules.

Deletion behavior depends on the type of data. Some operational records, such as meal logs, may be permanently deleted, while health-management entities such as chronic conditions, medication plans, and habits use active or status fields to preserve their history. Historical information is also maintained through medication dose logs, health readings, notification events, habit logs, and points transactions.

AI meal-analysis results are stored separately as temporary, expiring review sessions. The proposed dish and component records remain provisional until the user confirms the ingredient mappings and gram values. Only after confirmation is the final meal stored using VitaMate's canonical nutrition records.

5.3.1 Entity–Relationship Design

The final Entity–Relationship Diagram represents the major persistent entities of VitaMate and the relationships among them. The user is the principal owner of the personal and health-related records. Most operational entities are connected directly or indirectly to the authenticated user to ensure that records remain isolated between accounts.

The data model is organized into several related domains. The account and profile domain stores authentication-related references, personal information, health-profile data, measurements, and

user-specific settings. The daily-tracking domain stores hydration, beverage, sleep, activity, step, workout, meal, and nutrition records.

The medication and chronic-condition domain stores medications, schedules, dose events, adherence information, chronic conditions, health readings, and condition-related information. These records may contribute to the constraints evaluated by the Constraints Layer.

The goals and habits domain stores user-defined goals, habit plans, progress events, completion information, and relapse-related history. Historical events are preserved so that progress is not represented only by the latest state.

The notification domain stores user preferences, notification categories, quiet-hour settings, reminder plans, and synchronization information required for controlled local notification delivery. The motivation domain stores the transactions and state needed to calculate points, missions, streaks, badges, and motivational feedback.

Nutrition records may be created manually or through the AI-assisted meal workflow. AI-generated results remain temporary and reviewable until the user confirms them. Only the confirmed result is converted into a persistent meal record. This prevents an unverified intelligent-service estimate from being treated as final user data.

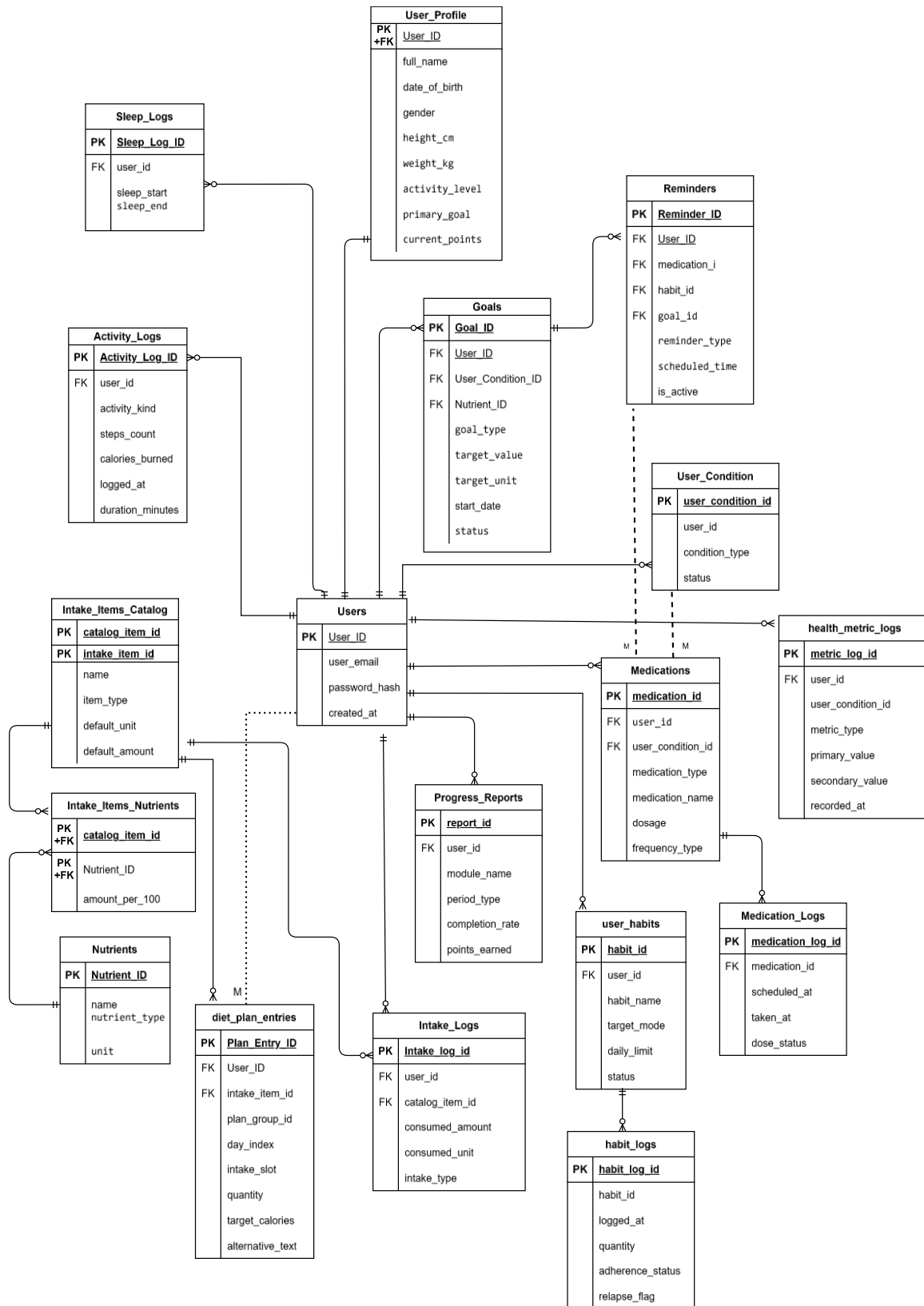


Figure 5.2. Final Entity–Relationship Diagram of the VitaMate System

The database design links each user account to a separate health and preference profile. User-owned records are connected to the authenticated user through foreign-key or one-to-one relationships, allowing the backend to isolate each user's health data and enforce ownership authorization. The main database domains and their representative entities are summarized in.

Database Domain	Representative Entities	Main Responsibility
Authentication and Profile	User, UserProfile	Stores account, personal, health, and preference information
Nutrition	FoodItem, NutritionFacts, MealLog, meal components	Stores the food catalog, canonical nutrient values, and logged meals
Hydration	Water-log records	Stores daily water consumption and progress
Activity and Steps	ActivityLog, ActivitySession, StepLog	Stores manual activities, guided sessions, and step records
Sleep	Sleep-tracking records	Stores sleep duration and related daily data
Chronic Care	Condition types, user conditions, health targets, indicator records	Stores chronic conditions, personalized targets, and health readings
Medication	Medication plans, schedules, and dose logs	Stores medication schedules, dose status, and adherence history
Unhealthy Habits	Habit plans and habit logs	Stores habit status, plans, reminders, and progress history
Motivation	PointsTransaction and motivation records	Stores points transactions and motivational history
Notifications	NotificationDevice, NotificationPreferenceProfile, NotificationPlan, NotificationPlanEvent	Stores registered devices, preferences, generated plans, and delivery events
AI Meals	MealAnalysisSession, MealAnalysisComponent	Stores temporary AI suggestions, user mappings, gram values, and review status
Health Constraints	Resolved constraint and resolution-run records	Stores the effective rules used to coordinate health-tracking modules

Table 8 :Main Database Domains and Entities

relationships between users and their operational data. Django's built-in `User` model is linked to `UserProfile` through a one-to-one relationship. The verified authentication identifier is the username, while nonblank email addresses are validated for case-insensitive uniqueness.

Data-integrity rules are also applied at the service and model levels. AI meal components cannot be finalized until the user has confirmed the required food mappings and supplied valid positive gram values. Analysis sessions contain status and expiration fields because their results remain provisional until final confirmation. The final nutritional values are calculated from VitaMate's canonical `FoodItem` and `NutritionFacts` records rather than being accepted directly from the AI runtime.

The system applies a mixed record-lifecycle strategy. Some operational records, including meal logs, can be permanently deleted. Other health-management entities use active, inactive, or lifecycle-status fields to preserve their historical context. Historical records are retained through medication dose logs, health-indicator readings, daily evaluations, habit logs, notification events, and points transactions.

This relational design supports consistency between the health modules while preserving the history required for progress tracking, adherence evaluation, notifications, and health-state calculations.

5.4 External Interface and API Design

VitaMate exposes its backend functionality through RESTful APIs implemented using Django REST Framework. The Flutter mobile application communicates with the backend through the `/api/` base path. URL-level versioning, such as `/api/v1/`, is not implemented in the current system.

The backend APIs support the principal VitaMate functions, including authentication, account and health-profile management, home and progress dashboards, nutrition, hydration and beverages, sleep, steps and physical activity, medications, chronic conditions, goals and unhealthy habits, motivation, notifications, health constraints, reports, and AI-assisted meal analysis.

Protected API operations use JWT bearer authentication. The backend provides access-token and refresh-token operations, while the Flutter application stores the authentication tokens using secure device storage. User-owned resources are retrieved and modified according to the authenticated user, and requests attempting to access records owned by another user are rejected.

Most application data are exchanged as JSON objects. File-based operations, particularly meal-image submission, use multipart form data where required. Responses are returned using standard HTTP status codes and structured response bodies containing the requested data or validation and error information.

The Flutter application does not communicate directly with the FastAPI food-analysis service. Meal-image requests are first submitted to the Django REST backend, which validates and forwards the required information to the FastAPI service. The returned estimated result is then provided to Flutter for user review, correction, confirmation, or rejection before a final meal record is stored.

5.4.1 API Architecture and Communication Flow:

The Django REST backend acts as the principal API gateway for the mobile application. It receives client requests, verifies authentication when required, validates submitted data, applies the relevant business rules, invokes the appropriate domain service, and returns the resulting data to Flutter.

The APIs are organized according to the system's functional domains. This organization keeps account, tracking, medication, notification, motivation, constraints, reporting, and nutrition operations separated while allowing coordinated processing through the Health Tracker Coordinator and Constraints Layer.

For ordinary tracking operations, the communication flow follows the sequence below:

Flutter Mobile Application → Django REST API → Domain Service → Django ORM → Relational Database

AI-assisted meal analysis adds a separate intelligent-service interaction:

Flutter Mobile Application → Django REST API → FastAPI Food-Analysis Service → Django REST API → Flutter Review Interface

After the user confirms the reviewed result, Django creates or updates the corresponding nutrition records. An unconfirmed or rejected AI result does not directly modify the user's final meal records or daily nutrition totals.

5.4.2 General API Design Conventions

Design Aspect	VitaMate Convention
Backend API framework	Django REST Framework
Base API path	/api/
URL versioning	No explicit URL-level version such as /api/v1/ is currently used
Primary client	Flutter mobile application
Standard data format	JSON
Image-upload format	Multipart form data for meal-image requests
Authentication	JWT bearer authentication for protected operations
Token operations	Access-token and refresh-token operations
Token storage	Secure storage within the Flutter application
Authorization scope	User-owned records are restricted to the authenticated user

Database access	Performed only through the Django backend and ORM
AI-service access	FastAPI is accessed through Django rather than directly from Flutter
Success responses	Structured data with the appropriate success status code
Validation failures	Structured field or request-level validation errors
Error handling	Standard HTTP status codes with explanatory response information
User confirmation	Required before an AI-generated meal result becomes a final meal record

Table 9General API Design Conventions

Table summarizes the general conventions applied to VitaMate’s external interfaces. These conventions centralize authentication, validation, authorization, business processing, and database access within the Django backend. They also prevent the mobile application and the AI service from directly modifying persistent application records.

5.4.3 Main Backend API Groups

API Group	Main Responsibility	Authentication
Authentication API	Registers users and manages access-token and refresh-token operations	Public for registration and token issuance; authenticated where applicable
Account and Profile API	Retrieves and updates personal, account, health-profile, and preference information	Required
Home and Dashboard API	Returns daily summaries, enabled tracker information, progress indicators, and dashboard data	Required
Nutrition API	Manages manual meals, food records, nutrient totals, meal history, and confirmed AI-assisted meals	Required
AI Meal API	Accepts supported meal images, coordinates AI analysis, returns estimated results, and finalizes confirmed meals	Required
Hydration and Beverage API	Manages water and beverage records, hydration progress, and relevant daily totals	Required
Activity and Steps API	Manages step information, activity records, workouts, live sessions,	Required

	history, and active-time data	
Sleep API	Manages sleep records, schedules, duration calculations, goals, and sleep history	Required
Medication API	Manages medication information, schedules, dose events, adherence, and medication history	Required
Chronic Conditions API	Manages supported conditions, health readings, targets, and related records	Required
Goals and Habits API	Manages personal goals, habit plans, progress, completion, and relapse-related events	Required
Notification API	Manages devices, preferences, quiet hours, notification plans, synchronization, and related events	Required
Motivation API	Returns points, transactions, missions, streaks, achievements, and motivational-feed information	Required
Constraints API	Provides or triggers the resolution of applicable user-specific health constraints	Required
Reports API	Returns daily or periodic summaries, trends, adherence indicators, and coordinated progress information	Required

Table 10: Main VitaMate API Groups

The API groups shown in Table represent the functional organization of the backend interfaces. They do not imply that every operation is exposed through a single endpoint. Each group may include list, retrieval, creation, update, deletion, calculation, synchronization, or domain-specific action endpoints according to the implemented module.

5.4.4 AI Food-Analysis Interface

The AI food-analysis interface connects the nutrition workflow with the separate FastAPI service. Django remains the external interface presented to the Flutter application and controls the full analysis and confirmation workflow.

The initial analysis operation receives a supported meal image and creates a temporary analysis request. Django validates the request and transfers the image and required metadata to FastAPI.

The intelligent service performs the supported image-processing stages and returns a structured estimated result.

Depending on the available analysis evidence, the response may contain detected or candidate food labels, estimated quantities or weights, nutritional values, confidence information, limitation messages, and information indicating whether the result is complete or partial.

The result is then displayed through the Flutter meal-review interface. The user may modify supported fields, confirm the result, or reject it. Finalization is treated as a separate action from initial analysis so that an uncertain estimate cannot automatically become a confirmed nutrition record.

When the result is confirmed, Django validates the reviewed values and creates the corresponding final meal and nutrition records. When the result is rejected, expired, invalid, or incomplete without confirmation, it does not update the final meal history or daily nutrition totals.

5.4.5 Request Validation and Error Handling

API requests are validated before business operations or persistent changes are performed. Django REST serializers validate ordinary structured input, while domain services apply additional rules related to ownership, dates, quantities, states, schedules, tracker dependencies, and applicable health constraints.

Authentication failures, missing records, invalid field values, unsupported state transitions, ownership violations, and external-service failures are returned using appropriate HTTP status codes and structured error information. Validation errors identify the invalid field or request condition whenever possible so that the Flutter interface can present a corrective message to the user.

Image-analysis requests include additional validation because the submitted file must satisfy the supported image and capture conditions. Unsupported, invalid, low-confidence, or partially processed cases are not silently converted into confirmed nutrition records. Instead, the API returns the available limitation or failure information and preserves the option of correction, retry, rejection, or manual meal entry.

Failures occurring during communication with the FastAPI service are handled without creating an incomplete final meal. Similarly, repeated finalization requests must not create duplicate confirmed meals when the same analysis operation has already been completed.

Response Category	Meaning
Successful retrieval	The requested resource or calculated result was returned
Successful creation	A validated resource was created
Successful update	An existing authorized resource was modified
Successful deletion	The authorized resource was removed or deactivated according to module rules

Validation error	One or more submitted values or request conditions were invalid
Authentication error	Valid authentication credentials were not provided
Authorization error	The authenticated user was not allowed to access or modify the requested resource
Not-found error	The requested resource, session, or analysis result was unavailable
Conflict error	The requested operation conflicted with an existing state or duplicate action
Service error	A backend or dependent-service operation failed
AI partial result	The service returned limited estimates with explicit confidence or limitation information

Table 11:Principal API Response Categories

5.4.6 OpenAPI and API Documentation

VitaMate exposes an authenticated OpenAPI-style schema at `/api/schema/`. The schema provides machine-readable documentation for the backend API and includes information about the available paths and operations.

The schema metadata uses version `1.0.0`; however, the implemented API routes remain under the unversioned `/api/` base path rather than a URI such as `/api/v1/`.

A separate Swagger UI or ReDoc interface was not verified in the current implementation. Therefore, the final system is documented as providing an authenticated OpenAPI-style schema without claiming the availability of an interactive API-documentation interface.

5.5 Security, Authentication, and Authorization Design

5.5.1 Authentication Design

VitaMate protects its backend APIs using JWT bearer authentication implemented through Django REST Framework and SimpleJWT. The authentication process issues an access token for authenticated API requests and a refresh token for obtaining a new access token when required.

The Flutter application stores both tokens using secure device storage and restores the authenticated session during application bootstrap. Tokens are not stored in SQLite or in ordinary application preferences.

User passwords are managed through Django's built-in authentication system. Account creation uses Django's standard user-management functions, which apply the framework's password-hashing mechanism rather than storing passwords as plain text.

5.5.2 Authorization and User Roles

The mobile application exposes the authenticated user as its principal application role. A separate administrative actor is not implemented within the Flutter interface.

Administrative functions are available through the Django administration site and staff-based backend permissions. These capabilities are used for backend data management and are separate from the ordinary mobile-user workflow.

Authorization for user-owned resources is enforced through owner-scoped queries, service-level checks, and endpoint-specific permission controls. Records such as meals and AI-analysis sessions are retrieved only for the authenticated user. Attempts to access another user's records are rejected, commonly through a not-found response that avoids exposing the existence of the protected resource.

Selected shared catalog operations apply staff-based permissions. For example, ordinary users may read supported nutrition-catalog data, while modification of protected catalog content is restricted to authorized staff users.

5.5.3 Data Protection and Secure Communication

Security-sensitive configuration values, including CORS origins, AI-service addresses, and service credentials, are supplied through environment variables rather than being fixed directly in the application logic. Communication between Django and the FastAPI food-analysis runtime is protected using a dedicated service token.

Production settings provide configurable HTTPS-related controls, including SSL redirection and HTTP Strict Transport Security. However, active HTTPS deployment was not verified from the repository; therefore, the report does not claim that the submitted system is currently operating through a live HTTPS environment.

Meal images are validated before being transferred to the AI runtime. The backend verifies supported formats, file signatures, file size, decoding validity, and configured image dimensions, then normalizes the image before further processing. These checks reduce the risk of invalid or unsupported file uploads reaching the intelligent service.

CORS access is restricted through environment-configured allowed origins. The repository does not confirm a global Django REST Framework throttling policy. Request-limiting controls are implemented specifically for the AI runtime through bounded queue capacity, concurrency limits, and timeout handling.

5.5.4 OWASP-Related Protection Considerations

The implemented security design addresses selected risks associated with the OWASP Top 10 without claiming complete OWASP compliance. Broken-access-control risks are reduced through authenticated and owner-scoped resource access. Cryptographic risks are addressed through Django password hashing, JWT authentication, and secure token storage. Injection risks are reduced through Django ORM access, serializer validation, and service-level validation rather than direct construction of database queries from user input.

Security-misconfiguration risks are reduced by keeping service credentials, CORS origins, and runtime addresses in environment-based configuration. File-upload risks are limited through server-side image validation, while resource-consumption risks in the AI runtime are controlled through queue, concurrency, and timeout limits.

Full penetration testing, production HTTPS verification, and complete OWASP compliance assessment are outside the evidence currently verified from the repository and must not be presented as completed security validation.

VitaMate centralizes authentication, authorization, ownership validation, credential handling, and sensitive-service configuration within the Django backend and the Flutter secure-storage layer. The verified controls provide a structured security foundation while acknowledging that active HTTPS deployment, global API throttling, and full OWASP compliance testing were not confirmed in the current repository.

5.6 User Interface and User Experience Design

The VitaMate mobile interface is implemented using Flutter and follows a light Material 3 design with a consistent purple and violet color palette. Shared theme definitions are used to maintain visual consistency across the application screens.

Navigation is based on Flutter named routes registered centrally within the application. The initial route is selected during application bootstrap according to the user's authentication state.

The interface includes loading indicators, error messages, retry actions, and form-validation feedback in the main implemented workflows. Some screens also include semantic annotations to improve accessibility. However, complete screen-reader coverage, dark mode, full localization, and RTL support were not verified in the current implementation and are therefore not claimed as completed features.

Compact-screen compatibility is supported through responsive Flutter layouts and selected interface tests. The repository confirms testing at a compact viewport and Android-emulator-oriented workflows, but it does not establish a complete physical-device compatibility matrix.

5.7 Detailed Module Design

VitaMate is divided into feature-oriented modules, each responsible for a defined health or application domain. The modules share the Django backend, relational database, authentication mechanism, and common coordination services.

The Flutter client provides the user-facing screens and controllers, while backend domain services implement validation, calculations, state transitions, and persistence. Cross-module summaries are assembled through the Health Tracker Coordinator, while the rule-based Constraints Layer resolves applicable goals and health restrictions before affected tracker information is presented.

Module	Main Responsibility
Authentication and Profile	Manages user accounts, authentication state, health-profile data, and preferences
Home and Progress	Combines tracker summaries, trends, warnings, daily health score, and progress information
Nutrition	Manages meals, food search, favorites, nutrient totals, micronutrients, and meal history
AI-Assisted Meals	Manages provisional image-analysis sessions, user corrections, confirmation, and final meal creation
Hydration	Manages water and beverage records, targets, progress, and hydration history
Activity and Steps	Manages steps, manual activity logs, and live activity-session states
Sleep	Manages sleep records, duration, schedules, targets, and history
Medication	Manages medication plans, schedules, dose actions, PRN records, history, and adherence
Chronic Conditions	Manages user conditions, health targets, indicator readings, summaries, and linked medications
Goals and Habits	Manages configurable daily goals and unhealthy-habit plans, logs, reminders, pause, and resume actions
Motivation	Manages points, transactions, streaks, missions, achievements, and motivational events
Notifications	Manages devices, user preferences, notification plans, synchronization, and local-delivery events
Constraints Layer	Resolves rule-based goals, restrictions, and tracker constraints
Health Tracker Coordinator	Provides coordinated read-side data for home, progress, and history views

Table 12: Main VitaMate Modules and Responsibilities

Each module owns its domain-specific operations, while shared services reduce duplicated logic between trackers. The Constraints Layer does not represent an AI recommendation service; it is a rule-based backend component. Likewise, motivational points are managed separately from the daily health score.

Detailed implementation classes, project folders, dependencies, and representative code are documented in Chapter Six.

5.8 Intelligent Service Design

The only intelligent service included in the final VitaMate scope is the AI-assisted meal-image analysis service. No separate LLM-based recommendation service is included in the implemented system.

The service is deployed as a separate FastAPI runtime and is accessed internally through the Django backend. The Flutter application does not communicate with the AI runtime directly. Django is responsible for authentication, image validation, temporary-session management, ownership checks, user confirmation, and final meal creation.

The designed communication flow is:

Flutter Application → Django REST Backend → FastAPI AI Runtime → Django Provisional Analysis Session → Flutter Review Interface → Django Meal Finalization

The AI runtime processes the submitted meal image and returns confidence-scored dish and ingredient suggestions. The returned information is stored by Django as a provisional, expiring analysis session rather than as a final meal. Exact underlying model architectures are not documented in this chapter because they were not verified from the current repository and belong to the implementation discussion in Chapter Six.

5.8.1 Input and Validation Design

The service accepts supported meal images in JPEG, PNG, or WebP format. Before forwarding an image to the AI runtime, Django validates its file type, signature, size, decoding validity, and dimensions, and performs the required normalization.

Invalid or unsupported images are rejected before intelligent processing. Additional capture restrictions that were not verified from the repository are not presented as implemented system requirements.

5.8.2 Provisional Analysis and User Review

Each AI result is stored as a temporary analysis session with an expiry time. The session contains the suggested dish, detected components, confidence information, candidate food mappings, and review status.

Before saving, the user can modify the selected dish, include or exclude components, map ingredients to catalog foods, correct gram values, add manual components, and update meal information such as meal type and consumption time.

The analysis cannot become a final meal until the required mappings and gram values are confirmed by the authenticated user. Rejected, expired, or incomplete sessions do not create final nutrition records.

5.8.3 Nutrition Finalization Design

Raw AI output is not treated as the final source of nutritional values. After user confirmation, Django maps the reviewed components to VitaMate's canonical nutrition catalog and calculates the final calories, macronutrients, vitamins, and minerals from the stored catalog information.

This design separates visual estimation from trusted nutrition calculation. The AI service proposes possible food information, while the Django nutrition services control the final persistent meal record and related daily nutrition totals.

Component	Design Responsibility
Flutter application	Uploads the image and provides the review and correction interface
Django REST backend	Validates requests, authenticates users, stores provisional sessions, and controls finalization
FastAPI runtime	Processes the image and returns confidence-scored food suggestions
Analysis session	Temporarily stores provisional results, review state, and expiry information
Nutrition catalog	Provides the canonical nutritional values used for final calculation
User confirmation	Prevents unreviewed AI results from becoming final meal records

Table 13:AI Food-Analysis Design Responsibilities

5.9 Chapter Summary

This chapter presented the architectural and detailed design of VitaMate. It described the interaction between the Flutter mobile client, the Django REST backend, the PostgreSQL database, the notification subsystem, and the separate FastAPI food-analysis runtime. It also documented the selected architectural structure, database design, REST API interfaces, authentication and authorization controls, user-interface design, and the responsibilities of the main application modules.

The design centralizes business logic and persistent-data control within the Django backend. Cross-module information is coordinated through the Health Tracker Coordinator and the rule-based Constraints Layer, while notifications follow a backend-planned and locally scheduled Android workflow. The AI meal-analysis service remains separate from the main backend and produces provisional results that require user review and confirmation before final meal records are created.

The following chapter presents the development environment, tools, frameworks, dependencies, project structure, and software implementation used to realize this design

Chapter 6:

Development

Environment and

Software

Implementation

This chapter documents the development environment and software implementation of VitaMate. It presents the technologies, frameworks, database environment, dependencies, source-control practices, test environment, and implementation of the main system modules. It also describes the implementation of the REST APIs, data-access layer, business logic, mobile user interface, notification subsystem, and AI-assisted meal-analysis service.

6.1 Senior Project 1 Implementation Baseline

By the end of Senior Project 1, VitaMate had an integrated implementation baseline covering the main mobile client, Django REST backend, and relational database. The implemented functional scope included authentication and onboarding, the home interface, nutrition tracking, hydration tracking, activity and steps, sleep tracking, chronic-condition management, and medication management.

These modules established the initial end-to-end foundation of the system, including Flutter screens, backend APIs, database models, and the core communication between the mobile application and the backend. This baseline was used during Senior Project 2 for further integration, refinement, correction, and feature expansion.

6.2 Development Tools and Libraries

VitaMate was developed using a set of mobile, backend, database, testing, and AI libraries. Table 6.1 presents the principal tools and verified versions used in the current repository. Where one project-wide version could not be verified, the version is reported according to its actual development or CI context.

Tool or Library	Verified Version or Constraint	Purpose
Dart	^3.10.1	Flutter application programming language
Flutter	>= 3.38.0; CI uses stable	Mobile application framework
Python	3.10.0 locally; 3.13 in CI	Backend and AI-runtime language
Django	5.2.10	Main backend framework
Django REST Framework	3.16.1	REST API implementation
SimpleJWT	5.5.1	JWT authentication
PostgreSQL	17.9-3 local portable setup; 17 in CI	Primary relational database
Psycopg	>=3.2,<4	PostgreSQL driver
Pillow	>=10.4,<13	Backend image processing
Dio	5.9.0	Flutter HTTP communication
Flutter Secure Storage	9.2.4	Secure JWT storage
Flutter Local Notifications	17.2.4	Android local reminder scheduling
Image Picker	1.2.3	Capturing and selecting meal images
FastAPI and Uvicorn	Exact versions not pinned	Internal AI-analysis runtime
Transformers	5.5.4	AI model loading and inference
PyTorch, Ultralytics YOLO, OpenCV,	Exact versions not fully	Meal-image analysis and

and NumPy	pinned	preprocessing
VS Code	Version not specified	Verified development editor
Git and GitHub	Version not specified	Source control and repository hosting
GitHub Actions	Repository workflow	Continuous integration

Table 14: Main Development Tools and Libraries

6.3 Development Environment and Project Structure

VitaMate is primarily developed and executed in a Windows-based local environment. The Django backend, PostgreSQL database, FastAPI meal-analysis runtime, and Flutter application are started through documented PowerShell scripts and localhost services.

The Flutter application is executed on an Android emulator during local development and automated integration testing. USB-device connectivity is also supported through ADB and port forwarding; however, completed physical-device testing was not verified from the repository.

Linux is used in the continuous-integration environment through GitHub Actions runners. Docker is not used as the principal local development or deployment environment; its verified use is limited to PostgreSQL service containers inside CI.

Visual Studio Code is the verified development editor. The repository also contains a JetBrains-family Flutter run configuration, but it does not identify Android Studio, IntelliJ IDEA, or PyCharm specifically.

Environment	Verified Use
Windows	Primary documented local development environment
Localhost	Runs Django, PostgreSQL, FastAPI, and Flutter integration
Android Emulator	Local application execution and automated integration testing
Physical Android Device	Connection procedure documented; completed testing not verified
Ubuntu Linux	GitHub Actions CI runner
Docker	PostgreSQL service containers in CI only
Production Server	Not verified
Staging Server	Not verified

Table 15 : Development and Runtime Environments

6.3.1 Project Organization

The repository contains a Flutter frontend, a Django backend, an internal FastAPI AI-runtime wrapper, automated tests, supporting documentation, scripts, and GitHub Actions configuration.

The Django backend is organized into active applications for users, core health domains, gamification, manager functions, notifications, and AI-assisted meals. Domain APIs are separated from business services, repositories, and orchestration components.

The Flutter client is organized by feature. Shared networking, secure storage, routing, and notification infrastructure are placed under the core layer, while each feature contains its models, data classes, controllers, screens, and reusable widgets. State is managed mainly through ChangeNotifier controllers and local widget state.

6.4 Programming Languages and Frameworks

VitaMate was implemented using Dart and Flutter for the mobile application and Python for the backend and intelligent-service components.

Flutter provides the mobile user interface, navigation, feature screens, client-side controllers, API communication, secure token storage, meal-image capture, and local Android notification scheduling. The client follows a feature-based structure and uses ChangeNotifier controllers together with local widget state.

The main backend is implemented using Django and Django REST Framework. Django manages authentication, business logic, database access, domain services, validation, and persistent records, while Django REST Framework exposes the application functionality through authenticated REST APIs.

The AI-assisted meal-analysis component runs as a separate FastAPI service. Django communicates with this service internally through HTTP and a protected service token. The Flutter application does not access FastAPI directly.

PostgreSQL is used as the primary relational database and is accessed through Django's ORM. SQLite is available only as an optional local-development fallback. Android local-notification libraries execute the reminder plans synchronized from the backend.

Technology	Implementation Role
Dart	Programming language of the Flutter mobile client
Flutter	Mobile UI, routing, controllers, API integration, and local reminders
Python	Backend and AI-runtime implementation language
Django	Authentication, domain logic, ORM, and persistent backend processing
Django REST Framework	Authenticated REST API layer
FastAPI	Internal meal-image-analysis runtime
PostgreSQL	Primary relational database
SQLite	Optional local-development database fallback
Android Local Notifications	Device-side scheduling and delivery of reminder plans

Table 16 :Main Languages and Frameworks

6.5 Database Server and Runtime Environment

PostgreSQL is the primary relational database used by the VitaMate backend. The verified local setup uses PostgreSQL 17.9-3, while the continuous-integration environment uses PostgreSQL 17.

Database configuration is managed through Django settings and environment variables, including `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_HOST`, and `POSTGRES_PORT`. Schema creation and updates are handled through Django migrations.

SQLite is available only as an explicit local-development fallback when `VITAMATE_USE_SQLITE=1`. It is not used as the principal backend database or presented as the production database.

The current repository verifies local and CI database environments, but it does not verify an active production or staging database deployment.

Item	Verified Configuration
Primary database	PostgreSQL
Local PostgreSQL version	17.9-3
CI PostgreSQL version	17
ORM	Django ORM
Schema management	Django migrations
Development fallback	SQLite when explicitly enabled
Production database deployment	Not verified
Staging database deployment	Not verified

Table 17 :Database and Runtime Configuration

6.6 Dependencies and Configuration Files

VitaMate uses separate dependency files for the backend, mobile client, and AI runtime. The Django backend dependencies are defined in `requirements.txt`, while Flutter dependencies and resolved package versions are managed through `pubspec.yaml` and `pubspec.lock`. The AI runtime combines a bundled vendor requirements file with a VitaMate-specific runtime overlay.

Application configuration is environment-driven. Django selects development or production settings through `DJANGO_ENV`. Database access, CORS, JWT authentication, AI-service communication, image limits, session expiry, and runtime limits are configured through settings files and environment variables rather than fixed production values in the source code.

File	Purpose
------	---------

vitamate_backend/requirements.txt	Django backend dependencies
vitamate_frontend/pubspec.yaml	Dart SDK constraints and direct Flutter dependencies
vitamate_frontend/pubspec.lock	Resolved Flutter package and SDK versions
ai_service_runtime/requirements-overlay.in	VitaMate-specific FastAPI runtime additions
requirements-ai-package.txt	Bundled AI-runtime dependencies
vitamate_project/settings.py	Selects development or production settings
settings_base.py	Shared database, DRF, JWT, and AI configuration
settings_dev.py	Local development configuration and optional SQLite fallback
settings_prod.py	Production-oriented security and PostgreSQL settings
.env.example	Documents required environment variables
.github/workflows/ci.yml	CI runtime, dependency installation, and test configuration

Table 18: Main Dependency and Configuration Files

6.7 Source Control and Repository Management

VitaMate is maintained as a Git repository hosted on GitHub. The active default branch is main, and repository changes are validated through GitHub Actions.

The continuous-integration workflow is triggered by pushes and pull requests targeting main and master. This supports review and automated validation before changes are incorporated into the principal codebase.

The repository does not document a complete branching policy, feature-branch naming convention, or formal merge strategy. Therefore, the report describes only the verified use of the main branch and pull-request-based workflow without claiming a broader branching model.

Item	Verified Configuration
Version-control system	Git
Repository platform	GitHub
Active default branch	main
CI trigger branches	main and master
Pull-request validation	Supported through GitHub Actions
Feature-branch naming policy	Not documented
Formal merge strategy	Not documented

Table 19:Source-Control Configuration

6.8 Continuous Integration and Continuous Delivery

VitaMate uses GitHub Actions for continuous integration. The active workflow, named `VitaMate CI`, runs automatically on pushes and pull requests targeting the main repository branches and can also be started manually.

The workflow performs secrets scanning, pre-commit quality checks, backend testing with PostgreSQL, migration validation, Flutter static analysis, Flutter tests, and Android-emulator integration testing. Integration-test logs are uploaded as workflow artifacts to support failure inspection.

Continuous delivery is not implemented in the current repository. No automated deployment, staging release, production publishing, Docker-based deployment, or mobile release-upload step was verified.

CI Job	Purpose
Gitleaks	Scans the repository for exposed secrets
Pre-commit	Executes configured source-quality checks
Backend Tests	Runs migrations and Django tests against PostgreSQL 17
Flutter Analyze	Performs Dart and Flutter static analysis
Flutter Test	Executes Flutter unit and widget tests
Flutter Integration Test	Runs end-to-end flows using Django, PostgreSQL, and an Android emulator
Artifact Upload	Stores integration-test logs for review

Table 20:Continuous-Integration Jobs

6.9 Test Environment

VitaMate uses separate but integrated testing environments for the Django backend, Flutter mobile client, and AI runtime.

Backend tests are implemented using Django’s `TestCase` and `TransactionTestCase`, together with Django REST Framework’s `APITestCase`. The default test configuration is PostgreSQL-oriented, while SQLite remains an optional local fallback. Project-specific helper functions are used to create authenticated users, profiles, and related test data.

Flutter testing includes unit tests, widget tests, golden tests, compact-screen checks, and integration tests. Integration flows run through Flutter’s `integration_test` tooling against a running Django backend and PostgreSQL database on an Android emulator.

The AI runtime is tested separately through vendor-side `pytest` and smoke tests. The main VitaMate repository verifies the AI integration contract and runtime wrapper behavior without loading the complete heavy model pipeline during ordinary backend tests.

Component	Test Environment and Tools
Django Backend	TestCase, TransactionTestCase, and DRF APITestCase
Backend Database	PostgreSQL-oriented test settings
Database Validation	Migration checks and schema migration execution
Flutter Client	Unit, widget, golden, and compact-screen tests
Mobile Integration	Flutter integration_test on Android emulator
AI Runtime	Vendor-side pytest and smoke tests
CI Environment	Ubuntu GitHub Actions runners with PostgreSQL and Android emulator

Table 21:Test Environments and Tools

6.10 Software Implementation

This section summarizes the implementation of VitaMate’s main software components. It covers the project structure, backend services, data access, REST APIs, mobile interface, notification subsystem, and AI-assisted meal-analysis service. Only representative implementation details are presented, while complete source code remains in the project repository or appendices.

6.10.1 Project and Folder Structure

VitaMate is organized as a single repository containing the Flutter mobile client, Django backend, internal AI runtime, automated tests, documentation, scripts, and continuous-integration configuration.

The backend separates API views, domain models, repositories, business services, orchestration services, notifications, gamification, user management, and AI-meal integration. The frontend follows a feature-based structure in which each domain contains its data layer, typed models, controllers, screens, and reusable widgets.

```
VitaMate/
├── .github/
│   ├── workflows/
│   └── ci.yml
├── docs/
├── Implementation/
│   ├── vitamate_backend/
│   │   ├── ai_meals/
│   │   ├── ai_service_runtime/
│   │   ├── core/
│   │   ├── api/
│   │   └── models/
```

```

/ / ├── repositories/
/ / ├── services/
/ / ├── tests/
/ ├── gamification/
/ ├── manager/
/ ├── notification_hub/
/ ├── users/
/ ├── scripts/
/ ├── docs/
/ ├── test_utils/
/ ├── vitamate_project/
/ ├── requirements.txt
/ ├── VitaMate_AI_Backend_Runtime_20260802.zip
└── vitamate_frontend/
    ├── lib/
    / ├── auth/
    / ├── core/
    / ├── features/
    / ├── shared/
    ├── test/
    ├── integration_test/
    ├── test_driver/
    ├── docs/
    ├── scripts/
    ├── pubspec.yaml
    └── pubspec.lock

```

Table 22 :Simplified VitaMate Project Structure

In the backend, `vitamate_project` contains the Django settings and central routing. The core package contains the main health-domain models, APIs, repositories, services, and orchestration logic. Separate applications manage users, gamification, manager functions, notifications, and AI-assisted meals.

In the Flutter client, application startup and routing are separated from shared infrastructure under `lib/core`. Functional modules are organized under `lib/features`, while authentication is maintained under `lib/auth`.

6.10.2 Backend and Business-Logic Implementation

The VitaMate backend is implemented as a layered Django application. API views receive and validate requests, service classes execute domain-specific business rules, repositories provide reusable data-access operations, and Django models persist the resulting records.

The backend separates the main health domains into dedicated modules for authentication and profiles, nutrition, hydration, activity and steps, sleep, chronic conditions, medications, unhealthy habits, motivation, health constraints, notifications, and unified health progress. Dashboard and progress payloads are coordinated through read-model and health-tracker orchestration services

Module	Main Responsibility
Authentication and Profile	Registration, authentication, user profile data, and derived health values
Nutrition	Food catalog, meal logging, nutrient snapshots, and meal finalization
Hydration	Water logging, daily hydration progress, and related projections
Activity and Steps	Activity records, live sessions, step logs, and progress calculations
Sleep	Sleep records, daily summaries, and sleep-improvement support
Chronic Conditions	User conditions, health indicators, and related health constraints
Medication	Medication plans, schedules, logs, and adherence information
Unhealthy Habits	Habit baselines, cessation plans, reminders, logs, and progress
Motivation	Points, levels, missions, and motivation-feed events
Constraints Layer	Resolution and storage of effective tracker targets
Notification Hub	Device registration, preferences, plan generation, and synchronization
Health Progress	Unified dashboard, progress history, and health-state coordination
AI-Assisted Meals	Provisional image analysis, user review, and final meal integration

Table 23 :Main Backend Modules and Responsibilities

Business logic is kept outside the API views whenever practical. For example, meal finalization is handled through a dedicated service that validates confirmed components, aggregates their nutritional values, creates the final meal and its components inside a database transaction, and then triggers related health-state, motivation, hydration, and habit integrations.

This separation reduces duplication and keeps request handling, business rules, and persistent data access independently maintainable.

Listing 6.1. Representative Django ORM and Data-Access Code

```
session = (
    MealAnalysisSession.objects
    .filter(user=user, id=analysis_id)
    .prefetch_related(
        "candidates__mapped_food_item__nutrition_facts"
    )
    .first()
)
```

This query retrieves a meal-analysis session only when it belongs to the authenticated user. It also preloads the related food mappings and nutrition records required for review and finalization.

Listing 6.2. Representative Backend Business-Logic Code

```
aggregate_snapshot = {
    snapshot_field: sum(...)
    for snapshot_field in SNAPSHOT_FIELD_MAP.values()
}

meal_log = MealLogRepository.create_for_user(
    user=user,
    meal_type=meal_type,
    **aggregate_snapshot,
)

MealLogComponent.objects.bulk_create(components)
cls._publish_integrations(meal_log)
```

The service aggregates the confirmed nutritional values into a canonical meal record, creates the meal components efficiently, and publishes the dependent integrations after successful persistence. The complete implementation is larger; therefore, only the central processing steps are shown.

6.10.3 Frontend and User Interface Implementation

The VitaMate mobile application is implemented using Flutter and organized around feature-based modules. Application startup and shared runtime initialization are handled separately from routing, networking, secure storage, notifications, and feature-specific interfaces.

Each principal feature contains its own data-access classes, typed models, ChangeNotifier controllers, screens, and reusable widgets. This structure is applied across authentication, home,

nutrition, hydration, activity, sleep, medications, chronic conditions, motivation, notifications, and profile-management workflows.

The user interface communicates with the Django backend through Dio-based API clients. JWT credentials are stored using secure storage, while controllers manage API loading, successful data retrieval, validation failures, and state updates presented by the screens.

The implemented screens use named application routes and shared visual components to maintain consistent navigation and interaction patterns. Input forms provide validation and feedback, while data-driven screens present loading, error, retry, empty, and successful-content states according to the corresponding operation.

6.10.3.1 Implemented Interface Modules

Frontend Module	Representative Implemented Interfaces
Authentication	Login, sign-up, and onboarding
Home	Unified home overview and daily health information
Nutrition	Nutrition dashboard, meal logging, daily meals, micronutrients, and AI meal capture and review
Hydration	Hydration overview, water logs, drink logging, and hydration settings
Activity	Activity, steps, and workout-session interfaces
Sleep	Sleep tracking and sleep-coach interfaces
Medications	Medication list, schedules, daily plan, history, details, and adherence
Chronic Conditions	Condition list, condition details, and related health information
Motivation	Motivation feed, points, missions, and profile progress
Notifications	Notification synchronization and manager-facing notification interfaces
Profile and Settings	Profile editing, health profile, goals, account security, medical data, and privacy

Table 24 :Main Frontend Modules and Interfaces

6.10.3.2 Representative Mobile Interfaces

Next figures present representative interfaces from the implemented Flutter application. The selected screens cover entry into the system, the unified dashboard, the principal health trackers, medication management, and profile settings. Together, they demonstrate the final navigation structure and the consistent visual identity applied across the application

Login Screen:

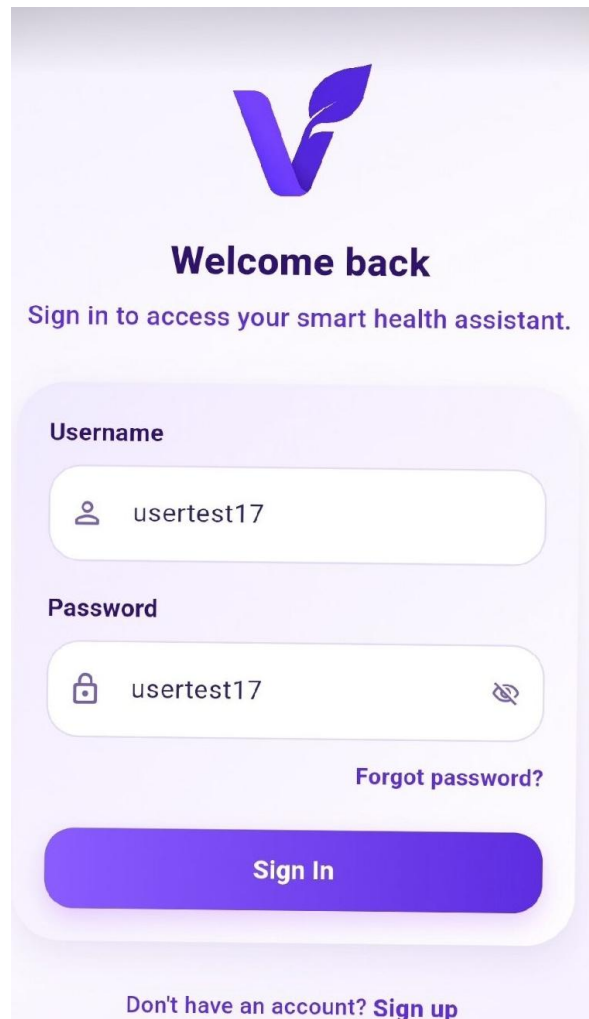


Figure 32: VitaMate Login Interface

Home Screen:

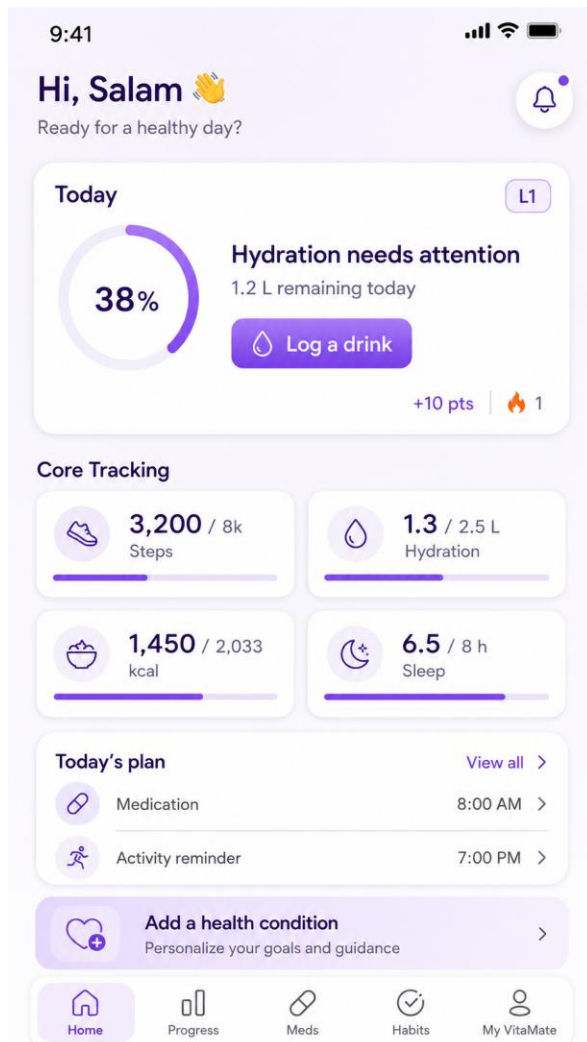


Figure 33 :VitaMate Home and Daily Health Overview

This interface provides a unified daily overview of hydration, steps, calorie intake, sleep, scheduled reminders, health conditions, and quick access to the main VitaMate modules.

Hydration Screen:

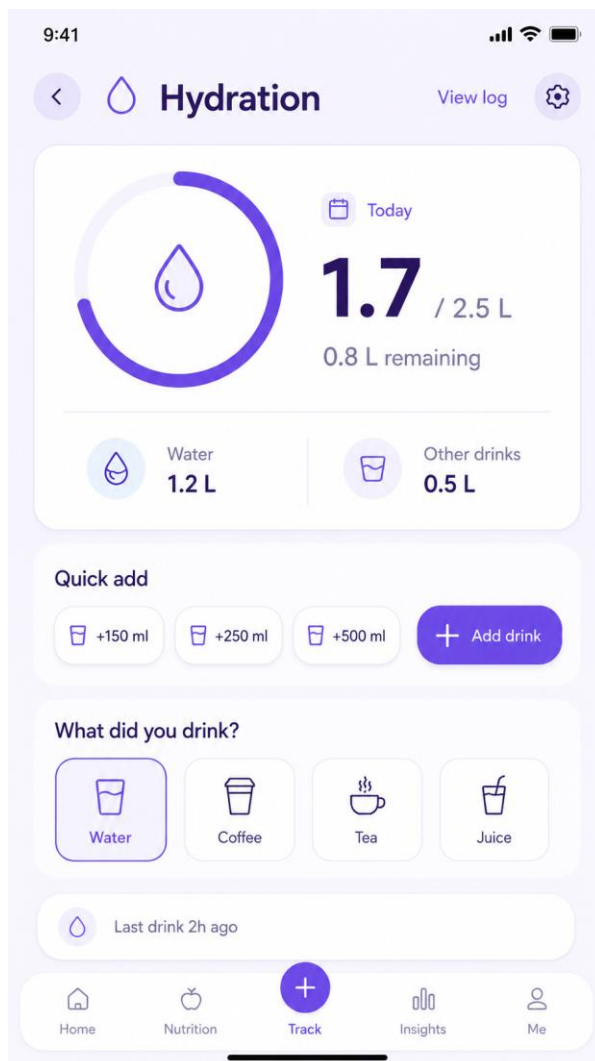


Figure 34:Hydration Tracking Interface

This interface displays daily hydration progress, separates water from other drinks, and provides quick controls for recording common drink amounts and reviewing hydration history.

Nutrition Dashboard:

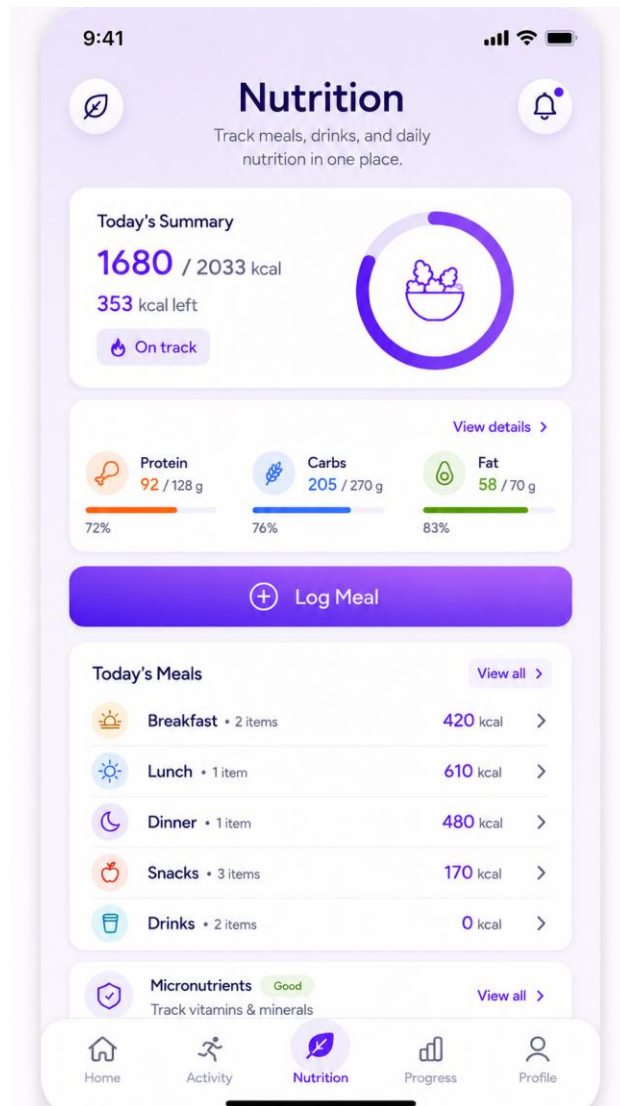


Figure 35 :Nutrition Tracking Interface

This interface summarizes daily calorie and macronutrient progress, provides access to meal logging and meal history, and supports detailed micronutrient tracking.

Activity Screen:

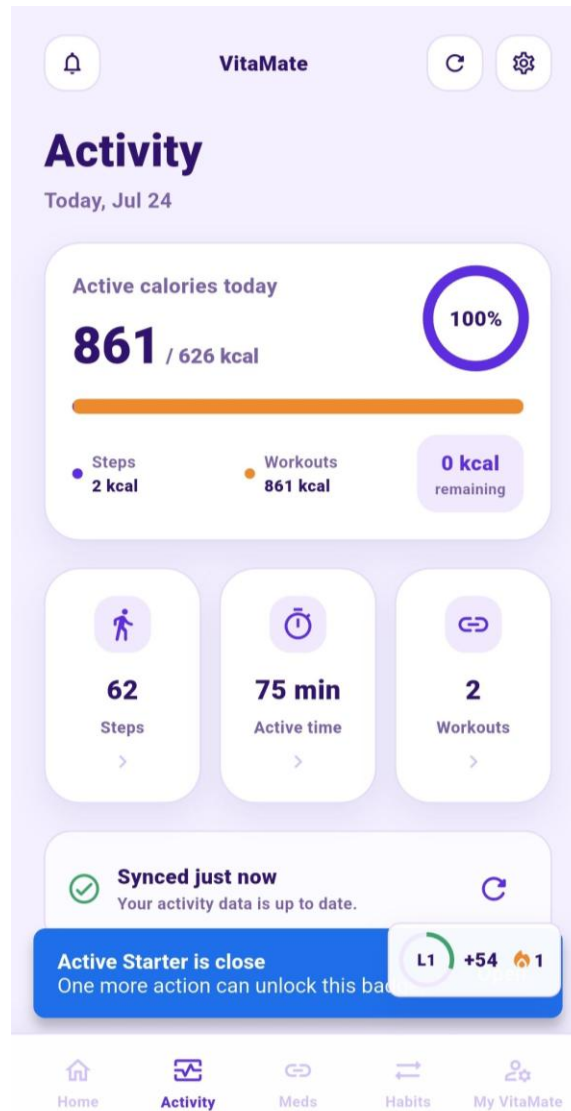


Figure 36 :Activity and Steps Interface

This interface presents daily active calories, steps, active time, and completed workouts, while also showing synchronization status and motivational progress feedback.

Habits Screen:

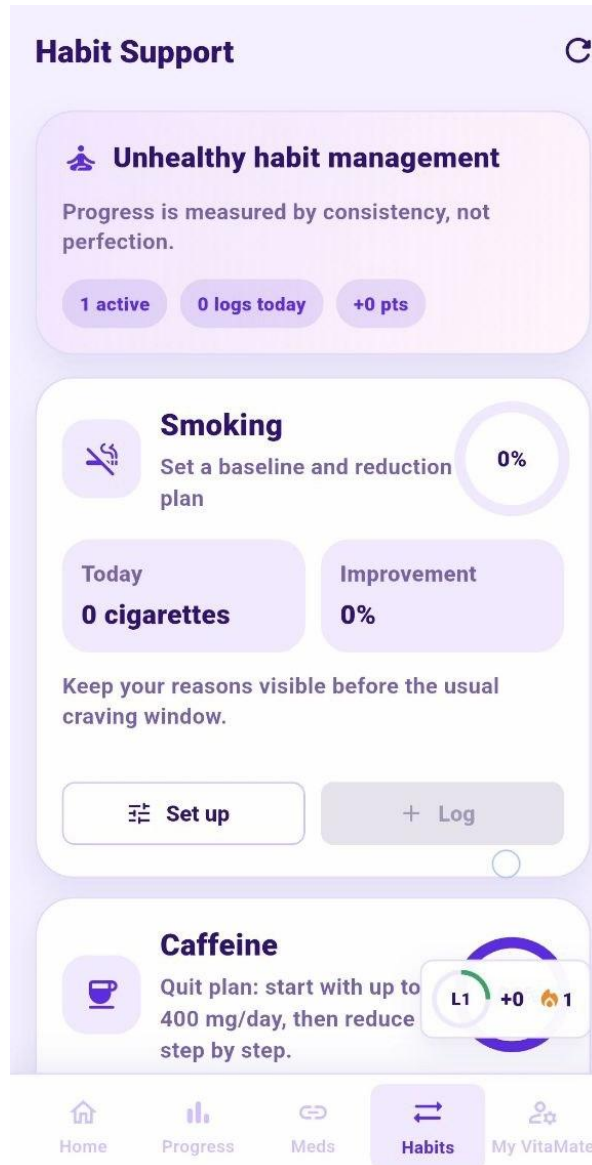


Figure 37: Unhealthy-Habit Management Interface

This interface supports unhealthy-habit reduction by presenting active habits, baseline setup, daily logging, improvement progress, and personalized guidance for behaviors such as smoking and caffeine consumption.

Medications Screen:

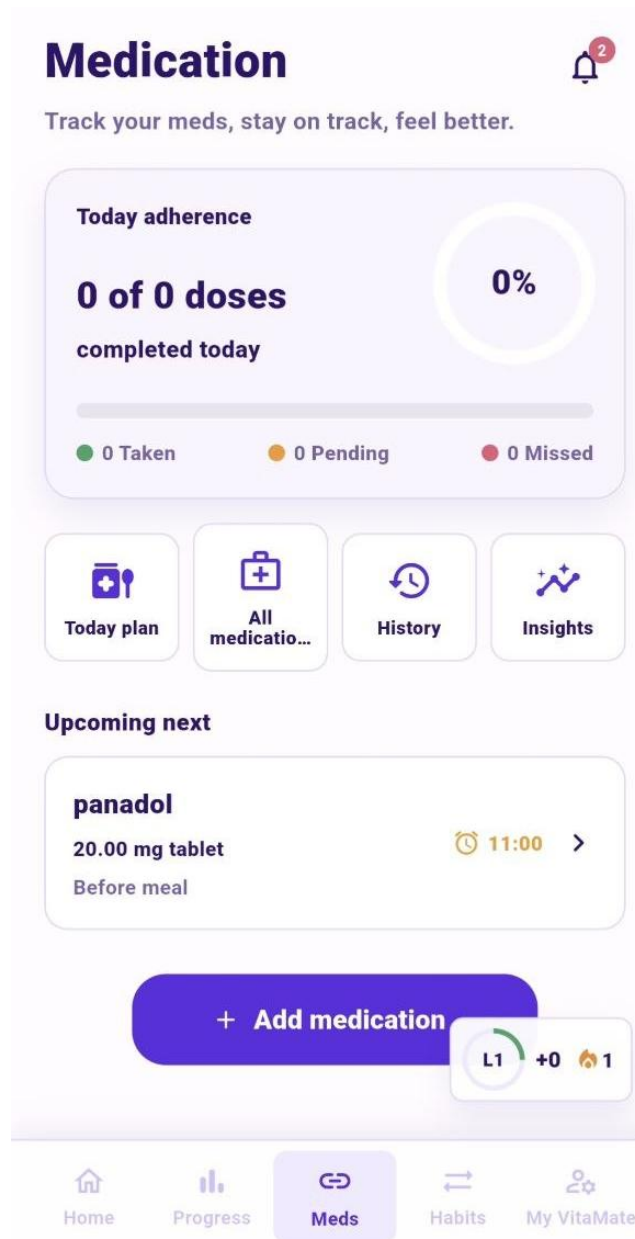


Figure 38 :Medication Management Interface

This interface summarizes daily medication adherence and provides access to the current plan, medication list, history, insights, upcoming doses, and medication registration.

My VitaMate / Profile Screen:

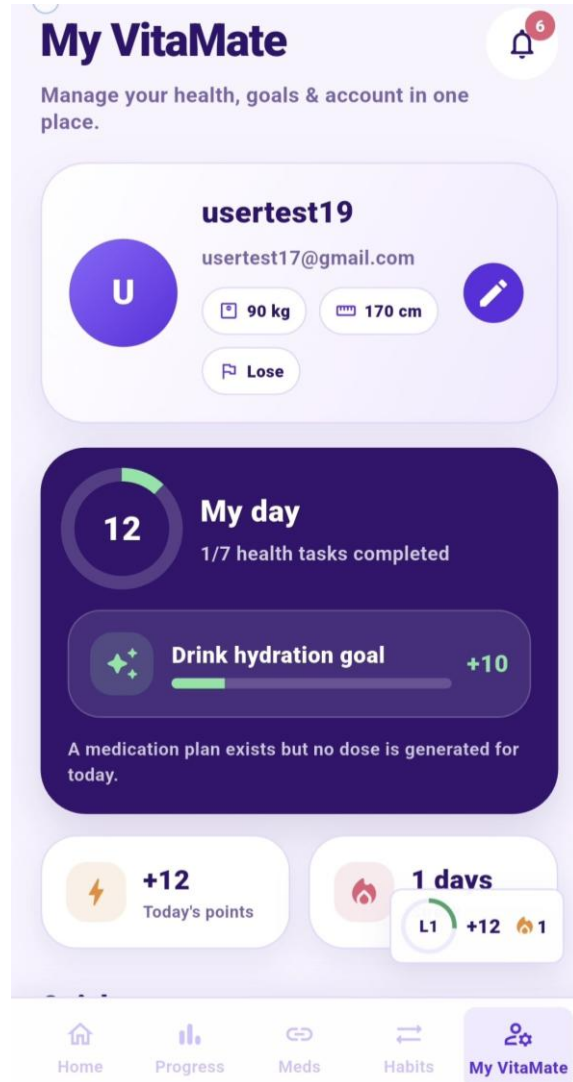


Figure 39 :Profile and Self-Management Interface

This interface centralizes personal health information, profile editing, daily health tasks, points, streak progress, notifications, and access to account and goal-management features.

6.10.4 API Implementation

VitaMate exposes its backend functionality through REST APIs implemented using Django REST Framework. API views receive authenticated requests, validate input through serializers,

delegate domain processing to dedicated service classes, and return endpoint-specific JSON responses with standard HTTP status codes.

The implemented APIs cover authentication, profiles, home and progress data, nutrition, hydration, sleep, activity and steps, medications, chronic conditions, unhealthy habits, motivation, health constraints, notifications, manager functions, and AI-assisted meals.

Protected endpoints use JWT access and refresh tokens. User-owned records are scoped to the authenticated user, which prevents access to another user's private health data. Search, filtering, and pagination are implemented according to the needs of each endpoint rather than through one global API policy.

Listing 6.3. Representative REST API Endpoint Implementation

```

serializer = AnalyzeMealSerializer(data=request.data)
serializer.is_valid(raise_exception=True)

analysis = service.analyze(
    user=request.user,
    analysis_key=analysis_key,
    **serializer.validated_data,
)

return Response(
    {
        "data": MealAnalysisSessionSerializer(
            analysis,
            context={"request": request},
        ).data
    },
    status=status.HTTP_201_CREATED,
)
    
```

This endpoint validates the uploaded meal-analysis request, associates the operation with the authenticated user, delegates the processing to `MealAnalysisService`, and returns the created provisional analysis session with the HTTP status 201 Created.

Item	Implementation
Endpoint	POST /api/nutrition/ai-meals/analyze/
Authentication	JWT-authenticated user
Request processing	Multipart input validated through <code>AnalyzeMealSerializer</code>
Business service	<code>MealAnalysisService.analyze()</code>

Successful response	201 Created
Invalid request	400 Bad Request
Non-retryable analysis rejection	422 Unprocessable Entity
AI runtime failure or timeout	503 Service Unavailable

Table 25 :Representative AI-Meal Analysis Endpoint

The API layer remains responsible for request validation, authentication, response serialization, and HTTP error mapping, while the analysis workflow itself is implemented in the service layer. This separation keeps the API views concise and prevents domain logic from being duplicated across endpoints.

VitaMate implements a complete notification workflow that connects backend planning with device-side execution. The Django backend registers user devices, stores notification preferences, generates notification plans, applies quiet-hour rules, and exposes synchronization and reporting endpoints.

The Flutter application retrieves the generated plans, cancels outdated schedules when necessary, and creates the corresponding local Android notifications. After scheduling or user interaction, the mobile client reports events such as scheduled, opened, suppressed, or failed back to the backend.

Notifications are organized into health-critical, routine, motivation, celebration, and system categories. Each category is mapped to an appropriate Android notification channel, allowing the application to apply different delivery behavior according to the notification purpose.

Device Registration



Notification Preferences



Backend Plan Generation



Quiet-Hour and Quota Processing



Flutter Plan Synchronization



Local Android Scheduling



Delivery and Interaction Reporting

Figure 40 :Notification Planning and Delivery Flow

The backend remains responsible for notification rules and planning, while Flutter and Android perform local scheduling and delivery on the registered device.

Component	Main Responsibility
Device Registry	Registers or updates the user's mobile device
Preference Service	Stores reminder categories, quiet hours, and user choices
Backend Planner	Compiles notification plans from health and application data
Synchronization API	Sends current plans to the Flutter client
Flutter Controller	Coordinates device registration, synchronization, and reporting
Local Plan Executor	Schedules, updates, or cancels Android reminders
Channel Registry	Maps notification categories to Android channels
Reporting API	Receives scheduling, opening, suppression, and failure events

Table 26 :Notification-Flow Responsibilities

This architecture replaced the earlier frontend-only reminder approach with a coordinated backend-to-device flow. It allows notification timing and health-related rules to be centrally generated while preserving reliable local delivery on the Android device.

6.10.6 AI Food-Analysis Implementation

VitaMate implements meal-image analysis as a separate FastAPI runtime integrated through the Django backend. The mobile application uploads a meal image to Django, which validates the request and forwards it to the internal AI service. The generated result is stored as a provisional analysis session and is not treated as a final meal until the user reviews and confirms the suggested dish, ingredients, and quantities.

The implemented pipeline combines food segmentation, dish recognition, optional ingredient evidence, and weight estimation. The active segmentation model was selected using human-annotated masks, while the dish-recognition model operates within VitaMate's supported food catalog. Ingredient predictions are used only as supporting evidence because their evaluation results were not sufficient for fully automatic activation. Automatic weight estimation also remains optional, and manual user-confirmed weight is used when confidence is insufficient.

After user confirmation, Django maps the reviewed components to VitaMate's canonical nutrition catalog and calculates the final nutritional values. Therefore, the AI service provides suggestions, while the backend and user-confirmation step remain authoritative for final meal creation.

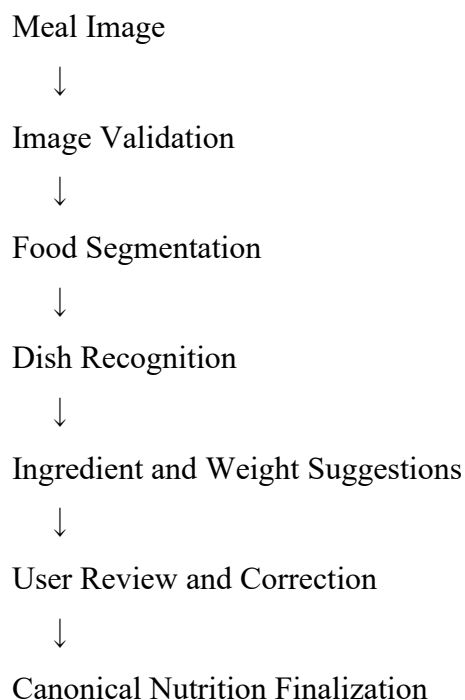


Figure 41 :AI Food-Analysis Flow

The analysis flow follows an analyze–confirm–finalize model to prevent uncertain predictions from being stored as final nutritional records.

Component	Implementation Role
Food Segmentation	Identifies the food region within the uploaded image
Dish Recognition	Returns ranked dish suggestions from the supported catalog
Ingredient Evidence	Provides optional component suggestions for user review
Weight Estimation	Produces an optional estimate when confidence conditions are satisfied
User Confirmation	Allows correction of dish, ingredients, and quantities
Nutrition Finalization	Calculates final values from VitaMate’s canonical nutrition database

Table 27 :Main AI Pipeline Components

he latest evaluated pipeline completed all 35/35 test images without runtime failure, compared with 16/35 completed images in the oldest baseline. However, the system remains a closed-set, review-assisted solution rather than a fully automatic open-world food-recognition system.

Human-mask segmentation evaluation reached a mean IoU of 90.74%, while dish Top-3 accuracy reached 57.14% on the 35-image evaluation set. Since ingredient and automatic-weight performance did not meet the required activation gates, catalog-based ingredient review and manual weight remain the reliable fallback path.

6.10.7 Senior Project 2 Integration and Refinement

Senior Project 2 extended the implementation baseline completed in Senior Project 1 through new features, architectural integration, defect correction, and interface refinement.

The unhealthy-habit cessation subsystem was added to support baseline assessment, reduction plans, daily logging, reminders, progress monitoring, and pause or resume workflows. Profile-management and related settings were also expanded to provide more complete user-data editing and self-management functions.

The notification subsystem was substantially redesigned. It evolved from a frontend-only reminder feature into a coordinated backend-to-device workflow that includes device registration, stored preferences, notification-plan generation, quiet-hour handling, Flutter synchronization, local Android scheduling, and event reporting.

The AI food-analysis feature was also redeveloped after the earlier implementation showed poor stability and limited practical performance. The final implementation introduced a protected FastAPI runtime, improved segmentation and dish-recognition stages, provisional analysis sessions, user review and correction, and backend-authoritative nutrition finalization.

In addition, previously identified functional and integration issues were corrected across the main health modules. The Flutter interfaces were updated and refined to improve consistency, navigation, readability, and the overall user experience.

Area	Senior Project 2 Work
Unhealthy Habits	Added cessation plans, baselines, reminders, logs, and progress workflows
Profile and Settings	Expanded profile editing and self-management options
Notifications	Replaced the frontend-only implementation with a complete backend-to-device flow
AI Food Analysis	Redeveloped the analysis pipeline, integration, review, and finalization process
Existing Modules	Corrected previously identified functional and integration problems
User Interface	Updated and refined the application screens and visual consistency
System Integration	Improved coordination between frontend, backend, database, and supporting services
Area	Senior Project 2 Work
Unhealthy Habits	Added cessation plans, baselines, reminders, logs, and progress-control workflows
Profile and Settings	Expanded profile editing and user self-management settings
Notifications	Replaced the frontend-only approach with a complete backend-to-device

	notification flow
AI Food Analysis	Redeveloped the analysis runtime, integration flow, review process, and nutrition finalization
Existing Modules	Corrected previously identified functional and integration issues
User Interface	Updated and refined the application screens and shared visual experience
System Integration	Improved coordination between frontend, backend, database, trackers, and supporting services

Table 28 :Main Senior Project 2 Additions and Refinements

6.11 Implementation Challenges and Solutions

These changes transformed the initial implementation baseline into a more integrated and stable final system while preserving Django as the authoritative layer for business logic and persistent health data.

These changes transformed the initial implementation baseline into a more integrated and stable final system while preserving Django as the authoritative layer for business logic and persistent data.

Challenge	Applied Solution	Remaining Limitation
Inconsistent health constraints and stale derived state	Introduced an effective-constraint reader, synchronized recomputation, and materialized health-state updates	Full parity with the previous constraint logic still requires continued verification
AI runtime instability and long processing time	Isolated image processing, reused loaded models, added runtime guards, reduced unnecessary stages, and separated semantic readiness from weight readiness	Ingredient recognition and automatic weight estimation remain review-assisted rather than fully automatic
Unreliable final nutrition from raw AI predictions	Kept Django and the canonical nutrition catalog authoritative and required user confirmation before finalization	Final accuracy still depends on supported catalog coverage and confirmed quantities
Frontend-only reminder design	Replaced it with backend planning, device registration, preference storage, synchronization, local Android scheduling, and event reporting	Delivery still depends on the registered device and local operating-system scheduling
Mobile end-to-end testing complexity	Replaced the earlier browser-oriented approach with Flutter integration_test on an Android emulator connected to a seeded backend	The test flow depends on emulator availability and prepared backend state
Cross-module consistency	Centralized domain services, projections, constraints, and unified read models	Further regression testing is required as health rules evolve

Table 29 : Main Implementation Challenges and Applied Solutions

The most significant AI improvement was operational stability. The earliest baseline completed only 16/35 images and failed on 19/35, whereas the latest evaluated pipeline completed 35/35 without runtime failure and reduced the average analysis time from 123.40 seconds to 6.81 seconds. Nevertheless, uncertain dish, ingredient, and weight results continue to require user review rather than unrestricted automatic finalization.

Health-state consistency was improved by replacing fragile fallback behavior with explicit effective-constraint reading and synchronized recomputation. The notification feature was also redesigned as a complete backend-to-device flow, while mobile integration testing was moved to Flutter's native Android-emulator tooling. These changes addressed the main implementation risks while preserving clear limitations where the current evidence does not support stronger claims.

6.12 Chapter Summary

This chapter presented the development environment and software implementation of VitaMate. It described the verified tools, frameworks, database environment, dependency files, source-control practices, continuous-integration workflow, and testing environments used across the project.

The chapter also documented the implementation of the principal software components, including the layered Django backend, Flutter mobile client, REST APIs, data-access and business-logic patterns, notification subsystem, and AI-assisted meal-analysis service.

Senior Project 2 expanded the original implementation baseline through unhealthy-habit support, profile and settings improvements, a complete backend-driven notification workflow, substantial AI-pipeline redevelopment, correction of previously identified issues, and refinement of the application interfaces.

The final implementation follows a review-assisted and backend-authoritative approach for sensitive health and nutrition operations. Remaining limitations, especially in fully automatic ingredient recognition, weight estimation, deployment evidence, and some evolving health-rule behavior, were explicitly preserved rather than overstated.

The next chapter evaluates the implemented system through backend, frontend, integration, performance, usability, and AI-related testing evidence.

Chapter 7: Testing and Evaluation

This chapter presents the testing and evaluation evidence for the final VitaMate implementation. It covers the executed test plan, scope, environments, backend and frontend results, integration testing, performance observations, security checks, quality metrics, acceptance status, and remaining defects. Only verified results are reported.

7.1 Test Plan

The VitaMate test plan was designed to verify the main backend, mobile, database, notification, and AI-integration functions before final submission. Testing was performed during the final implementation stage and included automated local execution and GitHub Actions validation.

The backend was tested using Django and Django REST Framework test classes. The Flutter client was tested through unit, widget, golden, compact-screen, and integration tests. Additional checks covered database migrations, API contracts, user-data isolation, reference-data stability, notification integration, and the AI-service contract.

A test was considered successful when the implemented behavior matched the expected result, no unhandled exception occurred, and the corresponding automated test completed with a passing status. Test logs and coverage artifacts were retained as execution evidence.

Test Area	Main Tools	Primary Objective
Backend	Django TestCase, TransactionTestCase, DRF APITestCase	Verify services, APIs, permissions, and database behavior
Frontend	flutter test, widget and golden tests	Verify controllers, screens, validation, and UI behavior
Integration	Flutter integration_test, Android emulator	Verify complete mobile-to-backend workflows
Database	Django migrations and PostgreSQL	Verify schema consistency and reference-data stability
Security	Permission tests and Gitleaks	Verify user isolation and detect exposed secrets
AI Integration	Mocked contracts, runtime smoke tests, and pipeline evaluation	Verify service communication and analysis behavior
CI	GitHub Actions	Re-run quality checks and automated tests consistently

Table 30 :Test Plan Summary

7.2 Test Scope

Testing covered the final VitaMate functions that had executable evidence in the backend, Flutter client, integration workflow, and AI pipeline. The scope included functional behavior, API contracts, data isolation, database stability, interface behavior, and selected performance and security checks

Scope Area	Status	Covered Items
Authentication and Profile	Included	Login, JWT access, profile retrieval, and user-specific data access
Nutrition and Hydration	Included	Meal logging, food search, hydration creation, update, and deletion
Activity, Sleep, and Medication	Included	Activity sessions, steps, sleep-coach access, medication flows, and related API behavior

Chronic Conditions and Constraints	Included	Condition data, restrictions, history context, reference-data stability, and cross-user isolation
Habits, Motivation, and Notifications	Included	Habit workflows, motivation behavior, device registration, preferences, and notification-plan integration
AI Meal Analysis	Included	Analysis request, provisional result, confirmation, finalization, runtime stability, and measured pipeline results
Frontend UI Behavior	Included	Widget behavior, modal compatibility, compact-screen behavior, and selected golden tests
CI Validation	Included	Backend tests, Flutter tests, coverage generation, and artifact upload
Chronic Flutter E2E in CI	Partially Included	The smoke integration path is active, while the chronic end-to-end path remains disabled
Production Deployment Testing	Excluded	No verified production or staging deployment test evidence was available
Large-Scale Load Testing	Excluded	No complete high-concurrency backend load test was documented
Formal Clinical Validation	Excluded	The system was not evaluated as a medically validated diagnostic product
Formal User Study	Excluded unless documented	No usability claims should be made without real participant evidence

Table 31 :Included and Excluded Test Scope

The scope therefore focuses on reproducible software evidence rather than unsupported claims. Areas without complete execution records, including production deployment, broad load testing, formal clinical validation, and the gated chronic Flutter integration path, are reported as limitations.

7.3 Test Strategy and Types

VitaMate followed a layered testing strategy in which individual components were verified first, followed by API and database interactions, mobile-interface behavior, and selected end-to-end workflows. Regression testing was repeated after major fixes to confirm that previously working functions remained stable.

Test Type	Purpose	VitaMate Application
Unit Testing	Verify isolated functions and services	Backend services, validators, helpers, and frontend controllers
API Testing	Verify request validation, responses, permissions, and status codes	Authentication, health modules, history, notifications, and AI-meal endpoints
Database Testing	Verify persistence, migrations, queries, and reference data	PostgreSQL models, ownership filters, migrations, and seeded reference data
Widget Testing	Verify Flutter components and screen behavior	Forms, dialogs, loading states, validation, and navigation behavior

Golden and Layout Testing	Detect visual regressions and overflow problems	Selected screens and compact-device layouts
Integration Testing	Verify communication between multiple components	Flutter client, Django APIs, PostgreSQL, notifications, and AI services
End-to-End Smoke Testing	Verify a complete representative user flow	Mobile application startup, authentication, backend connection, and main workflow
Regression Testing	Recheck corrected defects and contracts	History endpoints, hydration flows, chronic-condition data, and frontend test harness
Security Testing	Verify access control and exposed-secret risks	JWT permissions, cross-user isolation, and Gitleaks scanning
Performance Testing	Measure selected runtime behavior	AI pipeline completion and analysis time
Continuous-Integration Testing	Re-run automated checks in a consistent environment	GitHub Actions backend, frontend, coverage, and integration jobs

Table 32 :Executed Test Types

The strategy prioritized automated and reproducible tests. Manual verification was used only where visual or device-specific behavior could not be fully represented by isolated automated tests. Formal usability testing and large-scale load testing were not classified as completed because sufficient execution evidence was not available.

7.4 Test Environment

Testing was performed across local development and continuous-integration environments. The backend tests used the Django test runner with PostgreSQL-compatible application settings, while the Flutter tests were executed through the standard Flutter testing tools. Integration tests used an Android emulator connected to a seeded Django backend.

The test environment also included reusable seed commands and reference-data initialization to keep database-dependent scenarios stable after resets. GitHub Actions reproduced the main backend and frontend checks and stored test logs and coverage artifacts.

Environment Item	Configuration
Local Host	Windows development machine
Backend Runtime	Python virtual environment with Django and Django REST Framework
Frontend Runtime	Flutter SDK and Dart test tools
Database	PostgreSQL
Mobile Runtime	Android emulator
Backend Test Command	python manage.py test ... --keepdb --noinput
Frontend Test Command	flutter test
Frontend Coverage Command	flutter test --coverage
Integration Data	Seeded test user and reference data
CI Platform	GitHub Actions on Ubuntu runners

Evidence	Saved logs, coverage reports, and uploaded CI artifacts
-----------------	---

Table 33 :Test Environment

The reference-data bootstrap was executed after migrations and from shared test helpers to prevent chronic-condition and food-category tests from failing after database flush operations. This made repeated backend execution more consistent.

The CI workflow executed the backend application suites and the Flutter coverage command, then uploaded backend logs, coverage files, and the Flutter lcov.info artifact.

7.5 Test Cases and Execution Results

Representative test cases were selected from the automated backend, frontend, and integration suites. The table summarizes major functional and regression scenarios rather than listing every individual automated test.

ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	Valid user login	JWT tokens are returned	Authentication succeeded	Passed
TC-02	Cross-user data access	Private records are not exposed	Access was prevented	Passed
TC-03	Hydration log update and deletion	Record is updated and removed correctly	Current API contract passed	Passed
TC-04	Seven-day progress history	A complete seven-day response is returned	Both history endpoints returned seven days	Passed
TC-05	Expanded backend test suite	All selected backend tests pass	240 tests completed with OK	Passed
TC-06	Chronic Flutter E2E in CI	Full chronic flow runs on emulator	Test remained disabled in CI	Not Executed

Table 34 :Representative Test Cases and Results

The selected cases summarize the main functional, regression, privacy, and integration evidence. Detailed backend, frontend, and integration results are presented in the following sections.

7.6 Backend Testing

Backend testing verified the main Django applications, API contracts, permissions, database behavior, and regression fixes. The final expanded suite included the `users`, `core`, `gamification`, `manager`, `notification_hub`, and `ai_meals` applications.

The suite executed 240 tests in 1110.208 seconds and completed with the result OK. Targeted regression tests also confirmed the corrected seven-day history responses, chronic-condition

Item	Result
Test command	flutter test
Final result	All tests passed
Line coverage	53.26%
Covered lines	11,160

reference-data stability, nutrition search behavior, seed commands, and cross-user data isolation.

Item	Result
Executed tests	240
Final status	Passed
Test-runner result	OK
Covered applications	6
Main verified areas	APIs, services, permissions, database, regression

Table 35 : Backend Test Summary

The backend suite therefore provides direct evidence that the principal server-side functions and corrected defects operated successfully in the final local test run.

7.7 Frontend Testing

Frontend testing verified the main Flutter controllers, widgets, navigation flows, form validation, dialogs, loading states, and responsive layouts. The test harness was updated to match the current hydration, activity-session, sleep-coach, and food-search API contracts.

The final Flutter test run completed successfully. Coverage execution reported 53.26% line coverage, with 11,160 covered lines out of 20,955.

Total measured lines	20,955
-----------------------------	--------

Table 36 :Frontend Test Summary

The frontend results confirm that the tested interface and controller behaviors matched the current application contracts.

7.8 Integration and End-to-End Testing

Integration testing verified communication between the Flutter client, Django APIs, PostgreSQL database, and supporting services. The active smoke workflow used an Android emulator and seeded backend data to confirm that the mobile application could connect to the backend and complete a representative user flow.

The chronic-condition end-to-end path was not counted as completed because its CI flag remained disabled at the time of evaluation.

Flow	Status
Mobile-to-backend smoke flow	Passed
API and database interaction	Passed
Seeded test-user workflow	Passed
Chronic Flutter E2E in CI	Not Executed

Table 37: Integration Test Status

The available evidence supports the main integration path, while the chronic end-to-end flow remains an open testing item.

7.9 Performance and Load Testing

Performance evaluation focused on the AI food-analysis pipeline because it had verified baseline and final measurements. The average analysis time decreased from 123.40 seconds to 6.81 seconds, while completed analyses increased from 16/35 to 35/35.

A complete large-scale backend load test was not documented; therefore, no unsupported throughput or concurrency claims are reported.

Metric	Old Baseline	Final Result
Completed analyses	16/35	35/35
Runtime failures	19/35	0/35
Average analysis time	123.40 s	6.81 s

Table 38 :Verified Performance Results

The measured results demonstrate a substantial improvement in AI runtime stability and response time, while broader load testing remains outside the verified evidence.

7.10 Security and Privacy Testing

Security testing focused on authentication, authorization, user-data isolation, and exposed-secret detection. Protected APIs required JWT authentication, and permission tests verified that one user could not access another user’s private health records.

Repository scanning was also included in the CI workflow to detect accidentally committed secrets. No penetration test or formal external security audit was performed; therefore, the reported results are limited to the implemented automated checks.

Check	Result
JWT-protected API access	Passed
Cross-user data isolation	Passed
Permission regression tests	Passed
Secret scanning	Included in CI
Penetration testing	Not Performed
External security audit	Not Performed

Table 39 :Security and Privacy Test Summary

7.11 Usability and User Evaluation

A formal usability study with external participants was not conducted. Interface quality was evaluated through Flutter widget, golden, modal-compatibility, and compact-screen tests; however, these automated checks are not presented as a substitute for real-user evaluation.

7.12 Quality Metrics

The final quality metrics were derived from verified backend, frontend, integration, and AI test evidence.

Metric	Final Result
Backend tests	240 passed
Backend suite status	OK
Flutter test status	All tests passed
Flutter line coverage	53.26%
Covered Flutter lines	11,160 / 20,955
AI pipeline completion	35/35
AI runtime failures	0/35
Average AI analysis time	6.81 s
Chronic Flutter E2E in CI	Not executed

Table 40 :Verified Quality Metrics

These metrics confirm strong automated-test stability across the main implementation layers, while the disabled chronic end-to-end path remains a documented limitation.

7.13 Acceptance Criteria and Remaining Validation Items

The verified results demonstrate that the principal backend, frontend, privacy, integration, and AI-runtime acceptance criteria were achieved. Additional validation activities were identified for workflows that were not part of the final automated execution evidence.

Acceptance Criterion	Status
Main backend modules pass automated tests	Achieved
Flutter automated tests complete successfully	Achieved
User-owned health data remains isolated	Achieved
Main mobile-to-backend integration path operates	Achieved
AI pipeline completes the evaluated image set	Achieved
Chronic Flutter end-to-end execution in CI	Planned for Further Validation
Large-scale backend load evaluation	Future Validation
Formal participant-based usability evaluation	Future Validation

Table 41 :Acceptance Status

Additional Validation Items

The chronic-condition Flutter integration workflow remained gated pending final CI emulator validation. Large-scale load evaluation, production-environment testing, and participant-based usability evaluation were not included in the final automated test evidence.

For AI-assisted meal analysis, user confirmation and catalog-based finalization remain intentional reliability mechanisms for uncertain ingredient or weight predictions.

7.14 Chapter Summary

Additional validation remains recommended for the chronic Flutter integration path, large-scale load behavior, formal user evaluation, and advanced AI automation.

Chapter 8: Results and Analysis

This chapter presents the verified final results of VitaMate and analyzes them against the project objectives and measured indicators. It summarizes the completed system functions, testing outcomes, AI food-analysis results, strengths, weaknesses, and remaining limitations without repeating the implementation details of previous chapters.

8.1 Final System Results

The final VitaMate system provides an integrated mobile environment for personal health and lifestyle tracking. The implemented version combines authentication and profile management with nutrition, hydration, activity, steps, sleep, chronic-condition, medication, unhealthy-habit, motivation, notification, and AI-assisted meal-analysis functions.

The Flutter application communicates with the Django backend through authenticated REST APIs, while PostgreSQL stores user and health data. Notifications are generated through a coordinated backend-to-device workflow, and meal-image analysis follows an analyze–confirm–finalize process before nutritional data is stored.

System Area	Final Result
Account and Profile	Authentication, onboarding, profile editing, and settings
Health Tracking	Nutrition, hydration, activity, steps, sleep, and progress history
Health Management	Chronic conditions, medications, and health constraints
Behavior Support	Unhealthy-habit plans, daily logs, progress, and motivation
Notifications	Backend planning, preferences, synchronization, and Android delivery
AI Meal Analysis	Image analysis, ranked suggestions, user confirmation, and nutrition finalization
Quality Assurance	Automated backend, Flutter, integration, security, and regression checks

Table 42 :Final Implemented System Capabilities

Automated evaluation reported 240 passing backend tests, successful Flutter tests with 53.26% line coverage, and stable execution of the main mobile-to-backend workflow.

The AI pipeline completed all 35/35 evaluated images without runtime failure, compared with 16/35 in the oldest baseline, while average processing time decreased from 123.40 seconds to 6.81 seconds.

8.2 Comparative Analysis with Existing Systems

VitaMate was compared with MyFitnessPal, Samsung Health, Medisafe, and Google Health using the verified functions identified in Chapter 3. The comparison focuses on scope and integration rather than claiming superiority over specialized products.

Application	Main Focus	Main Strengths	Difference from VitaMate
MyFitnessPal	Nutrition and fitness	Food logging, nutrient tracking, exercise records, and progress monitoring	Does not combine medication support and chronic-condition constraints within the same coordinated workflow
Samsung Health	General health and wellness	Activity, sleep, nutrition, stress, and health-indicator monitoring	Does not provide VitaMate's centralized coordination of conditions, goals, medications, and tracker rules
Medisafe	Medication management	Medication schedules, reminders, refill alerts, and adherence support	Does not integrate nutrition, hydration, activity, sleep, and AI-assisted food analysis
Google Health	Activity, sleep, and connected health data	Health-data organization, activity tracking, sleep insights, and personalized coaching	Provides less integration with medication management, nutrition analysis, and health constraints
VitaMate	Integrated lifestyle and health support	Multi-domain tracking, medications, chronic conditions, constraints, motivation, notifications, and review-assisted food analysis	Broader functional integration, but with less external validation and product maturity than established applications

Table 43 :Comparative Analysis of VitaMate and Existing Applications

The comparison indicates that the reviewed applications provide strong capabilities within their principal domains. VitaMate’s main contribution is the integration of lifestyle tracking, medication support, chronic-condition constraints, behavioral motivation, notifications, and reviewable AI food analysis within one architecture.

This contribution is integrative rather than clinical. VitaMate does not replace specialized medical platforms and has not undergone the scale of deployment or external validation available to established commercial applications.

8.3 Achievement of Project Objectives

VitaMate achieved the main objective of developing an integrated mobile application for lifestyle and health tracking. The final system combines multiple health domains within one architecture and supports daily monitoring, reminders, progress review, and personalized health-related coordination.

The achievement status of the specific objectives was assessed using implemented functions and verified testing evidence rather than subjective percentages.

Project Objective	Evidence of Achievement	Status
Provide an integrated health and lifestyle platform	Nutrition, hydration, activity, sleep, medication, chronic-condition, habit, and profile modules were integrated	Achieved
Support daily health tracking	Logging, progress history, goals, and dashboard functions were implemented	Achieved
Coordinate health conditions and personal goals	The Constraints Layer connects conditions, restrictions, goals, and tracker behavior	Achieved
Support medication and unhealthy-habit management	Medication schedules, reminders, habit plans, daily logs, and progress tracking were implemented	Achieved
Provide reminders and motivational feedback	Backend-planned notifications and motivation functions were integrated	Achieved
Support AI-assisted food	Meal-image analysis, ranked suggestions, user	Achieved

analysis	confirmation, and nutrition finalization were implemented	with User Review
Provide reports and progress indicators	Home summaries, historical data, and health progress views were implemented	Achieved
Deliver a maintainable and testable system	Layered architecture, automated tests, CI, and coverage artifacts were established	Achieved

Table 44 :Achievement of Project Objectives

The results show that the principal functional and engineering objectives were achieved. AI-assisted analysis remains intentionally review-based, and the project does not claim clinical effectiveness or fully autonomous nutritional assessment.

8.4 Key Performance Indicators (KPIs)

The final KPIs were selected from verified testing and AI-evaluation evidence. They reflect system stability, test coverage, AI completion, and processing performance.

KPI	Final Result
Backend automated tests	240 passed
Flutter test status	All tests passed
Flutter line coverage	53.26%
AI pipeline completion rate	100% (35/35)
AI runtime failure rate	0% (0/35)
Average AI analysis time	6.81 s
Mean segmentation IoU	90.74%
Dish Top-3 accuracy	57.14%
Finalize mapping success	91.43%

Table 45 :summarizes the verified final performance and testing indicators of VitaMate

The results indicate stable system execution and improved AI performance, while user confirmation remains necessary for uncertain AI outputs.

8.5 AI Food-Analysis Results

The final AI food-analysis pipeline was evaluated on a 35-image release set covering completion, segmentation, dish recognition, ingredient support, weight estimation, processing time, and nutrition finalization.

The pipeline completed all 35/35 images without runtime failure, compared with 16/35 in the oldest baseline. Average processing time decreased from 123.40 seconds to 6.81 seconds, while 32/35 images reached a state suitable for user confirmation.

Metric	Final Result
Pipeline completion	35/35
Runtime failures	0/35
Ready for user confirmation	32/35
Average analysis time	6.81 s
Mean segmentation IoU	90.74%
Dish Top-3 accuracy	57.14%
Finalize mapping success	91.43%
Outside-catalog auto ingredients	0%

Table 46 :Final AI Food-Analysis Results

The results confirm improved pipeline stability and speed, while dish recognition, ingredient estimation, and automatic weight estimation still require user review.

8.6 Strengths and Weaknesses

The final VitaMate system demonstrates strong functional integration, modular architecture, automated-test stability, and a review-assisted AI workflow. Its principal weakness is that several advanced capabilities still depend on limited evaluation data or additional validation.

Strengths	Weaknesses
Integrates multiple health and lifestyle domains in one application	No formal clinical validation
Connects chronic conditions, goals, medications, and trackers through shared constraints	No formal usability study with external participants
Uses a layered and testable backend architecture	Large-scale load testing was not completed
Provides backend-planned notifications with	Chronic Flutter E2E remained gated in CI

local Android delivery	
Includes automated backend and Flutter testing with saved evidence	AI evaluation datasets remain limited
Uses user confirmation before storing uncertain AI results	Ingredient recognition and automatic weight estimation are not fully autonomous
Achieved stable AI execution and improved processing time	Less product maturity and external validation than established commercial systems

Table 47 :Main Strengths and Weaknesses

VitaMate’s strongest contribution is its integrated architecture rather than a single isolated feature. The system combines tracking, medication support, chronic-condition coordination, behavioral support, notifications, and AI-assisted nutrition within one platform.

Its main limitations relate to validation scale rather than core functionality. Broader user, clinical, security, load, and AI evaluation would be required before presenting the system as a mature health product.

8.7 Limitations of the Final Version

The final VitaMate version has several limitations that should be considered when interpreting the results. These limitations do not prevent the main system functions from operating, but they restrict the strength of broader technical and health-related claims.

Area	Limitation
Clinical Validation	The system was not clinically validated and should not be treated as a diagnostic or treatment tool
User Evaluation	No formal usability study with external participants was conducted
Performance Evaluation	Large-scale backend load and production-environment testing were not completed
Mobile Integration	The chronic Flutter end-to-end path remained gated in CI

AI Dataset	Model evaluation used limited test sets
Dish Recognition	Top-3 accuracy remained below the target level
Ingredient Recognition	The ingredient model was not accepted as a fully automatic decision source
Weight Estimation	Automatic weight estimation remained optional and required manual fallback
Deployment	Final production deployment evidence was not available

Table 48 :Final-Version Limitations

The AI subsystem is therefore limited to a closed set of supported foods and a review-assisted workflow. User confirmation remains necessary before uncertain dish, ingredient, or quantity estimates affect final nutrition records.

Additional validation is recommended through larger datasets, broader device and load testing, formal user studies, and clinical review of health-related rules.

8.8 Chapter Summary

This chapter presented the verified final results of VitaMate and analyzed them against the project objectives, reference applications, and measured performance indicators.

The final system achieved the principal functional objectives by integrating health tracking, medication support, chronic-condition constraints, behavioral support, notifications, and review-assisted food analysis within one application. Automated testing also confirmed stable backend and Flutter execution.

The AI pipeline showed its strongest improvement in runtime stability and processing speed, completing all 35/35 evaluated images without failure and reducing the average analysis time to 6.81 seconds. User confirmation remains necessary for uncertain dish, ingredient, and weight results.

The results demonstrate a complete and technically stable graduation-project implementation, while broader clinical, usability, load, deployment, and AI validation remain recommended for future development.

Conclusion

VitaMate was successfully developed as an integrated mobile application that combines nutrition, hydration, activity, sleep, medications, habits, goals, chronic-condition support, notifications, reports, and AI-assisted food analysis. Its main contribution is coordinating these services through a modular architecture and centralized Constraints Layer.

The final system demonstrated stable frontend and backend integration, successful testing, and improved AI pipeline performance. However, some AI outputs still require user review, and the application remains a lifestyle-support tool rather than a medical diagnostic system.

Main Achievements

The project delivered a functional Flutter application, Django REST backend, centralized database, integrated health-tracking modules, intelligent food-analysis workflow, and documented testing and evaluation.

Challenges and Lessons Learned

The main challenges involved module integration, AI reliability, device permissions, limited development time, and managing a broad scope with a two-member team. The project highlighted the importance of modular design, clear API contracts, continuous testing, and user confirmation for uncertain AI results.

Recommendations and Future Work

Future work should improve AI recognition and weight estimation, expand usability and user testing, increase mobile test coverage, support more wearable devices, and strengthen personalized and chronic-condition-aware guidance.

References

- [1] World Health Organization, *Classification of Digital Interventions, Services and Applications in Health: A Shared Language to Describe the Uses of Digital Technology for Health*, 2nd ed. Geneva, Switzerland: World Health Organization, 2023.
- [2] R. Berry, A. Kassavou, and S. Sutton, "Does self-monitoring diet and physical activity behaviors using digital technology support adults with obesity or overweight to lose weight? A systematic literature review with meta-analysis," *Obesity Reviews*, vol. 22, no. 10, Art. no. e13306, Oct. 2021, doi: 10.1111/obr.13306.
- [3] M. Aguiar, M. Trujillo, D. Chaves, R. Álvarez, and G. Epelde, "mHealth apps using behavior change techniques to self-report data: Systematic review," *JMIR mHealth and uHealth*, vol. 10, no. 9, Art. no. e33247, Sep. 2022, doi: 10.2196/33247.
- [4] R. Kaur, R. Kumar, and M. Gupta, "Deep neural network for food image classification and nutrient identification: A systematic review," *Reviews in Endocrine and Metabolic Disorders*, vol. 24, no. 4, pp. 633–653, Aug. 2023, doi: 10.1007/s11154-023-09795-4.
- [5] World Health Organization, "Digital health research: Background," *Global Observatory on Health Research and Development*. [Online]. Available: <https://www.who.int/observatories/global-observatory-on-health-research-and-development/analyses-and-syntheses/digital-health/background>. [Accessed: Jul. 31, 2026].
- [6] H. L. Tong, J. C. Quiroz, A. B. Kocaballi, S. Chan Moi Fat, K. P. Dao, H. Gehringer, C. K. Chow, and L. Laranjo, "Personalized mobile technologies for lifestyle behavior change: A systematic review, meta-analysis, and meta-regression," *Preventive Medicine*, vol. 148, Art. no. 106532, Jul. 2021, doi: 10.1016/j.ypmed.2021.106532.
- [7] R. Jakob, S. Harperink, A. M. Rudolf, E. Fleisch, S. Haug, J. L. Mair, A. Salamanca-Sanabria, and T. Kowatsch, "Factors influencing adherence to mHealth apps for prevention or management of noncommunicable diseases: Systematic review," *Journal of Medical Internet Research*, vol. 24, no. 5, Art. no. e35371, May 2022, doi: 10.2196/35371.
- [8] A. M. Belete, B. N. Gameda, T. Y. Akalu, Y. A. Aynalem, and W. S. Shiferaw, "What is the effect of mobile phone text message reminders on medication adherence among adult type 2 diabetes mellitus patients: A systematic review and meta-analysis of randomized controlled trials," *BMC Endocrine Disorders*, vol. 23, no. 1, Art. no. 18, Jan. 2023, doi: 10.1186/s12902-023-01268-8.
- [9] M. Arshed, A. B. Mahmud, H. S. Minhat, L. P. Ying, and M. F. Umer, "Effectiveness of mHealth interventions in medication adherence among patients with cardiovascular diseases: A systematic review," *Diseases*, vol. 11, no. 1, Art. no. 41, Mar. 2023, doi: 10.3390/diseases11010041.
- [10] M. Milne-Ives, S. R. Homer, J. Andrade, and E. Meinert, "Potential associations between behavior change techniques and engagement with mobile health apps: A systematic review," *Frontiers in Psychology*, vol. 14, Art. no. 1227443, Sep. 2023, doi: 10.3389/fpsyg.2023.1227443.

- [11] MyFitnessPal, “Calorie tracker and BMR calculator to reach your goals.” [Online]. Available: <https://www.myfitnesspal.com/>. [Accessed: Jul. 31, 2026].
- [12] Samsung Electronics, “Samsung Health.” [Online]. Available: <https://www.samsung.com/us/apps/samsung-health/>. [Accessed: Jul. 31, 2026].
- [13] Medisafe, “Download the app | Medisafe medication management.” [Online]. Available: <https://medisafe.com/download-the-app>. [Accessed: Jul. 31, 2026].
- [14] Google, “Explore the Google Health app,” *Google Health Help*. [Online]. Available: <https://support.google.com/googlehealth/answer/14237011?hl=en>. [Accessed: Jul. 31, 2026].
- [15] World Health Organization, “Noncommunicable diseases,” *WHO Newsroom*, 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/noncommunicable-diseases>. [Accessed: Jul. 31, 2026].
- [16] World Health Organization, *WHO Guidelines on Physical Activity and Sedentary Behaviour*. Geneva, Switzerland: World Health Organization, 2020. [Online]. Available: <https://www.who.int/publications/i/item/9789240015128>. [Accessed: Jul. 31, 2026].
- [17] American Diabetes Association Professional Practice Committee, “Standards of Care in Diabetes—2026,” *Diabetes Care*, 2026. [Online]. Available: <https://professional.diabetes.org/standards-of-care>. [Accessed: Jul. 31, 2026].
- [18] American Heart Association, American College of Cardiology, and partner organizations, “Guideline for the prevention, detection, evaluation, and management of high blood pressure in adults,” *Circulation*, 2025, doi: 10.1161/CIR.0000000000001356.
- [19] American Heart Association, “How to manage high blood pressure,” 2025. [Online]. Available: <https://www.heart.org/en/health-topics/high-blood-pressure/changes-you-can-make-to-manage-high-blood-pressure>. [Accessed: Jul. 31, 2026].
- [20] World Health Organization, “Sodium reduction,” *WHO Newsroom*, 2026. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/sodium-reduction>. [Accessed: Jul. 31, 2026].
- [21] Centers for Disease Control and Prevention, “About sleep,” 2024. [Online]. Available: <https://www.cdc.gov/sleep/about/index.html>. [Accessed: Jul. 31, 2026].
- [22] National Academies of Sciences, Engineering, and Medicine, “Dietary Reference Intakes collection,” *National Academies Press*. [Online]. Available: <https://nap.nationalacademies.org/collection/57/dietary-reference-intakes>. [Accessed: Jul. 31, 2026].
- [23] National Institutes of Health, Office of Dietary Supplements, “Nutrient recommendations and databases: Dietary Reference Intakes.” [Online]. Available: <https://ods.od.nih.gov/healthinformation/nutrientrecommendations.aspx>. [Accessed: Jul. 31, 2026].
- [24] U.S. Department of Agriculture, “FoodData Central API guide.” [Online]. Available: <https://fdc.nal.usda.gov/api-guide>. [Accessed: Jul. 31, 2026].

- [25] World Health Organization, *Adherence to Long-Term Therapies: Evidence for Action*. Geneva, Switzerland: World Health Organization, 2003. [Online]. Available: <https://iris.who.int/handle/10665/42682>. [Accessed: Jul. 31, 2026].
- [26] U.S. Food and Drug Administration, “Medication adherence: Landscape, strategies, and evaluation methods.” [Online]. Available: <https://www.fda.gov/drugs/news-events-human-drugs/medication-adherence-landscape-strategies-and-evaluation-methods-12102019>. [Accessed: Jul. 31, 2026].
- [27] Q. Thames, A. Karpur, W. Norris, F. Xia, L. Panait, T. Weyand, and J. Sim, “Nutrition5k: Towards automatic nutritional understanding of generic food,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 8903–8911.
- [28] A. Radford, J. W. Kim, C. Hallacy, *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, vol. 139, 2021, pp. 8748–8763.
- [29] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth Anything V2,” in *Advances in Neural Information Processing Systems*, 2024.
- [30] Ultralytics, “YOLO11 documentation,” 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11>. [Accessed: Jul. 31, 2026].